Cuprins

1. INTRODUCERE – STADIUL ACTUAL ȘI PREZENTARE GENERALĂ 4

1.1 Introducere 4

1.2 Descrierea proiectului 4

1.3 Noi propuneri pentru îmbunătățirea informatizării piețelor 4

1.4 Stadiul actual 5

1.4.1 Piata Obor Bucuresti 5

1.4.2 Piata Centrala Craiova 7

2. PRINCIPII ȘI METODE UTILIZATE PENTRU REZOLVAREA TEMEI DE PROIECT 8

2.1 Tehnologii utilizate 8

2.1.1 Java – Limbaj de programare 8

2.1.2 Spring Boot – Framework Java 8

2.1.3 HTML – Hyper Text Markup Language 9

2.1.4 CSS – Cascading Style Sheets 9

2.1.5 Bootstrap – Framework pentru Front-End 9

2.1.6 TypeScript – Limbaj de programare pentru aplicații Web scalabile 10

2.1.7 PostgreSQL – Server de baze de date 10

2.1.8 Angular 19 – Framework pentru Front-End 10

2.1.9 OpenAPI - Unealtă pentru descrierea interfețelor API 11

2.1.10 NGX-Translate – Unealtă pentru suportul multi lingvistic 11

2.1.11 MailDev – Unealtă pentru trimiterea de email-uri 11

2.1.12 Docker 11

2.2 Structura 12

2.2.1 Interfața utilizatorului 12

2.2.2 Interfața administratorului 13

2.2.3 Interfața proprietarului de piață 13

2.2.4 Interfața proprietarului de tarabă 13

2.3 Logica aplicației 14

3. PROIECTAREA DETALIATĂ A APLICAȚIEI INFORMATICE 15

3.1 Crearea unui proiect back-end și configurarea aplicației 15

3.2 Crearea bazei de date MySQL și conectarea la server 16

3.3 Realizarea modelelor pentru baza de date cu Java JPA 17

3.4 Realizarea back-end-ului în Java 21

3.4.1 Arhitectura modulelor 21
3.4.2 Logarea, înregistrarea și activarea contului utilizatorului 22
3.4.2 Cererile pentru roluri 28
3.4.3 Gestionarea piețelor 32
3.4.4 Gestionarea tarabelor din cadrul piețelor 35
3.4.5 Comenzile utilizatorilor 39
3.5 Realizarea front-end-ului în Angular19 ..... **44**
3.5.1 Arhitectura componentelor 44
3.5.2 Implementarea service-urilor 44
3.5.3 Implementarea rutărilor către componente 46
3.5.4 Implementarea uneltei NGX-Translate 47
3.5.5 Componenta Principală 48
3.5.6 Componenta Bara de navigare 49
3.5.7 Componenta Înregistrare 51
3.5.8 Componenta Activare cont 53
3.5.9 Componenta Logare 55
3.5.10 Componenta Profil 57
3.5.11 Componenta Cereri 60
3.5.12 Componenta Piețelor 66
3.5.13 Componenta Tarabelor 70
3.5.14 Componenta Tarabă 76
4. TESTAREA FUNCȚIONALITĂȚII APLICAȚIEI **83**
4.1 Testarea back-end-ului **84**
4.2 Testarea front-end-ului **86**
5. INTERFAȚA CU UTILIZATORUL **86**
5.1 Interfața cu administratorul **90**
5.2 Interfața cu proprietarul de piață **91**
5.3 Interfața cu proprietarul de tarabă **92**
6. PROIECTAREA TEHNOLOGICĂ A REPERULUI „FLANȘĂ DE MONTARE” DIN COMPONENȚA ANSAMBLULUI „POMPĂ CU PINIOANE” **93**
6.1 Date inițiale generale **93**
6.2 Date constructive – funcționale **93**
6.2.1 Schița ansamblului 93
6.2.2 Caracteristici constructive prescrise produsului 94

6.2.3 Caracteristici de material 96
6.2.4 Interdependența toleranțelor dimensionale 99
6.2.5 Analiza reperului 100
6.3 Proiectarea semifabricatului **101**
6.4 Mașini unelte folosite **106**
6.5 Sculele necesare **107**
6.6 Program Fusion 360 .... **109**
6.7 Fabricare reper prin fabricare aditivă **111**
7. CONCLUZII ȘI DEZVOLTĂRI ULTERIOARE **112**
7.1 Concluzii **112**
7.2 Dezvoltări ulterioare **112**
8. BIBLIOGRAFIE **113**

1. INTRODUCERE – STADIUL ACTUAL ȘI PREZENTARE GENERALĂ

1.1 Introducere

Pieţele agroalimentare joacă un rol esenţial în furnizarea de produse proaspete, locale şi de sezon, însă în ultimii ani acestea au pierdut teren în faţa supermakert-urilor şi hypermarket-urilor. Acestea oferă din urmă comoditate şi acces rapid la o gamă largă de produs, însă de multe ori calitatea, gustul şi prospeţimea alimentelor nu se compară cu cele oferite de producătorii locali din cadrul pieţelor.

În acelaşi timp, tinerii care reprezintă o mare parte din consumatori sunt puternic ancoraţi în mediul digital şi îşi doresc soluţii moderne şi rapide pentru orice tip de activitate, inclusiv cumpărăturile. Din acest motiv, digitalizarea unor activităţi din pieţele agroalimentare este o necesitate pentru adaptarea la cerinţele actuale ale consumatorilor.

Această idee a venit în cadrul stagii de practică din anul lII, unde am participat la un stagiu de pregătire pentru dezvoltarea aplicaţiilor Web. Unul dintre fondatorii firmei EvaSTAFF, firmă unde am făcut stagiul de practică, mi-a prezentat idea de digitalizare a unor activităţi din pieţele agroalimentare, iar de atunci mi-am propus să fac acest proiect şi cu ajutorul acestora să îl lansăm publicului larg.

1.2 Descrierea proiectului

Lucrarea urmăreşte realizarea unei apllicaţii care să ofere un mediu unde utilizatorii îşi pot comanda produsele ce le găsesc în pieţele locale fără a se mai deplasa, a vedea informaţii despre toate pieţele şi tarabele din cadrul pieţei înscrise în aplicaţie. Aplicaţia include şi o interfaţă special pentru cererile de colaborare în următoarea ordine: proprietarul de piaţă îşi va înscrie piaţa în aplicaţie, iar apoi în funcţie de răspunsul primit îşi va adăuga proprietarii de tarabe, unde aceştia îşi pot începe activitatea prin adăugarea inventarului de produse.

Interfaţa web este proiectată în jurul acestor roluri: utillizator obişnuit, proprietar piaţă, proprietar tarabă şi administrator, aceştia având interfeţe separate cu funcţionalităţi diferite. Există posibilitatea creării de cont cu ajutorul oricărui mail valid. Contul odată creat va avea acces la completarea unui formular pentru a obţine unul dintre rolurile prezentate, urmând ca cererea să fie evaluată de către un administrator. Aceste cereri apar în interfaţa administratorului, acesta având posibilitatea de a accepta sau respinge, oferind şi un motiv pentru aceste alegeri.

1.3 Noi propuneri pentru îmbunătăţirea informatizării pieţelor

Pieţele agroalimentare se află într-o scădere de popularitate şi vânzări foarte mare pe zi ce trece, iar cu apariţia tot mai multor supermarket-uri şi hypermarket-uri acestea pierd din teren foarte mult. Soluţia de digitalizare va avea un impact social şi economic foarte mare, iar cu promovarea necesară poate pune aceste locuri din nou într-o lumină bună.

Acest lucru va fi posibil doar dacă lumea se va adapta şi va accepta să folosească această soluţie.

1.4 Stadiul actual

În prezent proprietarii de piețe au ales să investească într-o pagină web care să își promoveze activitatea și să își arate produsele locale vândute de fiecare tarabă prezentă în piață. Aceste pagini sunt statice, fără vreun fel de interacțiune cu vânzătorii, având doar un program de funcționare și informații despre comercianții acestora. Câteva exemple relevante sunt prezentate în cele ce urmează.

1.4.1 Piata Obor Bucuresti

Piața Obor București[1] este una dintre cele mai mari piețe agroalimentare din București, iar aceasta este printre puținele care are o pagina web deschisă publicului larg. Aceasta are informații numeroase cu privire la programul de funcționare, o statistică despre punctele de vânzare, tonajul legumelor proaspete ce sunt aduse zilnic, suprafața locului, cât și câte magazine tradiționale există. În plus, pagina web dispune de suport multi lingvistic, ceea ce este un plus pentru turiștii care vin în România.

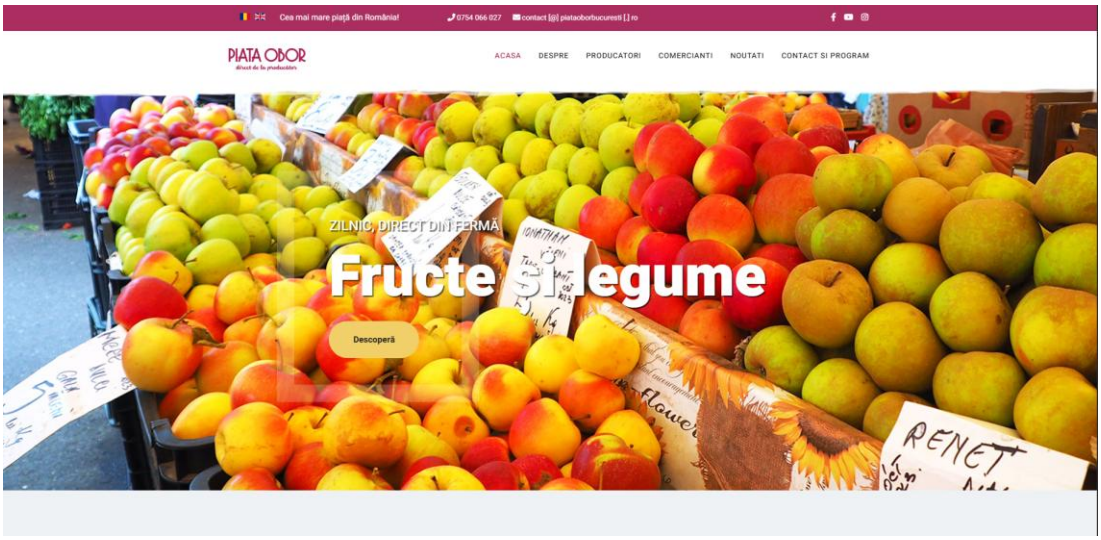


Fig 1.1 Pagina de start piataoborbucuresti.ro[1]

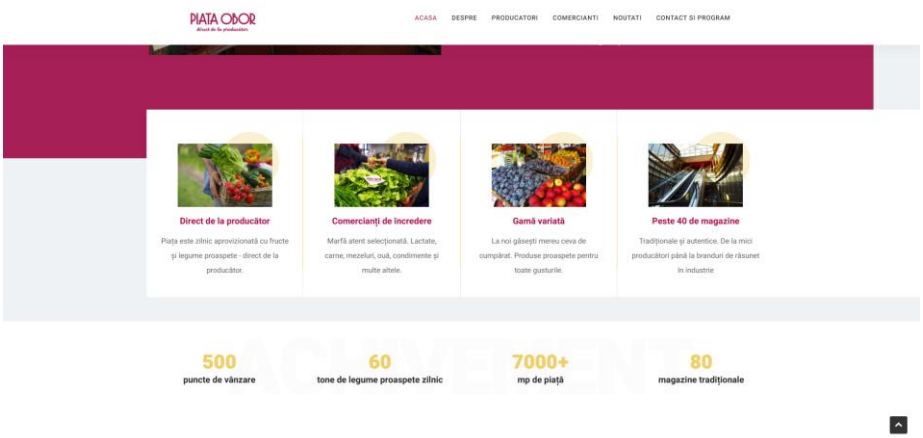


Fig 1.2 Informații utile, partea 1 piataoborbucuresti.ro[1]

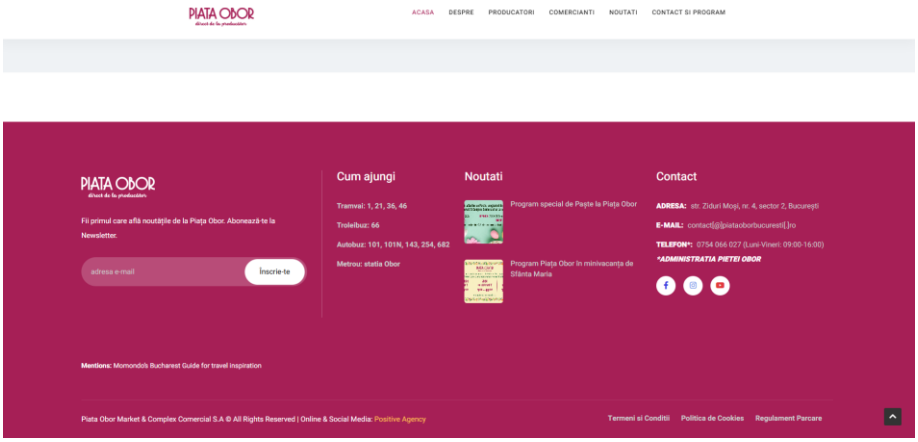


Fig 1.3 Informații utile, partea 2 piataoborbucuresti.ro[1]

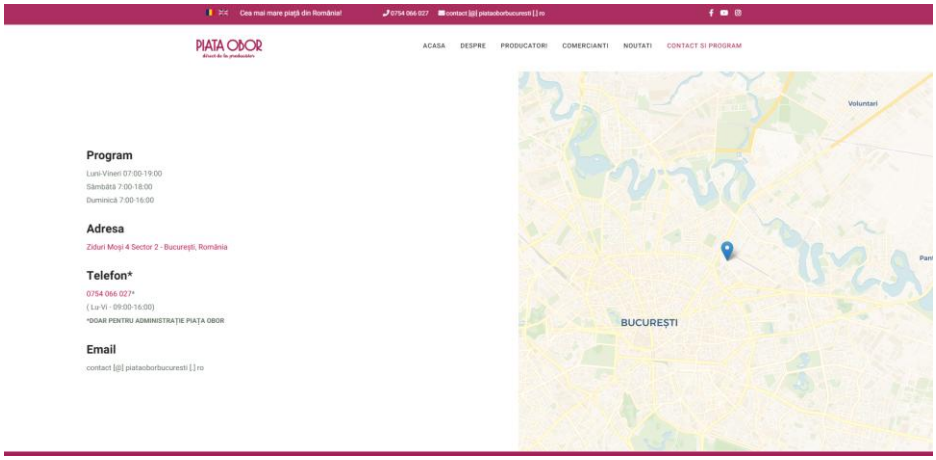


Fig 1.4 Informații utile, partea 3 piataoborbucuresti.ro[1]

1.4.2 Piata Centrala Craiova

Piața centrală Craiova[2] este una dintre cele mai mari piețe din Craiova, iar pagina web este una foarte simplistă, având informații despre programul de funcționare și cum sunt dispuse tarabele de vânzare.



Fig 1.5 Piața Centrala Craiova pietecraiova.ro/piata-centrala[2]

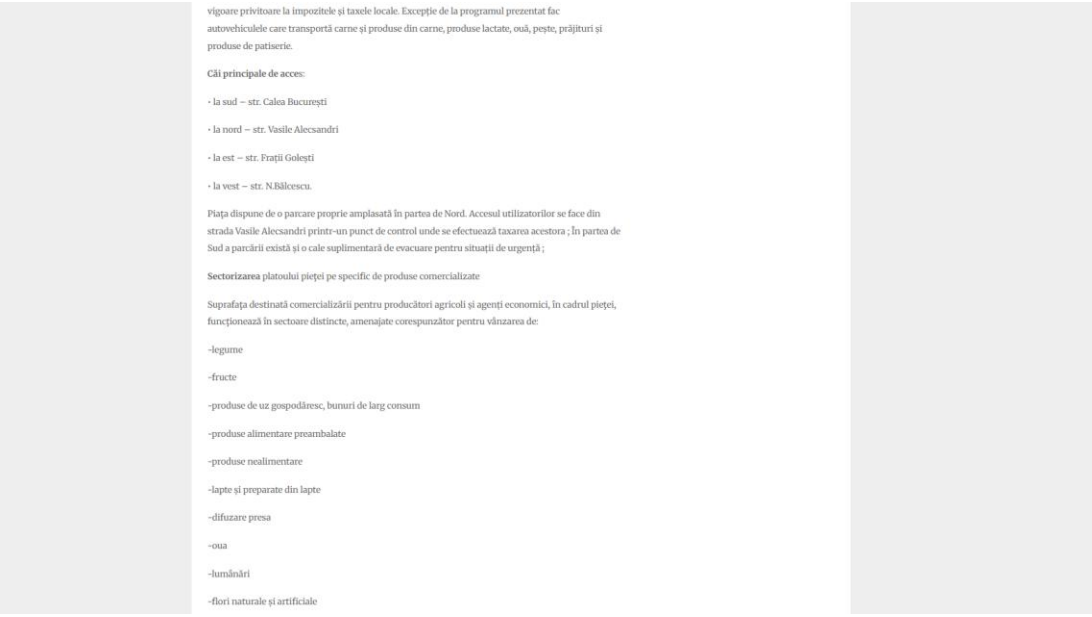


Fig 1.6 Piața Centrala Craiova, informații pietecraiova.ro/piata-centrala[2]

2. PRINCIPII ȘI METODE UTILIZATE PENTRU REZOLVAREA TEMEI DE PROIECT

2.1 Tehnologii utilizate

2.1.1 Java – Limbaj de programare

Java[3] este un limbaj de programare OOP (Orientat Pe Obiecte), cunoscut pentru portabilitate, securitate și performanță. Este un limbaj de programare cu sintaxă riguroasă, dar clară, fiind folosit pe scară largă în dezvoltarea aplicațiilor web. Java facilitează devoltarea aplicațiilor printr-o colecție vastă de biblioteci și framework-uri (precum Spring Boot), care susțin diferite tipuri de proiecte software.

Programarea orientate pe obiecte permite structurarea programului în obiecte, creare de clase cu proprietăți și acțiuni propria. Clasele sunt șablonate pe baza cărora pot fi create obiecte (instanțe). Metodele sunt funcții care există în cadrul unei clase. Acestea conțin atribute (acțiuni) și defines modul în care operează obiectul. Atributele reprezintă parametrii clasei. O instanță este un obiect concret realizat pe baza clasei menționate anterior.

Java suportă modularitatea prin împărțirea aplicației în pachete și clase reutilizabile. De asemenea, include un system robust de tratare a excepțiilor, ceea ce contribuie la o dezvoltare mai sigură. Aceste caracteristici, împreună cu suportul pe termen lung oferit de comunitate și companii mari, au făcut Java ca fiind unul d intre cele mai utilizate și longevive limbaje de programare.

2.1.2 Spring Boot – Framework Java

Spring Boot [4], [5] este un framework Java ce permite dezvoltatorilor să creeze aplicații web rapid și eficient. Acesta este o extensie a ecosistemului Spring și oferă o abordare aparte pentru crearea aplicațiilor, permițând inițializarea rapidă a proiectelor și rularea acestora cu un server incorporat. Spring Boot suportă lucrul cu Spring Data JPA[6] pentru managementul bazelor de date permițând interacțiunea cu baze de date relaționale printr-un strat de abstractizare a operațiilor SQL. De asemenea, rutele HTTP sunt definite ușor, oferind o integritate clară între logica aplicației și arhitectura web.

Spring Data JPA este un modul din ecosistemul Spring care oferă o soluție simplificată pentru accesul și manipularea datelor în aplicații Java. Acest modul permite interacțiunea cu baze de date relaționare printr-o abordare orientate pe obiecte, fără a fi nevoie să scriu interogări SQL manual. Acesta oferă un strat de abstractizare peste implementările JPA (Java Permisive API) care permite crearea rapidă de operații CRUD (Create, Read, Update și Delete).;

Spring Boot a fost dezvoltat de echipa Pivotal (acum parte din Vmware) ca parte din inițiativa de a simplifica configurarea și lansarea aplicațiilor Spring. Este construit pe baza framework-ului Spring și a unor concepte fundamentale precum Inversion of Control (IoC), Dependency Injection și aspecte modulare.

2.1.3 HTML – Hyper Text Markup Language

HTML (Hyper Text Markup Language)[7], [8] este un limbaj marker standard pentru crearea paginilor Web. Prin intermediul acestuia, se pot crea și organiza elemente de tip text, titlu, paragrafe, imagini, link, lista, tabel cu ajutorul etichetelor de tip tags, proces care se numește marcare. Etichetele au nume sugestiv și sunt încadrate între simbolurile “<” “>”. Acestea nu sunt case-sensitive, deci pot fi scrise cu litere mici sau majuscule, fără a fi afectată funcționalitatea. Etichetele sunt poziționate la începutul și la sfârșitul unei secțiuni, cum ar fi <title> Titlu </title>.

Secțiunea head a unui document HTML nu apare pe site-ul propriu-zis atunci când pagina este încărcată, conținând informații precum: titlul (<title>), link-uri către CSS, sau alte elemente utilizate pentru stilizarea paginii Web și date despre documentul HTML.

Secțiunea body a unui document HTML include elemente de tip heading (<heading>), paragraph (<p>), liste (<li>), formulare (<form>), care formează conținutul principal al paginii Web. Acestea pot fi așezate în pagină cu ajutorul containerelor de tip , elemente block non-semantice care se ocupă cu organizarea informațiilor din pagină în secțiuni separate.

2.1.4 CSS – Cascading Style Sheets

CSS (Cascading Style Sheets) [9] oferă posibilitatea de a înfrumusețea paginile Web, oferind control asupra modului în care acestea sunt prezentate către utilizator. În general, paginile Web sunt scrise în limbaje de marcare precum HTML sau XML care apar în browser ca niște documente text, fără o structură, culori sau stiluri de afișare specifice.

CSS este un limbaj bazat pe reguli, ceea ce înseamnă că dezvoltatorul poate crea reguli specificând grupuri de stiluri care vor fi aplicate către anumite elemente ale paginii Web. Acestea sunt definite prin intermediul unui sistem de tipul proprietate-valoare. Se pot aplica mai multe proprietăți în cadrul unui elemtn, care însumate, v or rezulta în stilul de afișare dorit.

2.1.5 Bootstrap – Framework pentru Front-End

Bootstrap[10] este un toolkit pentru Front-End robust și extensibil care permite personalizarea ușoară și rapidă a interfețelor Web folosind SASS (Syntactically Awesome Style Sheets), un sistem de așezare în pagină bazat pe o grilă de plugin-uri JavaScript ce pot fi incluse cu usurință în proiecte pentru a oferi un aspect plăcut și pentru a g enera elemente interactive în cadrul paginii Web, într-un timp scurt.

Printre clasele și elementele disponibile, se numără următoarele: butoane, formulare, elemtene de ti p carusel, dropdown-uri, bară de navigare, card-uri, modale, alerte și elemente ce țin de așezarea în pagină, reprezentate de sistemul grilă, împărțit în linii și coloane.

Bootstrap poate fi instalat prin intermediul npm (Node Package Manager), folosind comenzile “npm install bootstrap@5.3.3”.

2.1.6 TypeScript – Limbaj de programare pentru aplicații Web scalabile

TypeScript [11] este un limbaj de programare OOP (Orientat pe Obiecte) cu suport pentru tipuri statice și sistem de tipare avansat. Acesta a fost dezvoltat de către Microsoft ca o extensie a limbajului JavaScript[12] și este compilat în JavaScript pentru a putea fi rulat în orice browser.

TypeScript este un limbaj transpilat, codul scris în acest limbaj de programare la rulare este convertit în cod de JavaScript. Acesta fiind un limbaj de tip multi-paradigmă, suportând atât programarea orientată pe obiecte, cât și programare modulară.

2.1.7 PostgreSQL – Server de baze de date

PostgreSQL[13], [14] este un server de baze de date relațională de tipul SQL (Structured Query Language), acesta fiind performant, extensibil și stabil, totodată capabil să gestioneze un număr mare de interogări și operații în mod simultan, într-un mediu de multi-utilizatori și multi-thread. Este recunoscut pentru conformitatea sa strictă cu standardele SQL și pentru suportul avansat pentru tranzacții complexe, integritate referențială și proceduri stocate.

PostgreSQL suportă execuția paralelă a operațiilor, ceea ce îl face eficient din punct de vedere al utilizării de resurse, mai ales în aplicațiile de dimensiuni mari. Fiind un sistem open-source, acesta este dezvoltat activ de o comunitate globală și menținut de PostgreSQL Global Development Group. Mascota PostgreSQL este un elefant albastru numit Slonik, simbolizând robustețea și memoria puternică a sistemului.

2.1.8 Angular 19 – Framework pentru Front-End

Angular 19[15] este un framework Front-End de tip open-source pentru dezvoltarea aplicațiilor Web de tip SPA (Single Page Application), oferind suport pentru actualizarea dinamică a conținutului fără reîncărcarea paginii. Acest Framework permite extinderea HTML-ului cu directive proprii și integrarea ușoară a serviciilor pentru manipularea dateelor, oferind o experiență interactivă și flluidă în interfața utilizatorului.

Distribuirea paginii se face în structuri numite componente, acestea sunt o metodă modernă de creare a unei pagini Web din pricina reutilizării, de exemplu putem crea componente ce pot fi refolosite pentru a evita codul duplicat scris, un exemplu foarte comun ar fi o componentă care să conțină un card cu informații despre utilizator: nume, prenume, data nașterii, fotografie profil. Această componentă poate fi folosită în locuri precum: profilul utilizatorului, editarea setărilor de către un administrator sau afișarea utilizatorilor după un anumit criteriu, codul astfel devenind mult mai citibil și ușor de modificat.

Angular a avut de-a lungul timpului numeroase iterații, însă de la Angular 2, framework-ul a fost rescris complet față de versiunea originala AngularJS, adoptând TypeScript ca limbaj principal, o arhitectură bazată pe componente reutilizabile, un sistem modern de modularizare și performanțe îmbunătățite.

2.1.9 OpenAPI - Unealtă pentru descrierea interfețelor API

OpenAPI[16] este o specificație standardizată pentru descrierea interfețelor API accesibile prin HTTP, într-un format lizibil de către Front-End. Acesta permite documentarea clară și structurată a unei interfețe API prin fișiere YAML sau JSON, care conțin informații precum metodele disponibile, parametrii, tipuri de date acceptate și coduri de răspuns.

Integrarea acestuia se poate face folosind Framework-ul Angular și generând astfel un serviciu robust pentru apelurile HTTP către serverul de Back-End, având o structură clară despre tipurile de date ce vor veni ca răspuns de la server.

2.1.10 NGX-Translate – Unealtă pentru suportul multi lingvistic

NGX-Translate[17] este un tool pentru framework-ul Angular ce permite ca pagina să afișeze informații text în funcție de limba selectată. Pentru a face acest lucru folosește citirea din fișiere de tip JSON[18], unde mesajele sunt afișate sub forma unui dicționar, cheia este un cuvânt, iar valoarea cheii este întregul text scris în limba respectivă.

Spre exemplu, aplicația va dispune de două astfel de fișiere, denumite ro.json și en.json. Limba selectată la prima interacțiune va fi limba română, user-ul putând în orice clipă să schimbe acest aspect din bara de navigare ce va fi poziționată în partea de sus a paginii.

Pentru translatarea textelor în funcție de limba selectată se va adăuga pipe-ul tipic Angular numit translate. Pentru a funcționa afișarea corectă în funcție de limba selectată va trebui să precizăm între apostrofuri cheia din dicționarul precizat mai sus, spre exemplu: {{ ‘welcome-text’ | translate }}.

2.1.11 MailDev – Unealtă pentru trimiterea de email-uri

MailDev[19] este un tool ce stimulează un server de emailing. Această unealtă are o interfață web unde sunt afișate mail-uri doar cu scopul de testare. MailDev funcționeazaă ca un server STMP (Simple Mail Transfer Protocol) local, interceptând email-urile trimise de aplicație și afișându-le în interfața web, fără a le trimite către utilizator.

2.1.12 Docker

Docker[20], [21] este o platformă open-source care permite dezvoltatorilor să creeze, distribuie și să ruleze aplicații în containere. Un container reprezintă o unitate standard de software care împachetează codul aplicației împreuna cu toate dependințele necesare pentru ca aceasta să funcționeze. În cazul aplicației, uneltele PostgreSQL și MailDev au fost trecute într-un astfel de container.

În contextul dezvoltării software, Docker este extrem de util pentru a crea un mediu controlat în care pot fi testate și dezvoltate aplicațiile.

2.2 Structura

Aplicația este construită pentru a digitaliza activitățiile predominante într-o piață agroalimentară într-o manieră unde utilizatorului îi va fi ușor să facă orice acțiune, fie că este un simplu cumpărător de produse sau că este comerciant la o tarabă din cadrul pieței. Această aplicație va facilita comunicarea între utilizatori și comercianții din cadrul tarabelor.

Aplicația este bazată pe interacțiunea tuturor părților, administratorii vor fi cei care vor adăuga rolul de proprietar de piață unui utilizator în urmă completării unei cereri realizate după crearea contului. În funcție de răspuns, dacă acesta este pozitiv, administratorul va putea crea piața pe baza informațiilor din cerere. După creare, proprietarul de piață își va ptuea crea tarabe și să le asigneze către utilizatorii care se vor ocupa de acestea. În cadrul unei tarabe trebuie precizat inventarul produselor, acesta poate fi actualizat de către proprietarii tarabelor, iar atunci când se comandă din interiorul aplicației, cantitatea produselor scade în mod automat nefiind nevoie de acțiunea proprietariilor.

În tot acest timp, proprietarii de piețe au rolul de a supraveghea stockul și comenzile proprietarilor de piețe. Aceștia se vor asigura că totul funcționează în parametri normali, iar în cazul unor probleme sau înșelătorii pot contacta un administrator.

2.2.1 Interfața utilizatorului

Utilizatorul are acces la o interfață în cadrul căreia se poate autentifica sau își poate crea un cont. După crearea unui cont și logarea în aplicație acesta poate vizualiza tab-ul de Profil al acestuia, unde sunt afișate informații despre cont extrase din baza de date precum email, image de profil, dacă contul este sau nu activat și informații în legătură cu rolurile acestuia (dacă este administrator, proprietar de piață sau proprietar de tarabă). Pe lângă aceste informații, utilizatorul va vedea și o listă cu toate comenzile sale efectuate în cadrul aplicației.

După ce este existent un cont și utilizatorul este logat acesta poate vedea tab-ul de Cereri unde poate completa o cerere pentru fiecare tip, poate avea o cerere de administrator și una de proprietar de piață în același timp până i se dă un răspuns. În tabul de Cereri acesta va putea vedea în timp real status-ul cererii, cât și informațiile extrase din baza de date cu privire la cerere, precum descrierea, link-urile atașate, data completării și tipul cererii.

Pe lângă tabul de cereri, utilizatorul are acces la navigarea prin tab-ul de Piețe, unde poate vedea toate pieţele înscrise în aplicație, iar dacă dorește poate vedea toate tarabele din cadrul pieței. La navigarea într-una din piețe se pot vedea toate tarabele înscrise din cadrul pieței selectate, iar mai apoi se poate naviga în cadrul tarabei respective, unde se vor putea vizualiza produsele disponibile, cantitatea și prețul per kilogram. După completarea formularului, se va putea plasa comanda, iar din acest pas se ocupă proprietarul de tarabă, utilizatorul fiind nevoit să navigheze către tab-ul de Profil pentru a-și urmări comanda.

Pe lângă aceste funcționalități, orice utilizator poate schimba în timp real limba în care sunt afișate informațiile. În plus acesta poate oricând să se delogheze folosind butonul intuitiv de Delogare.

2.2.2 Interfața administratorului

Rolul de administrator presupune urmărirea și administratrea aplicației pentru o bună experiență a utilizatorilor. Având acest rol vor fi afișate informații în plus față de un utilizator obișnuit.

Administratorul va putea răspunde la cererile făcute de către utilizatori, acesta modificând status-ul cererii în acceptat sau respins adăugând și un motiv pentru alegerea făcută, informații care ulterior vor fi vizualizate și de către utilizatorul care a făcut cererea.

O altă funcționalitate a administratorului este cea de creare de piețe în urma cererilor. Cu informațiile extrase din cererea pentru proprietar de piață se va crea piața din tab-ul de Piețe, iar acolo se va asigna proprietarul, care este utilizatorul ce a făcut cererea.

Funcționalități conexe precum schimbarea parolei, imaginii de profil etc. sunt disponibile doar prin intermediul cererilor, selectând tipul cererii Asistență tehnică.

2.2.3 Interfața proprietarului de piață

Proprietarul de piață este asignat în momentul creări pieței de către un administrator. Din acel moment, utilizatorii cu rolul de proprietari de piețe vor putea crea tarabe în cadrul piețelor acestora. Crearea unei tarabe se face accesând tab-ul de Piețe și navigând către piața care are drept proprietar utilizatorul cu rolul aferent.

În acest tab, proprietarul va putea crea tarabe adăugând informații despre programul de funcționare, o poză a tarabei și produsele disponibile (numele produsului, cantitatea și prețul per kilogram). În acest formular va trebui să adauge și utilizatorul care se va ocupa de tarabă.

2.2.4 Interfața proprietarului de tarabă

Proprietarul de tarabă este asignat de către proprietarul de piață unde i se va adăuga taraba, utilizatorii cu rolul de proprietari de tarabă vor putea modifica inventarul de produse, informații privind numele produselor, cantitatea și prețul per kilogram și adăuga sau șterge produsele. Totodată, aceștia vor putea vedea în timp real toate comenzile din cadrul tarabei și modifica status-ul comenzii.

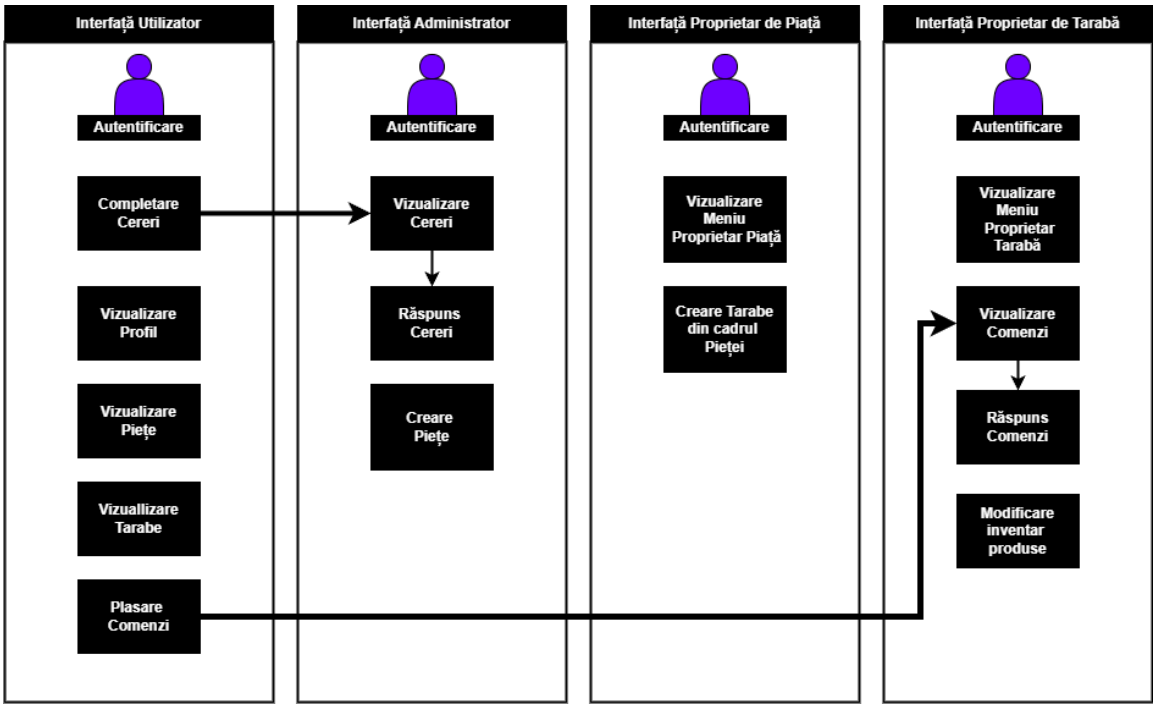


Fig 2.1 Modalitate de lucru a intefețelor

2.3 Logica aplicației

Utilizatorii sunt diferențiați prin roluri, care le oferă informații în plus de vizualizat sau de editat în anumite cazuri. Datele despre utilizator sunt stocate în sesiunea de LocalStorage a browser-ului, aplicația fiind structurată într-o singură pagina HTML, această soluție este cea mai eficientă, iar datele salvate se pot folosi ulterior la operații cu baza de date. Informațiile se rețin in browser cât timp utilizatorul este logat.

La operațiile cu baza de date se foloseste server-ul de Java pentru gestionarea cererilor de tip HTTP (GET, PUT, POST, DELETE). Astfel, datele pot fi transmise mai departe și pot fi efectuate operațiile de tip CRUD (Create, Read, Update, Delete) de PostgreSQL.

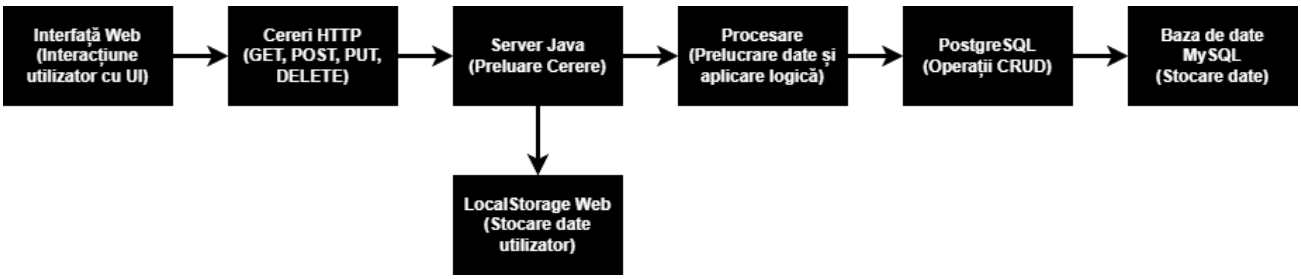


Fig 2. 2 Diagrama flux a aplicației

3. PROIECTAREA DETALIATĂ A APLICAȚIEI INFORMATICE

3.1 Crearea unui proiect back-end și configurarea aplicației

Pentru realizarea acestei aplicații, a fost necesară instalarea Java și MySQL. Au fost integrate ulterior și celelalte tehnologii și dependințe necesare în cadrul aplicației Java, precum PostgreSQL, Angular 19, OpenApi și Boostrap. Editorul de cod pentru aplicația de back-end este IntelliJ, care a oferit și vizibilitate asupra datelor din baza de date.

Pentru început s-a lansat editorul IntelliJ și au fost configurate setările proiectului, urmat de alegerea versiunii de Java folosită în cadrul aplicației și atribuirea unei locații a proiectului.

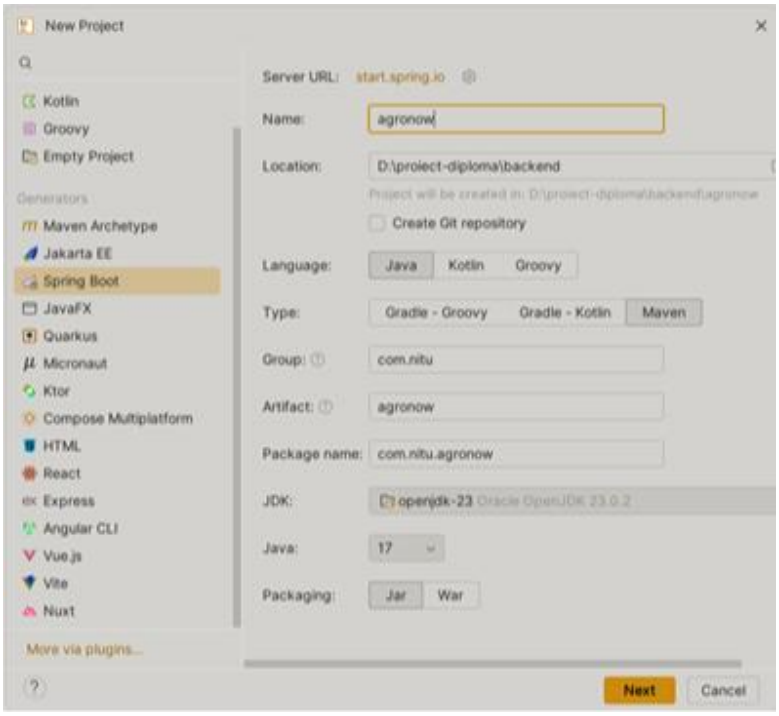
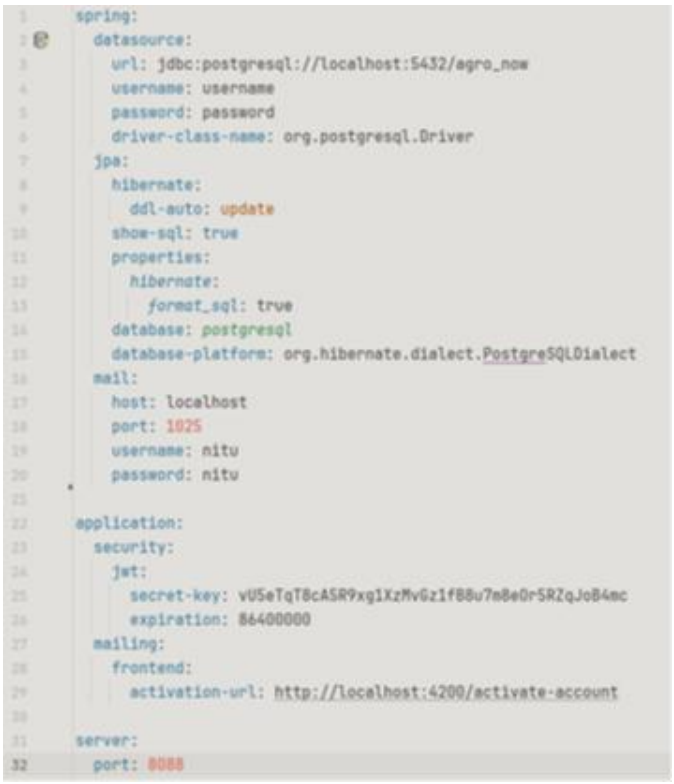


Fig 3. 1 Crearea aplicației „back-end”

Conform fig. 3.2, configurarea este un fișier de tipul YAML (YML), unde este configurată baza de date local, cât și server-ul de trimitere și vizionare de mail-uri, numit „MailDev”. Acesta conține informații despre datele de conectare către baza de date, cât și informații despre port-urile la care vor funcționa serviciile. Server-ul de back-end va funcționa pe portul 8080, iar server-ul de mail-ing pe portul 1025.



```yaml
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/agro_now
    username: username
    password: password
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        format_sql: true
    database: postgresql
    database-platform: org.hibernate.dialect.PostgreSQLDialect
  mail:
    host: localhost
    port: 1025
    username: nitu
    password: nitu

application:
  security:
    jwt:
      secret-key: vU5eTqT8cA5R9xg1XzMvGz1fB8u7m8eOr5RZqJoB4mc
      expiration: 86400000
  mailing:
    frontend:
      activation-url: http://localhost:4200/activate-account

server:
  port: 8088
```

Fig 3. 2 Configurarea fișierului YML

3.2 Crearea bazei de date MySQL și conectarea la server

Serverul MySQL a fost creat odată cu configurarea fișierului YML prezentat anterior, ceea ce a inclus setările de networking, datele de autentificare și numele server-ului.

După ce acestea au fost stabilite și baza de date a fost creată, a fost posibilă crearea legăturii cu baza de date. Pentru realizare acestui lucru, au fost create entități de tip tabel în Java, operațiile fiind realizate cu ajutorul Java JPA, un modul nativ de interacționat cu baza de date, fără a mai scrie manual interogări.

Editorul IntelliJ permite vizualizarea bazei de date, astfel rulând aplicația și adăugând credențialele precizate în fișierul YML a fost realizată cu succes adăugarea bazei de date, așa cum reiese din fig. 3.3.

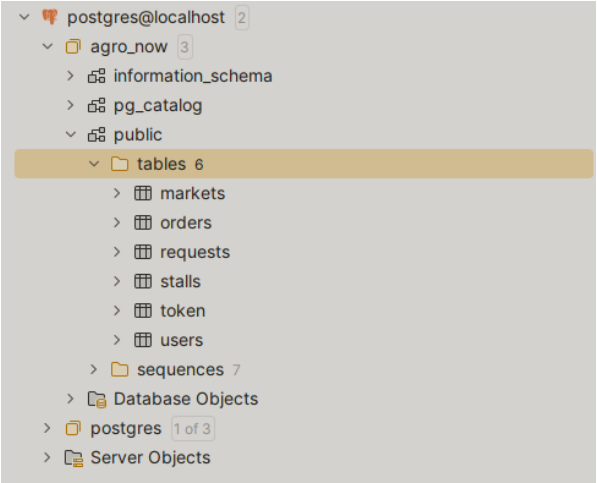


Fig 3. 3 Conexiunea cu baza de date vizibilă în IntelliJ

3.3 Realizarea modelelor pentru baza de date cu Java JPA

Pentru definirea unei structuri a tabelelor din baza de date, au fost necesare crearea unor clase de Java care vor avea asignate tag-urile de @Entity și @Table pentru recunoaștere în momentul rulării aplicației. În momentul rulării aplicației se vor crea tabelele respective doar dacă acestea nu există. Aceste tag-uri sunt incorporate în limbajul de programare Java cu ajutorul service-ului Java JPA, care este o unealtă specifică pentru lucrul cu baze de date folosind PostgreSQL.

Fiind clase scrise cu ajutorul limbajului de programare Java, acestea vor avea în plus asignate tag-urile de @Getter și @Setter, care vor fi utile în momentul în care vom crea o instanță nouă și vom avea nevoie sa adăugam sau să extragem date din clasa respectivă.

Modalitățile de realizare a celorlalte tabele este asemănătoare, fiind prezentare în detaliu, cu relații și atributele specifice, în Anexa 1.

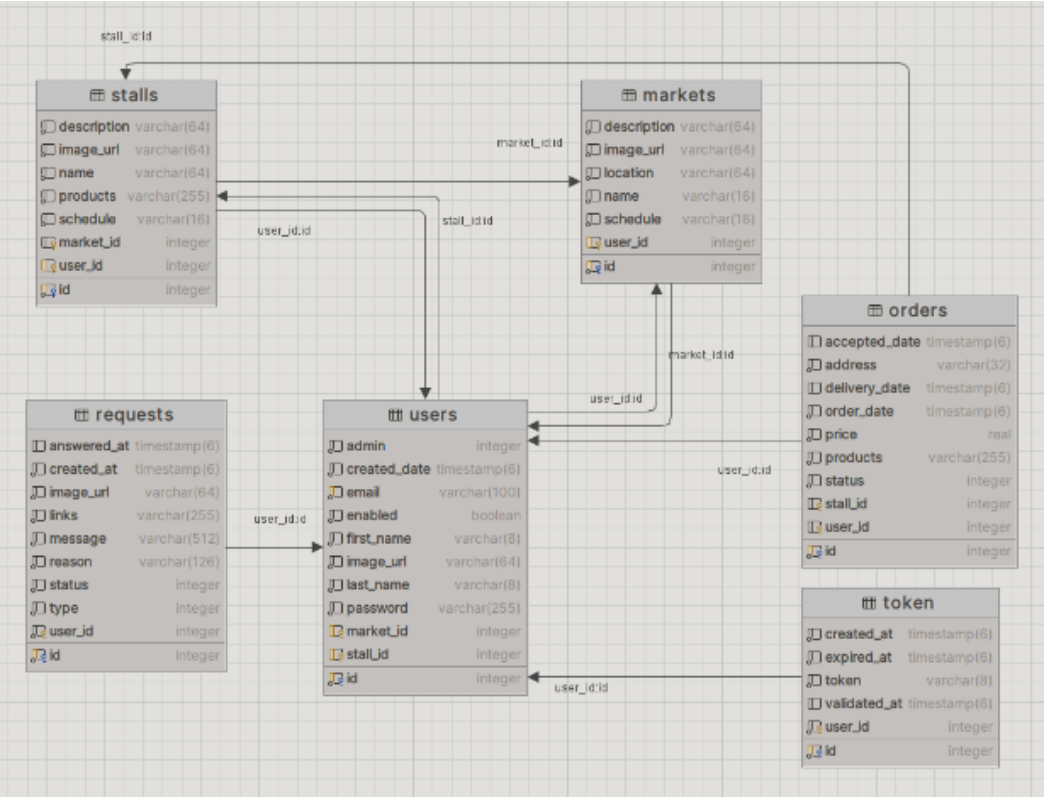


Fig 3. 4 Schema bazei de date în IntelliJ

Tabelele din baza de date, prezentate în schema anterioare, sunt detaliate în cele ce urmează:

users: tabel realizat cu scopul înregistrării și autentificării pe platformă.

Coloane:

ID = tip: număr întreg, autoincrementabil, cheie primară.

firstName = tip: text, 8 caractere, nu permite valori nule.

lastName = tip: text, 8 caractere, nu permite valori nule.

email = tip: text, 100 caractere, nu permite valori nule, valoare unică.

password = tip: text, 255 caractere, nu ptermite valori nule.

createdDate = tip: local date, nu permite valori nule, nu permite updatarea.

enabled = tip: boolean, valoare implicită: fals.

imageUrl = tip: text, 64 caractere, nu permite valori nule.

admin = tip: număr întreg, valoare implicită: 0.

Market_id = tip: număr întreg, permite valoare nulă, cheie străină pentru tabelul ,,markets”.

Stall_id = tip: număr întreg, permite valoare nulă, foreign_key cu tabela Stalls.

token: tabel realizat în vederea activării contului unui utilizator.

Coloane:

ID = tip: număr întreg, autoincrementabil, cheie primară.

token = tip: text, 6 caractere, nu permite valori nule.

createdAt = tip: local date, nu permite valori nule.

expiredAt = tip: local date, nu permite valori nule.

validatedAt = tip: local date, permite valori nule.

user_id = tip: număr întreg, relație de tipul Many-To-One cu tabelul „users”, cheie străină pentru tabelul ,,users”, nu permite valoară nulă.

requests: tabel cu rol de stocare a informațiilor despre cererile pentru roluri făcute de utilizatori.

Coloane:

ID = tip: număr întreg, autoincrementabil, cheie primară.

type = tip: număr întreg, nu permite valori nule.

status = tip: număr întreg, nu permite valori nule.

message = tip: text, 512 caractere, nu permite valori nule.

links = tip: text, 255 caractere, nu permite valori nule.

imageUrl = tip: text, 64 caractere, nu permite valori nule.

reason = tip: text, 126 caractere, , nu permite valori nule.

createdAt = tip: local date, nu permite valori nule.

answeredAt = tip: local date, permite valori nule.

user_id = tip: număr întreg, relație de tipul Many-To-One cu tabelul ,,users”, cheie străină pentru tabelul ,,users”, nu permite valoară nulă.

markets: tabel cu rol de stocare a informațiilor despre piețele agroalimentare înscrise în aplicație.

Coloane:

ID = tip: număr întreg, autoincrementabil, cheie primară.

name = tip: text, 64 caractere, nu permite valori nule.

location = tip: text, 16 caractere, nu permite valori nule.

description = tip: text, 64 caractere, nu permite valori nule.

schedule = tip: text, 16 caractere, nu permite valori nule.

imageUrl = tip: text, 64 caractere, nu permite valori nule.

user_id = tip: număr întreg, relație de tipul One-To-One cu tabelul ,,users'', cheie străină pentru tabelul ,,users", nu permite valoare nulă.

stalls: tabel cu rol de stocare a informațiilor despre tarabele din cadrul piețelor înscrise în aplicație.

Coloane:

ID = tip: număr întreg, autoincrementabil, cheie primară.

name = tip: text, 64 caractere, nu permite valori nule.

description = tip: text, 64 caractere, nu permite valori nule.

imageUrl = tip: text, 64 caractere, nu permite valori nule.

products = tip: text, 255 caractere, nu permite valori nule.

schedule = tip: text, 16 caractere, nu permite valori nule.

market_id = tip: număr întreg, relație de tipul Many-To-One cu tabelul ,,markets'', cheie străină pentru tabelul ,,markets'', permite valoare nulă.

user_id = tip: număr întreg, relație de tipul One-To-One cu tabelul ,,users'', cheie străină pentru tabelul ,,users'', permite valoare nulă.

orders: tabel cu rol de stocare a informațiilor despre comenzile făcute de către utilizatori.

Coloane:

ID = tip: număr întreg, autoincrementabil, cheie primară.

products = tip: text, 255 caractere, nu permite valori nule.

address = tip: text, 32 caractere, nu permite valori nule.

price = tip: număr zecimal, nu permite valori nule.

status = tip: număr întreg, valoare implicită: 0, nu permite valori nule.

orderDate = tip: local date, nu permite valori nule.

acceptedDate = tip: local date, permite valoare nulă.

deliveryDate = tip: local date, permite valoare nulă.

stall_id = tip: număr întreg, relație de tipul Many-To-One cu tabelul ,,stalls'', cheie străină pentru tabelul ,,stalls'', permite valoare nulă.

user_id = tip: număr întreg, relație de tipul Many-To-One cu tabelul ,,users'', cheie străină pentru tabelul ,,users'', permite valoare nulă.

3.4 Realizarea back-end-ului în Java

3.4.1 Arhitectura modulelor

În Java pentru a crea un anumit modul pentru baza de date este nevoie de a avea o serie de fișiere specifice. Conform fig. 3.5, pentru fiecare model trebuie să avem obligatoriu următoarele fișiere de tip Java: Controller, acest fișier are în componența sa tag-ul specific Java care specifică în momentul rulării aplicației care sunt rutele disponibile. În acest fișier sunt prezente operații HTTP, care au tag-uri specifice precum: @PostMapping, @GetMapping și @PutMapping, urmate de o rută care să diferențieze operațiile între ele. Repository, este al doilea fișier ce este nevoie să existe într-un modul, această clasă este o extensie din cadrul service-ului Java JPA care permite interacțiunea cu baza de date într-un mod mai simplu, nefiind nevoie de scris interogări clasice. Clasa care se extinde este „JpaRepository<>” urmată de modelul de baze de date specfici, cât și tipul care este definită cheia primară. Service, este al treilea fișier necesar unui modul. Acesta trebuie să aibe tag-ul de @Service specific Java. În cadrul acestui fișier va fi pusă întreagă logică a rutelor din fișierul anterior precizat, Controller. Următoarele două fișiere sunt opționale, în funcție de nevoie rutei respective. Response, este o clasă simplă de Java care are în alcătuire tipuri de date ce vor fi adăugate ca metode de returnare în cadrul funcțiilor scrise în clasa de Service. Ultima clasa, care la fel nu este obligatorie, este Data. În cadrul acestui fișier putem adăuga tipuri de date care să servească la tipul de date al parametrilor primiți în cadrul Controller-ului.

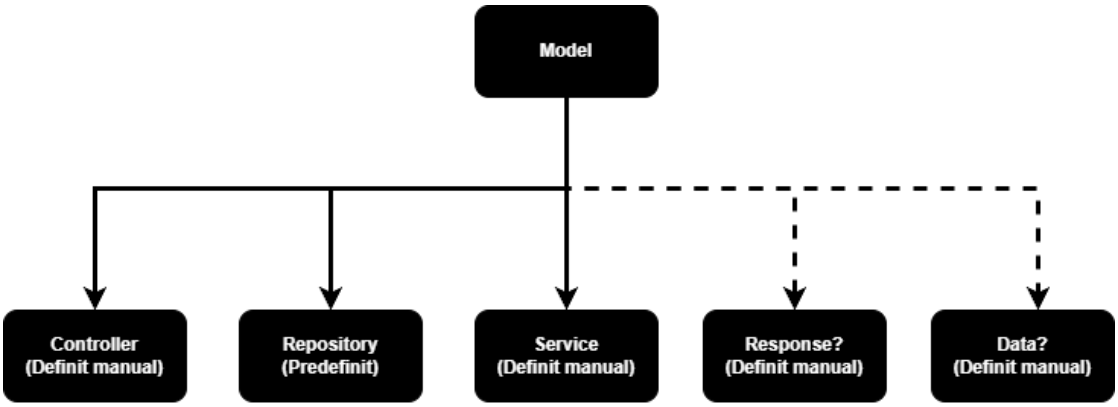


Fig 3. 5 Arhitectura unui modul în Java

3.4.2 Logarea, înregistrarea și activarea contului utilizatorului

Clasa de utilizator (user) a fost creată și explicată conform Fig. 3.5. Astfel, un utilizator pentru a-și crea un cont are nevoie de un email valid, cât și de celelalte câmpuri specificate mai sus. Pentru a diferenția operațiile, rutele de accesare ale back-end-ului către serverul de Java sunt diferite. Accesarea generală a service-ului de back-end cu server-ul de Java se face accesând adresa locală „http://localhost:8088/api". Iar pentru a accesa una dintre operațiile prezentate va fi nevoie să adăugam o rută în plus, dar și de parametrii separați în funcție de tipul de operație.

Operațiile au tipuri de cereri HTTP diferite: logarea și înregistrarea au tipul de cerere POST, pe când activarea contului este de tipul GET. Pentru crearea acestor rute au fost nevoie să creez clase de Java denumite controllere cu tag-ul de @RestController, specific Java JPA, unde în funcție de rută să se execute diferite funcții.

Rutele au în componență începutul „http", în loc de „https". Litera „s" de la final va apărea atunci când aplicația va fi lansată publicului larg, în urma obținerii unor certificate de tip SSL (Secure Sockets Layer). Așadar, toate apelurile și exemplele de adrese vor avea în componență „http.." în loc de clasicul „https" care apare pe toate site-urile.

```java
@RestController  Robert Nitu
@RequestMapping("auth")
@RequiredArgsConstructor
@Tag(name = "Authentication")
public class AuthenticationController {

    private final AuthenticationService service;

    @PostMapping("/register")  Robert Nitu
    @ResponseStatus(HttpStatus.ACCEPTED)
    public ResponseEntity<?> register(
            @RequestBody @Valid RegistrationRequest request
    ) throws MessagingException {
        service.register(request);
        return ResponseEntity.accepted().build();
    }

    @PostMapping("/authenticate")  Robert Nitu
    public ResponseEntity<AuthenticationResponse> authenticate(
            @RequestBody @Valid AuthenticationRequest request
    ) {
        return ResponseEntity.ok(service.authenticate(request));
    }

    @GetMapping("/activate-account")  Robert Nitu
    public void confirm(@RequestParam String token) throws MessagingException {
        service.activateAccount(token);
    }

}
```

Fig 3. 6 Rutele de autentificare

Pentru ruta de înregistrare ,,http://localhost:8088/api/register” se vor trimite datele esențiale creări unui cont, mai exact un obiect cu următoarele valori: „firstName”, „lastName”, „email”, „password” și „imageUrl”. Ruta de înregistrare verifică dacă există deja un cont creat dupa adresa de email primită ca parametru în urma apelului către server-ul de back-end. În cazul în care s-a găsit un utilizator cu aceași adresă de email, se va trimite un mesaj de eroare specific. Iar dacă nu s-a găsit un utilizator cu adresa de email primită se va crea utilizatorul și se va adăuga în baza de date. Înainte de salvare se va face criptarea parolei folosind encriptarea Java din clasa specifică Java Security, un modul specific limbajului de programare Java. După criptarea parolei se vor popula coloanele de createdDate, care reprezintă data la care s-a creat contul și admin, care reprezintă funcția de administrator a contului care este setată ca implicit valoarea 0, ce reprezintă că nu este administrator.

```
public void register(RegistrationRequest request) throws MessagingException {  1 usage  Robert Nitu
    if (userRepository.findByEmail(request.getEmail()).isPresent()) {
        throw new RuntimeException("Email is already registered");
    }

    var user = User.builder()
            .firstName(request.getFirstName())
            .lastName(request.getLastName())
            .email(request.getEmail())
            .password(passwordEncoder.encode(request.getPassword()))
            .enabled(false)
            .imageUrl(request.getImageUrl())
            .createdDate(LocalDateTime.now())
            .admin(0)
            .build();
    userRepository.save(user);

    sendValidationEmail(user);
}
```

Fig 3. 7 Funcția „register”

După ce utilizatorul este salvat în baza de date urmează următorul pas, anume trimiterea email-ului de validare a contului. Această măsură a fost implementată pentru a spori securitatea aplicației. Următorul pas după crearea utilizatorului este acela de a genera un Token de activare specific utilizatorului, care să expire după 30 minute. În email-ul respectiv va fi un mesaj cu informații legate de aplicația de digitalizare, urmate de codul token format din 6 caractere, dar și de link-ul către pagina de activare. Conform fig. 3.8 generarea de token este făcută în funcția denumită ,,generateActivationCode” care preia caractere din constanta ,,characters’’, ce conține toate cifrele, într-o ordine aleatorie. După ce este generat, aceasta valoare este returnată sub tipul de String, iar în funcția numită ,,generateAndSaveActionToken’’ este creată o inserare în baza de date în tabelul ,,token’’, având parametrii createdAt, reprezentând data și ora în momentul în care se inserează o intrare, iar expiredAt, reprezentând când va expira token-ul de activare al contului, formula fiind data și ora curentă plus 30 minute.

```java
private String generateAndSaveActivationToken(User user) {  1 usage  Robert Nitu
    String generatedToken = generateActivationCode();

    var token = Token.builder()
            .token(generatedToken)
            .createdAt(LocalDateTime.now())
            .expiredAt(LocalDateTime.now().plusMinutes(30))
            .user(user)
            .build();
    tokenRepository.save(token);

    return generatedToken;
}

private String generateActivationCode() {  1 usage  Robert Nitu
    int length = 6;
    String characters = "0123456789";
    StringBuilder codeBuilder = new StringBuilder();
    SecureRandom secureRandom = new SecureRandom();

    for (int i = 0; i < length; i++) {
        int randomIndex = secureRandom.nextInt(characters.length());
        codeBuilder.append(characters.charAt(randomIndex));
    }

    return codeBuilder.toString();
}
```

Fig 3. 8 Funcțiile „generateAndSaveActivationToken” și „generateActivationCode”

Conform fig. 3.8, funcțiile prezentate mai sus sunt folosite pentru a genera token-ul de activare al utilizatorului și crea o intrare în baza de date, care în funcția de bază ,,sendValidationEmail'' este salvată în variabila numită ,,newToken''. Folosind service-ul numit ,,emailService'' se va trimite email-ul de activare către utilizator, acest lucru se va face folosind unealta de simulare a mail-urilor MailDev[19]. Conform fig. 3.9 funcția este una de tipul asincronă, aceasta va rula pe thread-ul principal în cadrul aplicației Java, va aștepta să se termine de rulat întreaga funcție înainte de a se face altă acțiune. Pentru trimiterea email-ului de validare, acesta estre construit în jurul unui template HTML conform fig. 3.10, în funcția de ,,sendEmail'' definim niște proprietăți cu anumitei chei, precum codul de activare ,,activation_code'' care reprezintă token-ul, ,,username'' care este numele întreg al utilizatorului și ,,confirmationUrl'' reprezentând adresa unde utilizatorul își poate active contul. Aceste valori le putem apela în fișierul de tip HTML. Totodată, sunt setate elemente de bază din cadrul unui email, cum ar fi subiectul email-ului, setarea destinatarului și a expeditorului.

```java
@Async  1 usage  Robert Nitu
public void sendEmail(
        String to,
        String username,
        EmailTemplateName emailTemplateName,
        String confirmationUrl,
        String activationCode,
        String subject) throws MessagingException {

    String templateName;
    if (emailTemplateName == null) {
        templateName = "confirm-email";
    } else {
        templateName = emailTemplateName.name();
    }

    MimeMessage mimeMessage = mailSender.createMimeMessage();
    MimeMessageHelper helper = new MimeMessageHelper(
            mimeMessage,
            MimeMessageHelper.MULTIPART_MODE_MIXED,
            UTF_8.name()
    );

    Map<String, Object> properties = new HashMap<>();
    properties.put("username", username);
    properties.put("confirmationUrl", confirmationUrl);
    properties.put("activation_code", activationCode);

    Context context = new Context();
    context.setVariables(properties);

    helper.setFrom("contact@agronow.ro");
    helper.setTo(to);
    helper.setSubject(subject);

    String template = templateEngine.process(templateName, context);

    helper.setText(template,  html: true);

    mailSender.send(mimeMessage);
}
```

Fig 3. 9 Funcția „sendEmail”

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Account Activation</title>
    <style...>
</head>
<body>
<div class="container">
    <div class="center">
        <img src="https://i.imgur.com/foH99ZV.png" style="..." alt="logo" />
    </div>
    <br/>
    <br/>
    <h1>Account Activation</h1>
    <p th:text="'Buna Ziua ' + ${username} + ','"></p>
    <p>Îți mulțumim pentru înregistrare! Te rugăm să folosești următorul cod de activare pentru a-ți activa contul:</p>
    <div class="activation-code"><span th:text="${activation_code}"></span></div>
    <div class="activation-link">
        <a th:href="${confirmationUrl}" target="_blank">Activează-ți contul!</a>
    </div>
</div>
</body>
</html>
```

Fig 3. 10 Template-ul de mail pentru activare cont

Pentru ruta de logare a utilizatorului ,,http://localhost:8080/api/auth/authenticate'' unde se vor trimite informații despre conectarea utilizatorului. Conform Fig. 3.16 se va trimite un obiect de tipul ,,AuthenticationReques'' care conține informații despre email și parolă. Clasa construită are câteva verificări la aceste două valori, acestea trebuie să nu fie valori nule, email-ul să aibe structura generică a unui email, iar parola să aibe cel puțin 8 caractere. Având acest obiect se va putea face acțiunea de logare.

Conform fig. 3.11, în funcția definită ,,authenticate'' se verifică dacă există un utilizator cu credențialele introduse. În variabila ,,auth'' se extrage și se reține o copie locală a utilizatorului doar dacă modulul ,,authenticationManager'' găsește un utilizator cu datele primite ca parametru. În cazul în care utilizatorul există și credențialele sunt corecte se va construi cu ajutorului service-ului Java JWT, un token ce va fi trimis pe server-ul de front-end, unde pe întreaga sesiune utilizatorul va avea salvate cu ajutorul storage-ului local al browser-ului date despre cont, cum ar fi ID-ul specific din baza de date, rolul acestuia, dacă este administrator, proprietar de piață sau proprietar de tarabă. Funcția de ,,authenticate'' returnează un obiect de tipul ,,AuthenticationResponse'' care conține variabile setate din utilizator, unde pe partea de front-end vor fi de folos pentru începerea navigării din punct de vedere al unui utilizator.

```
public AuthenticationResponse authenticate(AuthenticationRequest request) {  1 usage  Robert Nitu
    var auth = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                    request.getEmail(),
                    request.getPassword()
            )
    );

    var claims = new HashMap<String, Object>();
    var user = (User)auth.getPrincipal();
    claims.put("fullName", user.getFullName());

    var jwtToken = jwtService.generateToken(claims, user);
    var response = AuthenticationResponse
            .builder()
            .token(jwtToken)
            .id(user.getId())
            .admin(user.getAdmin());

    if (user.getMarket() != null)
        response.marketId(user.getMarket().getId());
    if (user.getStall() != null)
        response.stallId(user.getStall().getId());

    return response.build();
}
```

Fig 3. 11 Funcția „authenticate”

Ultima rută din cadrul operațiilor de gestionare a contului utilizatorului este aceea de activare a contului valabilă pe ruta ,,http://localhost:8080/auth/activate-account''. Singurul parametru primit este acela de token, care este de tipul String de 6 caractere. Conform Fig. 3.12, funcția de activare cont denumită ,,activateAccount'' verifică în primul rând dacă există un token creat în baza de date,

în cazul negativ se va returna ca răspuns un mesaj de eroare intuitiv. Token-urile create au o dată de expirare, neputând fi folosite după mai mult de 30 minute de la crearea contului. Dacă codul este expirat, au trecut cele 30 minute, se va genera un alt token și se va trimite pe mail-ul utilizatorului. În acest caz este refolosită funcția de ,,sendValidationEmail'' prezentată mai sus. Acest lucru oferă robustitate codului și îl face mult mai optimizat din punct de vedere al performanței.

Dacă codul token este încă activ, se va extrage instanța de utilizator în variabila denumită ,,user'', unde se va seta coloana ,,enabled'' pe valoarea true, iar din acest moment utilizatorul este liber să folosească aplicația și se poate loga cu succes. După acest pas urmează updatarea coloanei ,,validatedAt'' din cadrul token-ului, fiind ultimul pas din cadrul operațiilor de gestiune a utilizatorilor.

```
public void activateAccount(String token) throws MessagingException {  1 usage   Robert Nitu
    Token savedToken = tokenRepository.findByToken(token)
            .orElseThrow(() -> new RuntimeException("Invalid token"));

    if (LocalDateTime.now().isAfter(savedToken.getExpiredAt())) {
        sendValidationEmail(savedToken.getUser());
        throw new RuntimeException("Activation Token has expired. A new token has been send to the same email address.");
    }

    var user = userRepository.findById(savedToken.getUser().getId())
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));
    user.setEnabled(true);
    userRepository.save(user);

    savedToken.setValidatedAt(LocalDateTime.now());
    tokenRepository.save(savedToken);
}
```

Fig 3. 12 Funcția „activateAccount”

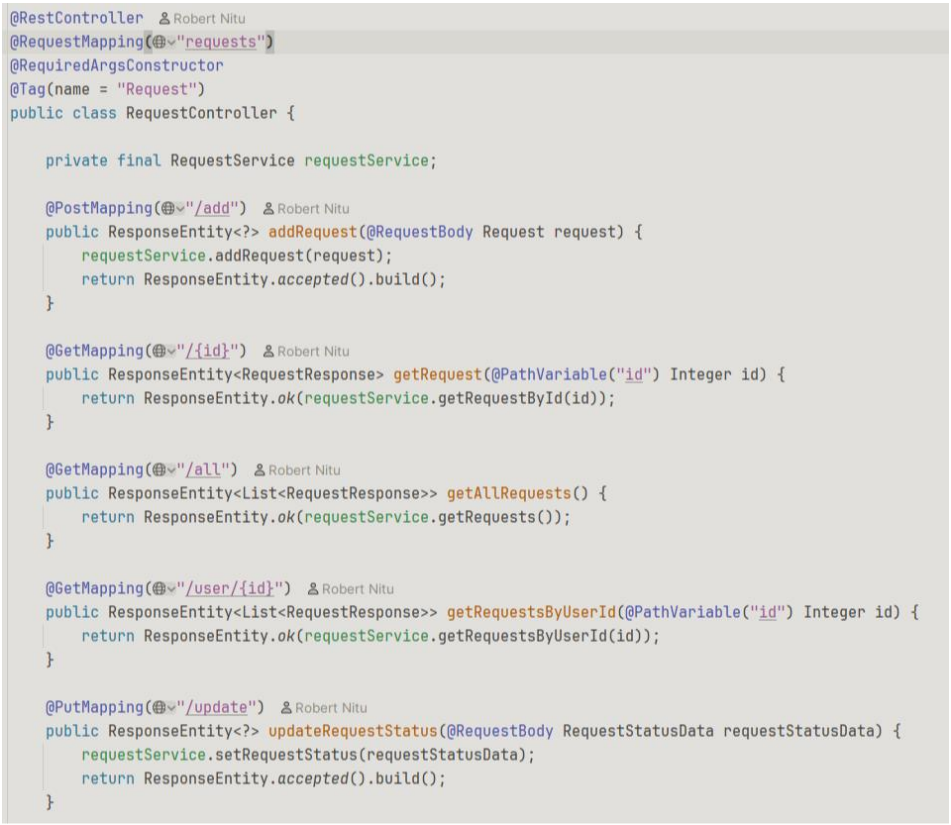
3.4.2 Cererile pentru roluri

Operațiile sunt cereri de HTTP diferite, pentru crearea de noi cereri se folosește tipul POST, pentru a extrage informații despre cererile utilizatorului se folosește tipul GET, iar pentru a schimba status-ul unei cereri se folosește tipul PUT. Aceste 3 operații sunt făcute în funcție de operația dorită.

Pentru a accesa aceste operații ruta este una diferită față de punctul anterior, adresa generală fiind ,,http://localhost:8080/api/requests/'', urmată de rute stabilite pentru fiecare operație. Conform Fig. 3.13 a fost creată o clasă de Java cu tag-ul de @RestController, iar operațiile găsite în cadrul rutei de ,,request'' sunt de adăugare cerere (add), de extragere a tuturor cererilor (all), această funcționalitate este doar pentru utilizatorii ce au rolul de administrator, de extragere a tuturor cererilor unui utilizator (user/id), iar ultima fiind cea de actualizare a unei cereri, mai exact de actualizat status-ul și motivul.

Pentru a adăuga o cerere nouă se va accesa ruta de ,,add'' conform adresei ,,http://localhost:8080/api/requests/add'' urmată de un obiect de tipul ,,Request'' care a fost prezentat în descrierea bazei de date. Conform Fig. 3.14, funcția ,,addRequest'' se ocupă de toată logica adăugării unei cereri noi. Restricția generală privind cererile este că utilizatorul poate avea câte cereri

dorește, dar nu poate avea mai multe cereri de același tip în același timp active, de exemplu utilizatorul poate avea simultan doar o cerere de administrator și una de proprietar de piață, ambele având status-ul de PENDING (în așteptare). În variabila ,,userRequests'' se vor stoca toate cererile utilizatorului care au tip-ul de cerere conform obiectului primit ca parametru și au status-ul de PENDING (în așteptare). Dacă această listă are mai mult decât un element, acțiunea este oprită și se va trimite un mesaj de eroare specific. Următorul pas este să extragem o copie a utillizatorului pe baza ID-ului primit din obiectul ca parametru, acest lucru ne asigură că cererea are un utilizator care a făcut-o. Ultimul pas este crearea propriu-zisă a cererii cu valorile din obiectul primit ca parametru, setând pe partea de server de back-end coloanele următoare cu valori implicite: status, are valoarea implicită de PENDING (în așteptare), reason, are valoarea implicită un String gol, iar answeredAt, are valoarea implicită nulă.



```
@RestController  Robert Nitu
@RequestMapping("requests")
@RequiredArgsConstructor
@Tag(name = "Request")
public class RequestController {

    private final RequestService requestService;

    @PostMapping("/add")  Robert Nitu
    public ResponseEntity<?> addRequest(@RequestBody Request request) {
        requestService.addRequest(request);
        return ResponseEntity.accepted().build();
    }

    @GetMapping("/{id}")  Robert Nitu
    public ResponseEntity<RequestResponse> getRequest(@PathVariable("id") Integer id) {
        return ResponseEntity.ok(requestService.getRequestById(id));
    }

    @GetMapping("/all")  Robert Nitu
    public ResponseEntity<List<RequestResponse>> getAllRequests() {
        return ResponseEntity.ok(requestService.getRequests());
    }

    @GetMapping("/user/{id}")  Robert Nitu
    public ResponseEntity<List<RequestResponse>> getRequestsByUserId(@PathVariable("id") Integer id) {
        return ResponseEntity.ok(requestService.getRequestsByUserId(id));
    }

    @PutMapping("/update")  Robert Nitu
    public ResponseEntity<?> updateRequestStatus(@RequestBody RequestStatusData requestStatusData) {
        requestService.setRequestStatus(requestStatusData);
        return ResponseEntity.accepted().build();
    }
```

Fig 3. 13 Ruta „requests”

```
public void addRequest(Request request) {  1 usage   Robert Nitu
    List<Request> userRequests = requestRepository
            .findAll() List<Request>
            .stream() Stream<Request>
            .filter( Request requestData ->
                    requestData.getUser().getId().equals(request.getRequesterId()) &&
                    requestData.getType().equals(request.getType()) &&
                    requestData.getStatus() == RequestStatus.PENDING.ordinal())
            .toList();
    if (!userRequests.isEmpty()) {
        throw new EntityNotFoundException("Can't add Request, already created one with this type.");
    }

    User user = userRepository
            .findById(request.getRequesterId())
            .orElseThrow(() -> new EntityNotFoundException("User not found"));

    Request requestToAdd = new Request();
    requestToAdd.setType(request.getType());
    requestToAdd.setStatus(RequestStatus.PENDING.ordinal());
    requestToAdd.setMessage(request.getMessage());
    requestToAdd.setLinks(request.getLinks());
    requestToAdd.setImageUrl(request.getImageUrl());
    requestToAdd.setReason("");
    requestToAdd.setCreatedAt(LocalDateTime.now());
    requestToAdd.setAnsweredAt(null);
    requestToAdd.setUser(user);

    requestRepository.save(requestToAdd);
}
```

Fig 3. 14 Funcția „addRequest”

Pentru a actualiza o cerere deja creată, se va folosi ruta de update (actualizare) care are următoarea adresă „http://localhost:8080/api/requests/update’’ urmată de un obiect de tipul „RequestStatusData’’ conform Fig. 3.15. Funcția responsabilă acestei operații este „setRequestStatus’’, conform Fig. 3.16 prima dată se va extrage din baza de date cererea cu ID-ul respectiv, în cazul în care aceasta nu există se va opri execuția și se va trimite un mesaj de eroare specific. Dacă cererea există se vor seta valorile primite ca parametru, mai exact status, reason și answeredAt, iar apoi se va actualiza cererea în baza de date.

Pentru cererile de administrator care au primit ca răspuns valoarea ACCEPTED (acceptat), va trebui să setăm utilizatorul ca având rolul de administrator, acest lucru se va face extrăgând utilizatorul utilizând cheia străină din cererea extrasă anterior, iar dacă cererea are tipul de ADMINISTRATOR (admin) și răspunsul este ACCEPTED (acceptat), se va actualiza utilizatorul.

```
@Getter  3 usages   Robert Nitu
@Setter
public class RequestStatusData {

    private Integer id;
    private RequestStatus Status;
    private String reason;

}
public enum RequestStatus {
    PENDING,
    ACCEPTED,
    REJECTED,
}
```

Fig 3. 15 Clasa „RequestStatusData” și enum-ul „RequestStatus”

```java
public void setRequestStatus(RequestStatusData requestStatusData) {  1 usage  Robert Nitu
    Request request = requestRepository
            .findById(requestStatusData.getId())
            .orElseThrow(() -> new EntityNotFoundException("Request not found"));

    request.setStatus(requestStatusData.getStatus().ordinal());
    request.setReason(requestStatusData.getReason());
    request.setAnsweredAt(LocalDateTime.now());
    requestRepository.save(request);

    if (request.getType() == RequestTypes.ADMINISTRATOR.ordinal() &&
        request.getStatus() == RequestStatus.ACCEPTED.ordinal())
    {
        User user = request.getUser();
        user.setAdmin(1);
        userRepository.save(user);
    }
}
```

Fig 3. 16 Funcția „setRequestStatus”

Ultimele două operații sunt foarte asemănătoare, având ambele la baza cererea HTTP de tipul GET, însă au rute separate. Conform Fig. 3.18, sunt două metode de extragere a informațiilor despre cereri în funcție de rolul utilizatorului. Ambele operații vor returna un obiect de tipul ,,RequestResponse'' care este o clasă de Java ce are date despre cerere, cât și despre utilizator, acestea fiind simplificate și create într-un singur obiect. Pentru a popula acest obiect a fost creată funcția ,,toRequestResponse'', care folosește tag-urile specifice limbajului de programare Java, și face crearea obiectului foarte ușoară.

```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class RequestResponse {

    private Integer id;
    private Integer type;
    private Integer status;
    private String message;
    private String links;
    private String imageUrl;
    private String reason;

    private LocalDateTime createdAt;
    private LocalDateTime answeredAt;

    private Integer userId;
    private String userFullName;
    private String userEmail;
    private String userImageUrl;

}
```

```java
private RequestResponse toRequestResponse(Request request) {  3 usages  Robert Nitu
    User user = request.getUser();

    return RequestResponse.builder()
            .id(request.getId())
            .type(request.getType())
            .status(request.getStatus())
            .message(request.getMessage())
            .links(request.getLinks())
            .imageUrl(request.getImageUrl())
            .reason(request.getReason())
            .createdAt(request.getCreatedAt())
            .answeredAt(request.getAnsweredAt())
            .userId(user.getId())
            .userFullName(user.getFullName())
            .userEmail(user.getEmail())
            .userImageUrl(user.getImageUrl())
            .build();
}
```

Fig 3. 17 Clasa „RequestResponse” și funcția „toRequestResponse”

Pentru ruta de extragere a tuturor cererilor, valabilă pentru utilizatorii ce au rolul de administrator, este necesară folosirea adresei ,,http://localhost:8080/api/requests/all’’. Acest apel către server-ul de back-end nu are niciun parametru, iar conform Fig. 3.18 funcția ,,getRequests’’ va returna o listă de toate cererile din tabelul ,,requests’’ din baza de date. În variabila ,,requests’’ se vor stoca toate cererile existente în tabelul specific din baza de date, iar dacă acesta este gol, nu are nicio intrare, execuția funcției se va opri și se va returna un mesaj de eroare specific. Dacă sunt intrări se vor face niște operații asupra listei, iar cu ajutorul funcției descrise anterior ,,toRequestResponse’’ se va returna o lista simplificată cu toate cererile existente.

Pentru ruta de extragere a unor cereri, mai specific de extras a cererilor unui utilizator, se va folosi adresa „http://localhost:8080/api/requests/user’’ urmat de un parametru de ID. Conform Fig. 18 funcția „getRequestsByUserId’’ va returna o lista simplificată a tuturor cererilor unui utilizator. În cazul în care utilizatorul nu are nicio cerere create, se va returna o lista goală. Funcția extrage toate cererile existente și le filtrează, impune o condiție de tip WHERE, în funcție de ID-ul utilizatorului care are cererea create. Următorul pas este de a simplifica datele cu ajutorul funcției „toRequestResponse’’.

```
public List<RequestResponse> getRequests() {  1 usage   Robert Nitu
    List<Request> requests = requestRepository.findAll();

    if (requests.isEmpty()) {
        throw new EntityNotFoundException("Requests not found");
    }

    return requests
            .stream()  Stream<Request>
            .map(this::toRequestResponse)  Stream<RequestResponse>
            .collect(Collectors.toList());
}

public List<RequestResponse> getRequestsByUserId(Integer id) {  1 usage   Robert Nitu
    return requestRepository
            .findAll()  List<Request>
            .stream()  Stream<Request>
            .filter( Request request -> request.getUser().getId().equals(id))
            .map(this::toRequestResponse)  Stream<RequestResponse>
            .collect(Collectors.toList());
}
```

Fig 3. 18 Funcțiile „getRequests” și „getRequestsByUserId”

3.4.3 Gestionarea piețelor

Operațiile HTTP pentru gestionarea piețelor sunt de două tipuri, una este de tipul POST, aceasta fiind metoda de adăugare a unei noi piețe în baza de date pe baza unei logici predefinite, iar cealalta fiind GET, unde se extrag informații cu privire la piețele deja existente în tabela ,,markets’’ din baza de date.

Pentru a accesa acest serviciu este nevoie de o altă rută specială, mai exact ruta „http://localhost:8080/api/markets”, urmată ulterior de tipul de operație dorită. Conform Fig. 3.19, ruta de ,,markets’’ are 3 metode.

Prima metodă este cea de adăugare a unui market (piață) în baza de date. Aceasta este accesibilă la adresa „http://localhost:8080/api/markets/add”, urmată de un obiect de tip ,,Market’’, definit conform descrierii bazei de date descrise mai sus. Funcția definită „addMarket” conform Fig. 3.20 are rolul de a dăuga o intrare nouă în tabela de ,,markets’’ după anumite verificări. În acest caz, singura verificare este cea de a exista un utilizator ce va fi asignat drept proprietar al pieței în curs de creare. Dacă utilizatorul nu exista, execuția funcției de oprește și returnează un mesaj intuitiv de eroare. În cazul în care utilizatorul a fost găsit se va începe construirea obiectului de tip ,,Market’’ cu valori din obiectul ,,market’’ primit ca parametru, iar în momentul salvării se va verifica dacă utilizatorul este deja proprietar al unei piețe deja create, acest lucru fiind posibil prin intermediul relației de tipul One-To-One stabilită în momentul în care s-a creat tabela de ,,markets’’. Ultimul pas în cadrul acestei funcții este asignarea utilizatorului coloana „market_id” din proprietățile tabelului ,,users’’.

```java
@RestController  Robert Nitu
@RequestMapping("markets")
@RequiredArgsConstructor
@Tag(name = "Market")
public class MarketController {

    private final MarketService marketService;

    @PostMapping("/add")  Robert Nitu
    public ResponseEntity<?> addMarket(@RequestBody Market market) {
        marketService.addMarket(market);
        return ResponseEntity.accepted().build();
    }

    @GetMapping("/{id}")  Robert Nitu
    public ResponseEntity<MarketResponse> getMarket(@PathVariable("id") Integer id) {
        return ResponseEntity.ok(marketService.getMarketById(id));
    }

    @GetMapping("/all")  Robert Nitu
    public ResponseEntity<List<MarketResponse>> getAllMarkets() {
        return ResponseEntity.ok(marketService.getAllMarkets());
    }

}
```

Fig 3. 19 Ruta „markets”

```java
public void addMarket(Market market) {  1 usage  Robert Nitu
    User user = userRepository
            .findById(market.getOwnerId())
            .orElseThrow(() -> new EntityNotFoundException("User not found"));

    Market finalMarket = new Market();
    finalMarket.setName(market.getName());
    finalMarket.setLocation(market.getLocation());
    finalMarket.setDescription(market.getDescription());
    finalMarket.setSchedule(market.getSchedule());
    finalMarket.setImageUrl(market.getImageUrl());
    finalMarket.setUser(user);

    marketRepository.save(finalMarket);

    user.setMarket(finalMarket);
    userRepository.save(user);
}
```

Fig 3. 20 Funcția „addMarket”

Următoarele operații au rolul de a extrage informații din baza de date și a le transforma în tipul de date „MarketResponse”, conform Fig. 3.21. Această clasă are tag-urile de @Getter și @Setter specifice limbajului de programare Java, care fac setatul și extrasul de câmpuri ale clasei mult mai ușor și sub o formă mult mai curate. Conform Fig. 3.21, funcția „toMarketResponse” primește ca parametru o entitate de tip „Market”, aceasta reprezentând intrările în tabela „markets”, și le convertește în tipul de date precizat mai sus, cu ajutorul tag-urilor de setare și extragere.

```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class MarketResponse {

    private Integer id;
    private String name;
    private String location;
    private String description;
    private String schedule;
    private String imageUrl;

    private String ownerFullName;
    private Integer ownerId;

}
```

```java
private MarketResponse toMarketResponse(Market market) {  2 usages  Robert Nitu
    return MarketResponse.builder()
            .id(market.getId())
            .name(market.getName())
            .description(market.getDescription())
            .location(market.getLocation())
            .description(market.getDescription())
            .schedule(market.getSchedule())
            .imageUrl(market.getImageUrl())
            .ownerFullName(market.getUser().getFullName())
            .ownerId(market.getUser().getId())
            .build();
}
```

Fig 3. 21 Clasa „MarketResponse” și funcția „toMarketResponse”

Extragerea tuturor informațiilor în legătură cu piețele înscrise în aplicație se face utilizând adresa ,,http://localhost:8080/api/markets/all”, aceasta neavând niciun parametru. Conform Fig.

3.22, funcția apelată în cazul acestei rute este definită cu numele „getAllMarkets”. Această funcție salvează în variabila „markets” care este de tipul listă de variabile de tip „Market”, acest tip de variabilă este modelul tabelei cu numele „markets”, toate intrările din baza de date din tabelul cu același nume. Dacă nu au fost găsite intrări, metoda „findAll” definită în mod automat prin intermediul Java JPA, va returna o listă goală. În cazul unei liste goale înseamnă că nu sunt intrări, astfel se va opri execuția funcției și se va returna un mesaj de eroare intuitiv. În celelalt caz, în care sunt intrări în tabela „markets”, acestea vor fi transformate într-o listă de tipul de date „MarketResponse” explicată mai sus.

Ultima operație în cadrul gestionării piețelor este cea de extragere de informații cu privire la o singură piață, aceasta se face utilizând adresa ,,http://localhost:8080/api/markets/”, urmată de un parametru de tip număr întreg, care face referire la ID-ul din baza de date al pieței respective. Conform Fig. 3.31, funcția definită cu numele „getMarketById” primește ca parametru o variabilă de tip număr întreg, care ulterior face o operație de găsire după ID în baza de date. Dacă nu există date specifice se va opri execuția funcției și va returna un mesaj de eroare specific. În cazul în care s-a găsit intrare cu ID-ul dat ca parametru, se va utiliza funcția „toMarketResponse” definită mai sus pentru a converti datele din baza de date în tipul „MarketResponse”.

```
public List<MarketResponse> getAllMarkets() {  1 usage  Robert Nitu
    List<Market> markets = marketRepository.findAll();

    if (markets.isEmpty()) {
        throw new EntityNotFoundException("No markets registered!");
    }

    return markets
            .stream()  Stream<Market>
            .map(this::toMarketResponse)  Stream<MarketResponse>
            .collect(Collectors.toList());
}

public MarketResponse getMarketById(int id) {  1 usage  Robert Nitu
    return marketRepository.findById(id)  Optional<Market>
            .map(this::toMarketResponse)  Optional<MarketResponse>
            .orElseThrow(() -> new EntityNotFoundException("No market found with ID:: " + id));
}
```

Fig 3. 22 Funcțiile „getAllMarkets” și „getMarketById”

3.4.4 Gestionarea tarabelor din cadrul piețelor

Operațiile HTTP pentru gestionarea tarabelor din cadrul piețelor sunt de trei tipuri, una este de tipul POST, aceasta fiind metoda de adăugare unei tarabe în cadrul unei piețe pe baza unor verificări, a doua fiind de tipul GET, unde se extrag informațiile din baza de date cu privire la tarabele existente în tabela „stalls”, iar ultima fiind de tipul PUT, unde se actualizează stocul de produse din cadrul tarabei.

Pentru a accesa acest serviciu este nevoie de o altă rută specială, mai exact ruta „http://localhost:8080/api/stalls”, urmată ulterior de tipul de operație dorit. Conform Fig. 3.23, ruta de „stalls” are 4 metode.

Prima operație este cea de adăugare a unui stall (tarabă) în baza de date. Aceasta este accesibilă la adresa „http://localhost:8080/api/stall/add”, urmată de un obiect de tip „Stall”, definit conform descrierii bazei de date descries mai sus. Funcția definită „addStall” conform Fig. 3.24, are rolul de a adăuga o intrare nouă în tabela de „stalls” după anumite verificări. Prima verificare este de a verifica dacă utilizatorul există în tabela de „users”. În cazul în care nu este funcția se oprește și se returnează un mesaj de eroare intuitiv. Următoarea verificare, dacă utilizatorul există, este de a verifica dacă există piața în tabelul „markets” pentru asignarea tarabei asupra pieței respective. În cazul în care se va încerca să se creeze o tarabă („stall”) în cadrul unei piețe („market”) inexistent, funcția se oprește și se returnează un mesaj de eroare intuitiv. Dacă ambele entități există se va crea o instanță de tip-ul modelului tabelei din baza de date a „stalls” și se va popular cu datele din obiectul dat ca parametru. După ce se va insera în baza de date, se va actualiza coloana de „stall_id” pentru utilizatorul pentru care s-a creat taraba.

```java
@RestController  Robert Nitu
@RequestMapping("stalls")
@RequiredArgsConstructor
@Tag(name = "Stall")
public class StallController {

    private final StallService stallService;

    @PostMapping("/add")  Robert Nitu
    public ResponseEntity<?> addStall(@RequestBody Stall stall) {
        stallService.addStall(stall);
        return ResponseEntity.accepted().build();
    }

    @GetMapping("/{id}")  Robert Nitu
    public ResponseEntity<StallResponse> getStall(@PathVariable("id") Integer id) {
        return ResponseEntity.ok(stallService.getStallById(id));
    }

    @GetMapping("/stalls-by-market/{id}")  Robert Nitu
    public ResponseEntity<List<StallResponse>> getStallsByMarketId(@PathVariable("id") Integer id) {
        return ResponseEntity.ok(stallService.getAllStallsByMarketId(id));
    }

    @PutMapping("/update")  Robert Nitu
    public ResponseEntity<?> updateProducts(@RequestBody UpdateStallData updateStallData) {
        stallService.setNewProducts(updateStallData);
        return ResponseEntity.accepted().build();
    }
}
```

Fig 3. 23 Ruta „stalls”

```
public void addStall(Stall stall) {  1 usage   Robert Nitu
    User user = userRepository
            .findById(stall.getOwnerId())
            .orElseThrow(() -> new EntityNotFoundException("User not found"));

    Market market = marketRepository
            .findById(stall.getMarketPlacedId())
            .orElseThrow(() -> new EntityNotFoundException("Market not found"));

    Stall finalStall = new Stall();
    finalStall.setName(stall.getName());
    finalStall.setDescription(stall.getDescription());
    finalStall.setImageUrl(stall.getImageUrl());
    finalStall.setProducts(stall.getProducts());
    finalStall.setSchedule(stall.getSchedule());

    finalStall.setMarket(market);
    finalStall.setUser(user);

    stallRepository.save(finalStall);

    user.setStall(finalStall);
    userRepository.save(user);
}
```

Fig 3. 24 Funcția „addStall”

Următoarele operații sunt de tipul GET, mai precis pentru extragere a tuturor tarabelor din cadrul unei piețe și de extras strict pentru o singură tarabă. Prima operație, de extragere a tuturor tarabelor din cadrul unei piețe este accesibilă la adresa „http://localhost:8080/api/stalls/stalls-by-market/” urmat de o variabilă de tip număr întreg ce reprezintă ID-ul pieței (market-ului) din care acestea fac parte. În ambele cazuri se va returna ori un obiect, ori o listă de obiecte de tipul „StallResponse”. Acest obiect este construit ca o clasa simplă de Java, având tag-urile de @Getter și @Setter pentru a putea interacționa într-un mod mai ușor cu câmpurile acestora, așa cum reiese din Fig. 3.25. Pentru ca datele să fie transformate în tipul de date „StallResponse”, a fost creată funcția „toStallResponse”, conform Fig. 3.25, care transformă datele de tipul „Stall” în tipul de date prezentat anterior. Conform Conform Fig. 3.26, funcția definită „getAllStalsByMarketId” extrage din baza de date toate intrările din tabela „stalls”, iar apoi aplică o condiție de tipul WHERE, pentru a filtra doar tarabele ce sunt în cadrul pieței cu ID-ul dat ca parametru, iar apoi sunt transformate în tipul de date cerut, mai exact având tipul „StallResponse”. În variabila „stallls” este salvată o listă de tipul de date precizat anterior, iar o ultimă verificare este de a vedea dacă lista respectivă e goală, asta însemnând că nu există tarabe în cadrul pieței cu ID-ul dat ca parametru. În acest caz se oprește execuția funcției și se returnează un mesaj de eroare intuitiv. În celelalt caz, se returnează lista cu tarabele având tipul de date simplificat.

Ultima operație de tipul GET este de extragere a unei singure tarabe, aceasta este accesibilă utilizând adresa „http://localhost:8080/api/stalls/”, urmată de o variabilă de tip număr întreg. Funcția definită „getStallById” conform Fig. 3.26 va face o operație de căutare utilizând metoda „findById” implementată de către service-ul Java JPA, iar în cazul în care există o intrare în tabelul de „stalls” cu ID-ul dat ca parametru, se va construi obiectul de tip „StallResponse” și se va returna. În caz

contrar, dacă nu există nicio intrare cu ID-ul respectiv, se va opri execuția funcției și se va returna un mesaj de eroare intuitiv.

```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class StallResponse {

    private Integer id;
    private String name;
    private String description;
    private String schedule;
    private String imageUrl;
    private String products;

    private String ownerFullName;
    private Integer ownerId;

    private String marketName;
    private Integer marketId;

}
```

```java
private StallResponse toStallResponse(Stall stall) { 2 usages  Robert Nitu
    return StallResponse.builder()
            .id(stall.getId())
            .name(stall.getName())
            .description(stall.getDescription())
            .schedule(stall.getSchedule())
            .imageUrl(stall.getImageUrl())
            .products(stall.getProducts())
            .ownerFullName(stall.getUser().getFullName())
            .ownerId(stall.getUser().getId())
            .marketName(stall.getMarket().getName())
            .marketId(stall.getMarket().getId())
            .build();
}
```

Fig 3. 25 Clasa „StallResponse” și funcția „toStallResponse”

```java
public List<StallResponse> getAllStallsByMarketId(Integer marketId) {  1 usage  Robert Nitu
    List<StallResponse> stalls = stallRepository.findAll() List<Stall>
            .stream() Stream<Stall>
            .filter( Stall stall -> stall.getMarket().getId().equals(marketId))
            .map(this::toStallResponse) Stream<StallResponse>
            .toList();

    if (stalls.isEmpty()) {
        throw new EntityNotFoundException("No stalls registered!");
    }

    return stalls;
}

public StallResponse getStallById(Integer id) {  1 usage  new *
    return stallRepository.findById(id) Optional<Stall>
            .map(this::toStallResponse) Optional<StallResponse>
            .orElseThrow(() -> new EntityNotFoundException("No stall found with ID:: " + id));
}
```

Fig 3. 26 Funcțiile „getAllStallsByMarketId” și „getStallById”

Ultima operație HTTP este cea de PUT, mai exact cea de actualizare a produselor din cadrul unei piețe. Această operație este accesibilă folosind adresa „http://localhost:8080/api/stalls/update”, urmat de un parametru de tip „UpdateStallData”. Conform Fig. 3.27, clasa „UpdateStallData” este folosită pentru actualizarea produselor din cadrul unei tarabe (stall). Valorile sunt de tipul număr întreg pentru ID-ul din baza de date a tarabei, iar cealaltă de tipul String pentru produsele actualizate.

Conform Fig. 3.27, funcția definită „setNewProducts” actualizează coloana de „products” (produse) din cadrul pieței cu ID-ul dat ca parametru în tipul de date precizat anterior. Prima operație este de a extrage taraba din tabelul „stalls” din baza de date folosind metoda „findById”, definită de către service-ul Java JPA. În cazul în care intrarea în baza de date nu a fost găsită, se va opri execuția funcției și se va returna un mesaj de eroare intuitiv. Dacă intrarea în baza de date a fost găsită, înseamnă că taraba există și se poate actualiza lista de produse din cadrul tarabei. Utlimul pas este de setare al noilor produse cu ajutorul tag-urilor specifice Java, anume @Getter și @Setter, puse în clasele de „UpdateStallData” și modelul de tabelă al „stalls” precizate în cadrul descrierii bazei de date.

```java
@Getter
@Setter
public class UpdateStallData {

    private Integer id;
    private String products;

}
```

```java
public void setNewProducts(UpdateStallData updateStallData) {  1 usage  Robert Nitu
    Stall stall = stallRepository
            .findById(updateStallData.getId())
            .orElseThrow(() -> new EntityNotFoundException("Stall not found"));

    stall.setProducts(updateStallData.getProducts());
    stallRepository.save(stall);
}
```

Fig 3. 27 Clasa „UpdateStallData” ș funcția „setNewProducts”

3.4.5 Comenzile utilizatorilor

Operațiile HTTP pentru comenzile utilizatorilor sunt de trei tipuri, una dintre ele este de tipul POST, aceasta fiind metoda de adăugare a unei comenzi pe baza unor verificări în tabela „orders” din baza de date, a doua fiind de tipul GET pentru a extrage informații din aceași tabelă cu privire la comenzile plasate de către utilizator în funție de scenariu, iar ultima fiind de tipul PUT, aceasta actualizând status-ul unei comenzi plasate.

Pentru a accesa acest serviciu este nevoie de o rută specială, mai specific adăugând la finalul adresei „http://localhost:8080/orders/” operația dorită. Conform Fig. 3.28, ruta „oders” are 4 metode.

Prima operație este cea de adăugare a unei comenzi în tabela specifică. Această operație este valabilă folosind adresa „http://localhost:8080/orders/add”, urmată de un parametru de tipul modelului tabelei „Orders”, conform descrierii bazei de date descrise mai sus. Funcția definită „addOrder” conform Fig. 3.29, verifică în primă fază dacă există utilizatorul care încearcă să introducă o nouă inserare în tabela „orders”. Dacă utilizatorul nu există se va opri execuția funcție și se va ret urna un mesaj de eroare intuitiv. Următoarea verificare este dacă există intrare în tabela „stalls” pentru a opera ulterior pe instanța din variabila „stall”. În cazul în care nu există intrare în tabelă se va opri execuția funcției și se va returna un mesaj de eroare intuitiv. Următorul pas este de a modifica din tabela „Stalls” noile produse după ce se validează comanda. Comanda este validată în momentul în care se găsesc cele două intrări, de utillizator și tarabă prezentate anterior. Pentru a face acest lucru a fost nevoie ca produsele din cadrul unei tarabe (stall) sa fie salvate sub formă de String (text) care să fie un obiect transformat sub formă de String, care ulterior pe partea de server de back-end

se face operația de deserializare a unui string cu ajutorul service-ului Gson din librăria Java. Conform Fig. 3.30, clasa „ProductData” reține informații despre ID-ul produsului de tip număr întreg, name (nume) produsului de tip String, quantity (cantitate) de tip String și price (preț) de tip număr cu zecimale. Prin intermediul funcției „toProductDataArray” prezentate în Fig. 3.30, se realizează operația de deserializare a valorii de tip String dat ca parametru și returnează o listă de tipul „ProductData”, pe care se pot face operații. Conform funcției „addOrder”, în variabila „orderProductData” se stochează sub formă deserializată de tip listă de „ProductData”, produsele din cadrul comenzii create de către utilizator, iar în variabila „stallProductsData” se stochează sub formă deserializată de tip listă de „ProductData”, produsele deja existente în tabela „stalls”. Având aceste valori într-o formă operațională se realizează operații de scădere a cantității produselor din cadrul listei deserializate de comandă a utilizatorului cu ce este deja în stocul tarabei. Astfel, cu o instrucțiune repetitivă de tipul for, se ia fiecare element din lista de cumpărături a utilizatorului și se scade cantitatea din lista de produse din cadrul tarabei. Pentru a evita valorile negative în urma scăderilor, a fost folosită metoda „max” din cadrul libăriei „Math”, care oferă maximul dintre diferența cantităților și valoarea 0. Odată făcută această operație de actualizare a noului stoc de produse al tarabelor, se crează o instanță nouă de tipul „Order” și se populează cu datele primite ca parametru, iar variabila de tip LocalDate sub numele de „orderDate” are valoarea implicită ca fiind data și ora când se creează instanța respectivă. La final se setează cheile străine pentru comandă, mai exact se setează coloanele „stall_id” cu instanța de tarabă (stall) și „user_id” cu instanța de user (utilizator). Ultimul pas fiind actualizarea instanței de tarabă, mai exact produsele calculate în urma scăderii stocului.

```java
@RestController  Robert Nitu
@RequestMapping("orders")
@RequiredArgsConstructor
@Tag(name = "Order")
public class OrderController {

    private final OrderService orderService;

    @PostMapping("/add")  Robert Nitu
    public ResponseEntity<?> addOrder(@RequestBody Order order) {
        orderService.addOrder(order);
        return ResponseEntity.accepted().build();
    }

    @GetMapping("/stall/{id}")  Robert Nitu
    public ResponseEntity<List<OrderResponse>> getOrdersFromStallId(@PathVariable("id") Integer id) {
        return ResponseEntity.ok(orderService.getAllOrdersFromStallId(id));
    }

    @GetMapping("/user/{id}")  Robert Nitu
    public ResponseEntity<List<OrderResponse>> getOrdersFromUserId(@PathVariable("id") Integer id) {
        return ResponseEntity.ok(orderService.getAllOrdersFromUserId(id));
    }

    @PutMapping("/update")  Robert Nitu
    public ResponseEntity<?> updateOrder(@RequestBody UpdateOrderData updateOrderData) {
        orderService.updateOrder(updateOrderData);
        return ResponseEntity.accepted().build();
    }

}
```

Fig 3. 28 Ruta „orders”

```
public void addOrder(Order order) {  1 usage   Robert Nitu
    User user = userRepository
            .findById(order.getRequesterId())
            .orElseThrow(() -> new EntityNotFoundException("User not found"));

    Stall stall = stallRepository
            .findById(order.getStallId())
            .orElseThrow(() -> new EntityNotFoundException("Stall not found"));

    List<ProductData> orderProductsData = toProductDataArray(order.getProducts());
    List<ProductData> stallProductsData = toProductDataArray(stall.getProducts());

    for (ProductData orderedProduct : orderProductsData) {
        Integer orderedId = orderedProduct.getId();
        int orderedQty = Integer.parseInt(orderedProduct.getQuantity());

        for (ProductData stallProduct : stallProductsData) {
            if (stallProduct.getId().equals(orderedId)) {
                int currentQty = Integer.parseInt(stallProduct.getQuantity());
                int newQty = Math.max(currentQty - orderedQty, 0);
                stallProduct.setQuantity(String.valueOf(newQty));
                break;
            }
        }
    }

    Order finalOrder = new Order();
    finalOrder.setProducts(order.getProducts());
    finalOrder.setAddress(order.getAddress());
    finalOrder.setPrice(order.getPrice());
    finalOrder.setStatus(order.getStatus());
    finalOrder.setOrderDate(LocalDateTime.now());

    finalOrder.setStall(stall);
    finalOrder.setUser(user);

    orderRepository.save(finalOrder);

    stall.setProducts(toProductDataString(stallProductsData));
    stallRepository.save(stall);
}
```

Fig 3. 29 Funcția „addOrder”

```
@Getter
@Setter
public class ProductData {

    private Integer id;
    private String name;
    private String quantity;
    private Float price;

}
```

```
private List<ProductData> toProductDataArray(String data) {  2 usages   Robert Nitu
    Gson gson = new Gson();
    Type listType = new TypeToken<List<ProductData>>() {}.getType();   Robert Nitu
    return gson.fromJson(data, listType);
}

private String toProductDataString(List<ProductData> data) {  1 usage   Robert Nitu
    Gson gson = new Gson();
    Type listType = new TypeToken<List<ProductData>>() {}.getType();   Robert Nitu
    return gson.toJson(data, listType);
}
```

Fig 3. 30 Clasa „ProductData” și funcțiile „toProductDataArray”, „toProductDataString”

Următoarele operații sunt de tipul GET, acestea sunt folosite pentru a extrage informații din baza de date în funcție de metoda apelată. În ambele scenarii se va returna o listă de tipuri de date conform Fig. 3.31. Clasa „OrderResponse” conține informații simplificate despre comenzile făcute, această clasă are atribuiți tag-urile specifice Java, mai exact @Getter și @Setter, unde se poate realiza mult mai ușor o instanță de astfel de clasă și se pot extrage câmpuri cu ușurință. Câmpurile prezente sunt tipuri de date prezente într-o instanță de tipul „Order”, folosind cheile străine cu privire la user (utilizator), market (piață) și stall (tarabă). Astfel, conform Fig. 3.31, funcția definită cu numele „toOrderResponse” transformă parametrul de tip model de tabelă „Order” într-un tip de date „OrderResponse”, extrăgând cu ajutorul tag-ului @Getter prezent în clasa „Order”, prezentată în descrierea bazei de date și populând instanța nouă de tipul „OrderResponse”.

Prima operație care are la bază tipul GET, este prezentată conform Fig. 3.32 prin funcția cu numele „getAllOrdersFromStallId”. Pentru a accesa această funcție este nevoie de a folosi adresa „http://localhost:8080/orders/stall/”, urmată de un parametru cu numele ID de tipul număr întreg.

Funcția va folosi metoda de „findAll” specifică Java JPA, unde va lua toate instanțele de comenzi existente în tabela „Orders”, iar apoi aplicând o condiție de tipul WHERE, se vor filtra datele în funcție de coloana „stall_id”. În filtrare se vor verifica ID-urile tarabei din cadrul comenzii dacă sunt egale cu valoarea ID dat ca parametru funcției. După ce se vor termina de filtrat, acestea vor fi convertite în tipul de date „OrderResponse” prezentat mai sus, iar apoi în ultimă fază va fi returnată lista de tipul „OrderResponse. În cazul în care filtrarea nu returnează niciun element, se va returna o listă goală.

A două operație, și ultima de tipul GET, este prezentată în Fig. 3.32 prin funcția cu numele „getAllOrdersFromUserId”. Pentru a accesa această funcție este nevoie de a folosi adresa „http://localhost:8080/api/orders/user/”, urmată de un parametru cu numele ID de tipul număr întreg. Funcția va folosi metoda de „findAll” specifică Java JPA, unde va lua toate instanțele de comenzi existente în tabela „Orders”, iar apoi aplicând o condiție de tipul WHERE, se vor filtra datele în funcție de coloana „user_id”. În filtrare se vor verifica ID-urile tarabei din cadrul comenzii dacă sunt egale cu valoarea ID dat ca parametru funcției. După ce se vor termina de filtrat, acestea vor fi convertite în tipul de date „OrderResponse” prezentat mai sus, iar apoi în ultimă fază va fi returnată lista de tipul „OrderResponse. În cazul în care filtrarea nu returnează niciun element, se va returna o listă goală.



```java
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Builder
public class OrderResponse {

    private Integer id;
    private String products;
    private String address;
    private Float price;
    private Integer status;

    private LocalDateTime orderDate;
    private LocalDateTime acceptedDate;
    private LocalDateTime deliveryDate;

    private String fullUserName;

    private String marketName;
    private Integer marketId;

    private String stallName;
    private Integer stallId;

}
```

```java
private OrderResponse toOrderResponse(Order order) {  2 usages  Robert Nitu
    return OrderResponse.builder()
            .id(order.getId())
            .products(order.getProducts())
            .address(order.getAddress())
            .price(order.getPrice())
            .status(order.getStatus())
            .orderDate(order.getOrderDate())
            .acceptedDate(order.getAcceptedDate())
            .deliveryDate(order.getDeliveryDate())
            .fullUserName(order.getUser().getFullName())
            .marketName(order.getStall().getMarket().getName())
            .marketId(order.getStall().getMarket().getId())
            .stallName(order.getStall().getName())
            .stallId(order.getStall().getId())
            .build();
}
```

Fig 3. 31 Clasa „OrderResponse” și funcția „toOrderResponse”

```
public List<OrderResponse> getAllOrdersFromStallId(Integer stallId) {  1 usage  Robert Nitu
    return orderRepository.findAll() List<Order>
            .stream() Stream<Order>
            .filter( Order order -> order.getStall().getId().equals(stallId))
            .map(this::toOrderResponse) Stream<OrderResponse>
            .toList();
}

public List<OrderResponse> getAllOrdersFromUserId(Integer userId) {  1 usage  Robert Nitu
    return orderRepository.findAll() List<Order>
            .stream() Stream<Order>
            .filter( Order order -> order.getUser().getId().equals(userId))
            .map(this::toOrderResponse) Stream<OrderResponse>
            .toList();
}
```

Fig 3. 32 Funcțiile „getAllOrdersFromStallId” și „getAllOrdersFromUserId”

Ultima operație din cadrul acestei rute, este cea de PUT. Pentru a accesa această operație este nevoie de adresa „http://localhost:8080/api/orders/update/”, urmată de un parametru de tipul „UpdateOrderData”, prezentată în Fig. 3.33. Acest tip de date are în componență două variabile, mai anume una de tip număr întreg sub numele de id, iar cealaltă de tip număr întreg sub numele de status. Conform funcției numite „updateOrder” prezentate în Fig. 3.33, în primă fază se caută dacă există o instanță cu ID-ul precizat în variabila din parametru din tabela „Orders”. Acest lucru se realizează folosind funcția „findById” specifică Java JPA. Dacă comanda nu există în tabela specifică se oprește executarea funcției și se returnează un mesaj de eroare specific. Următorul pas este de a seta status-ul comenzii folosind funcția de „setStatus” cu ajutorul tag-ului @Setter din cadrul clasei de „Order”, prezentat în descrierea bazei de date. Ulterior, în funcție de status-ul primit ca parametru, se vor actualiza coloanele de tipul LocalDate astfel: dacă noua valoare din variabila „status” are valoarea IN_DELIVERY (1), se va actualiza coloana „acceptedDate” cu data și ora curentă, iar dacă noua valoare din variabila „status” are valoarea DELIVERED (2), se va actualiza coloana „deliveryDate” cu data și ora curentă. În final se actualizează instanța de „order” în baza de date.

```
public void updateOrder(UpdateOrderData updateOrderData) {  1 usage  Robert Nitu
    Order order = orderRepository
            .findById(updateOrderData.getId())
            .orElseThrow(() -> new EntityNotFoundException("Order not found"));

    order.setStatus(updateOrderData.getStatus());
    if (updateOrderData.getStatus() == OrderStatus.IN_DELIVERY.ordinal())
        order.setAcceptedDate(LocalDateTime.now());
    if (updateOrderData.getStatus() == OrderStatus.DELIVERED.ordinal())
        order.setDeliveryDate(LocalDateTime.now());
    orderRepository.save(order);
}
```

```
@Getter
@Setter
public class UpdateOrderData {

    private Integer id;
    private Integer status;

}
```

```
public enum OrderStatus {
    PENDING,
    IN_DELIVERY,
    DELIVERED,
}
```

Fig 3. 33 Clasa „UpdateOrderData”, enum-ul „OrderStatus” și funcția „updateOrder”

3.5 Realizarea front-end-ului în Angular19

3.5.1 Arhitectura componentelor

În Angular, structura elementelor este împărțită în multiple componente. Conform Fig. 3.34, o componentă este alcătuită din 3 fișiere. Primul fișier fiind cel de HTML, unde se află toată partea vizuală a componentei. În acest fișier se pot afișa variabile prin intermediul operației de interpolare, specifică Angu, folosind operatorii „{{ }}", iar în mijloc fiind adăugată variabila dorită în afișare. Tot în acest fișier se pot folosi numeroase operații specifice, cum ar fi afișarea unui div doar în anumite condiții, adăugând tag-ul de „ngIf" urmată de condiția dorită. O altă operație ar fi folosirea buclei repetitive de tip „for" în cadrul paginii HTML, acest lucru este posibil prin intermediul tag-ului „ngFor", această funcție fiind extrem de utilă în alcătuirea codului, fiind mult mai curat și ușor de citit. Următorul fișier este cel de stilizare, mai anume fișierul CSS. Este un fișier normal de stilizare folosit pentru fișierul HTML prezentat anterior. Ultimul fișier și cel mai important, este cel de tipul TypeScript. Acest fișier este o clasă de TypeScript care are tag-ul specific Angular de @Component. În cadrul acestui tag sunt definite niște valori, mai anume este definit selectorul care poate fi folosit în alte componente, import-urile de module specifice Angular, dar și importurile prezentate anterior pentru stilizare și fișierul HTML. În această clasă de TypeScript se pot implementa și alte clase, cele mai folosite fiind „OnInit" și „OnDestroy". Acestea sunt niște funcții predefinite de Angular care se apelează când se inițializează, respectiv când se distruge componenta, pentru a putea face diferite operații.

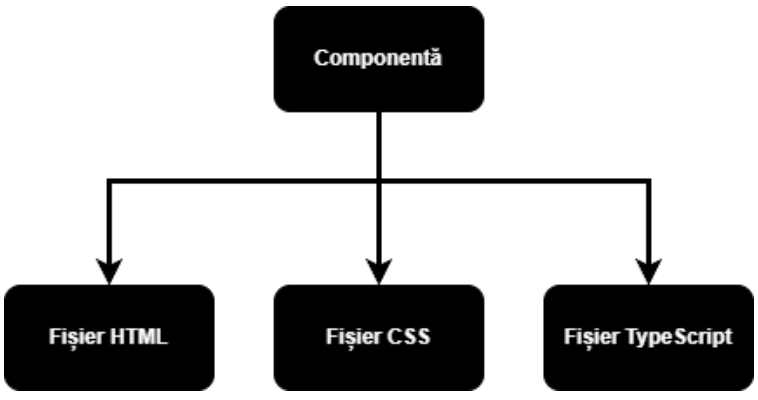


Fig 3. 34 Arhitectură componente

3.5.2 Implementarea service-urilor

Service-urile în Angular sunt create și folosite pentru a interacționa și a naviga cu datele din aplicație dintr-o componentă în alta. Conform Fig. 3.35, un serviciu poate fi folosit de câte componente are în componență aplicația în Angular. Aceste service-uri pot fi generate folosind unelte

precum OpenApi, cum este în cazul acestei aplicații, care pe baza codului de pe back-end generează un serviciu complex care are la bază toate funcțiile necesare apelării server-ului de back-end, cât și tipurile de date folosite. Folosind o unealtă de genul scade riscul de inconsistență a codului pe partea de front-end și pe partea de back-end, iar totodată scalabilitatea crește fiind ușor de implementat și de gestionat. Un alt exemplu de serviciu este cel din Fig. 3.36. Service-ul „TokenService” este responsabil cu status-ul utilizatorului cu privire la logarea pe site. Salvarea datelor se face la nivel de browser, folosind unealta de LocalStorage, existentă pe orice browser. În momentul în care utilizatorul s-a logat cu succes, se vor popula variabilele prezentate în figură, iar în unele componentese va verifica status-ul de logare al utilizatorului, utilizând funcția „isAlreadyLoggedIn” din cadrul service-ului prezentat. Această funcție extrage din „LocalStorage” variabila de tip „token”, care este setată în momentul în care utilizatorul se loghează, și returnează „true” sau „false” dacă aceasta există. De altfel, funcția „watchLoggedStatus”, verifică în orice moment dacă utilizatorul încă este logat pe aplicație, aceasta fiind apelată într-o altă componentă prezentată în cele ce urmează.

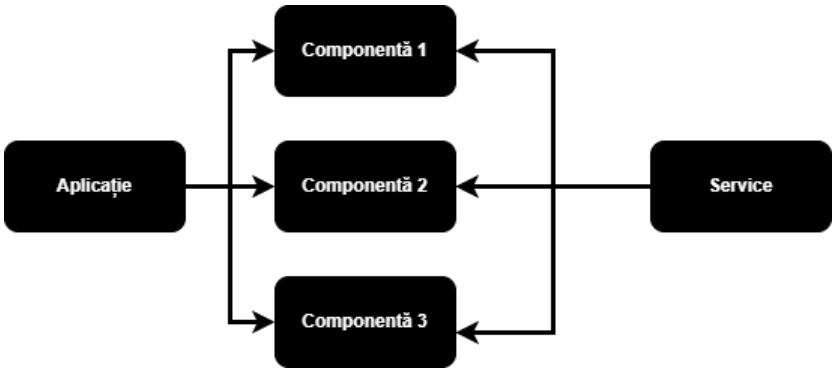
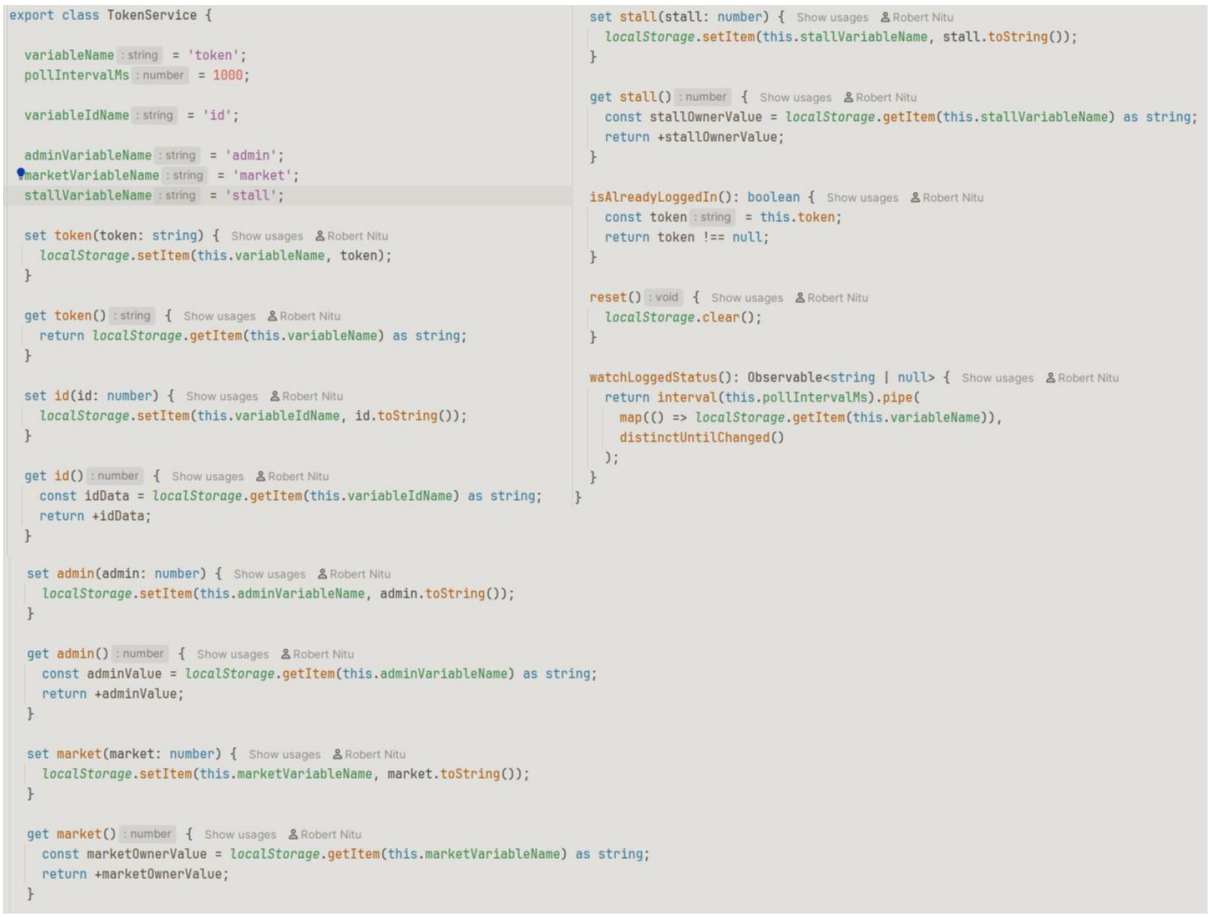


Fig 3. 35 Arhitectură service-uri



```
export class TokenService {

  variableName : string  = 'token';
  pollIntervalMs : number  = 1000;

  variableIdName : string  = 'id';

  adminVariableName : string  = 'admin';
  marketVariableName : string  = 'market';
  stallVariableName : string  = 'stall';

  set token(token: string) {  Show usages  Robert Nitu
    localStorage.setItem(this.variableName, token);
  }

  get token() : string  {  Show usages  Robert Nitu
    return localStorage.getItem(this.variableName) as string;
  }

  set id(id: number) {  Show usages  Robert Nitu
    localStorage.setItem(this.variableIdName, id.toString());
  }

  get id() : number  {  Show usages  Robert Nitu
    const idData = localStorage.getItem(this.variableIdName) as string;
    return +idData;
  }

  set admin(admin: number) {  Show usages  Robert Nitu
    localStorage.setItem(this.adminVariableName, admin.toString());
  }

  get admin() : number  {  Show usages  Robert Nitu
    const adminValue = localStorage.getItem(this.adminVariableName) as string;
    return +adminValue;
  }

  set market(market: number) {  Show usages  Robert Nitu
    localStorage.setItem(this.marketVariableName, market.toString());
  }

  get market() : number  {  Show usages  Robert Nitu
    const marketOwnerValue = localStorage.getItem(this.marketVariableName) as string;
    return +marketOwnerValue;
  }

  set stall(stall: number) {  Show usages  Robert Nitu
    localStorage.setItem(this.stallVariableName, stall.toString());
  }

  get stall() : number  {  Show usages  Robert Nitu
    const stallOwnerValue = localStorage.getItem(this.stallVariableName) as string;
    return +stallOwnerValue;
  }

  isAlreadyLoggedIn(): boolean {  Show usages  Robert Nitu
    const token : string  = this.token;
    return token !== null;
  }

  reset() : void  {  Show usages  Robert Nitu
    localStorage.clear();
  }

  watchLoggedStatus(): Observable<string | null> {  Show usages  Robert Nitu
    return interval(this.pollIntervalMs).pipe(
      map(() => localStorage.getItem(this.variableName)),
      distinctUntilChanged()
    );
  }
}
```

Fig 3. 36 Service-ul „TokenService”

3.5.3 Implementarea rutărilor către componente

Pentru a accesa componentele unei aplicații Angular, este nevoie de adăugarea unui tag în fișierul de bază de tip HTML, anume tag-ul „<router-outlet></router-outlet>”. Acest tag este responsabil de navigarea între componente fără a se schimba ruta de bază a paginii. Cu alte cuvinte, aplicația este una de tipul OnePage specifică Angular, iar totul se realiează cu ajutorul acestui tag. Rutele sunt definite conform Fig. 3.37, rutele sunt o variabilă de tipul de date „Routes”, care au în componență un câmp numit „path”, care reprezintă calea dorită, „component”, care reprezintă componenta care trebuie să se afișeze în funcție de calea introdusă, iar un câmp care nu este obligatoriu este cel de „pathMatch”, care reprezintă cazul de bază în care utilizatorul introduce o altă rută care nu este prezentă în constanta prezentată, astfel va exista mereu un caz de bază în care îl va redirecționa mereu în componenta principală.



```
export const routes: Routes = [  Show usages  Robert Nitu
  {
    path: 'home',
    component: HomePageComponent,
  },
  {
    path: 'login',
    component: LoginComponent,
  },
  {
    path: 'register',
    component: RegisterComponent,
  },
  {
    path: 'activate-account',
    component: ActivateAccountComponent,
  },
  {
    path: 'profile',
    component: ProfileComponent,
  },
  {
    path: 'markets',
    component: MarketsComponent,
  },
  {
    path: 'stalls/:id',
    component: StallsComponent,
  },
  {
    path: 'stall/:id',
    component: StallComponent,
  },
  {
    path: 'requests',
    component: RequestsComponent,
  },
  {
    path: '',
    redirectTo: '/home',
    pathMatch: 'full',
  },
  {
    path: '**',
    redirectTo: '/home',
    pathMatch: 'full',
  },
];

@NgModule({  no usages  Robert Nitu
  imports: [RouterModule.forRoot(routes)],
})
export class AppRoutingModule {}
```

Fig 3. 37 Rutele aplicației Angular

3.5.4 Implementarea uneltei NGX-Translate

Pentru implementarea uneltei de translatare a aplicației, am folosit NGX-Translate[17]. Pentru acest lucru a fost nevoie de adăugarea unor fișiere în fișierul de TypeScript principal al aplicației, sub numele de „app.config.ts”. Conform Fig. 3.38, a trebuit creat directorul „i18n” în cadrul aplicației, unde vor fi stocate fișierele de tip JSON cu translatările specifice fișierului din care fac parte. Implementarea uneltei prezentate anterior a fost prin adăugarea în cadrul constantei „appConfig” a modulului „TranslateModule”. Pentru a avea acces în cadrul aplicației la acest serviciu, trebuie adăugat în fiecare clasa din componentele create, însă pentru a seta valorile implicite din fișierele prezentate anterior, trebuie în fișierul de bază de tip TypeScript, anume „app.component.ts”, setate valorile din fișierele de tip JSON și apoi setarea cu limba implicită română. Pentru a putea afișa text-ul în limba setată va trebui să injectăm în fiecare componentă service-ul „TranslateService”, conform Fig. 3.39. Pentru a translata texte, este nevoie de a interpola în cadrul fișierului HTML o valoare de tip text, care să fie găsită în fișierul JSON anterior creat, un exemplu de folosire al service-ului de translatare: „{{ ‘home-page.first-paragraph’ | translate }}”.

```
const httpLoaderFactory: (http: HttpClient) => TranslateHttpLoader = (http: HttpClient) : TranslateHttpLoader =>
  new TranslateHttpLoader(http, prefix: './i18n/', suffix: '.json');

export const appConfig: ApplicationConfig = {  Show usages  Robert Nitu
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    importProvidersFrom(HttpClient),
    provideHttpClient(),
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideHttpClient(),
    importProvidersFrom([TranslateModule.forRoot({
      loader: {
        provide: TranslateLoader,
        useFactory: httpLoaderFactory,
        deps: [HttpClient],
      },
    })])
  ]
};
```

Fig 3. 38 Fișierul „app.config.ts”

```
import roLanguage from '../i18n/ro.json';
import enLanguage from '../i18n/en.json';

@Component({  Show usages  Robert Nitu
  selector: 'app-root',
  imports: [RouterOutlet, NavigationBarComponent],
  templateUrl: './app.component.html',
  styleUrl: './app.component.css'
})
export class AppComponent {

  constructor(  no usages  Robert Nitu
    private translate: TranslateService
  ) {
    this.translate.setTranslation( lang: 'ro', roLanguage);
    this.translate.setTranslation( lang: 'en', enLanguage);

    this.translate.use( lang: 'ro');
  }

}
```

Fig 3. 39 Fișierul „app.component.ts”

3.5.5 Componenta Principală

Componenta principală are în structura sa o referință către componenta de bară de navigare (navigation bar), care va fi vizibilă indiferent de ruta la care este utilizatorul, fiind necesară pentru interacțiunea cu aplicația. În cadrul acestei componente s-a adăugat selectorul de „router-outlet” unde se vor afișa componentele în funcție de ruta la care se află utilizatorul.

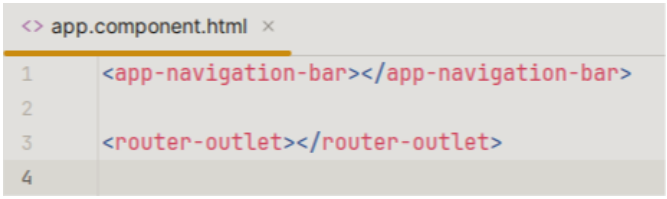


Fig 3. 40 Componenta principală

3.5.6 Componenta Bara de navigare

Această componentă este menită să fie afișată pe tot parcursul interacțiunii cu aplicația, aceasta fiind necesară pentru navigarea pe site. Bara de navigare este construită în mod dinamic, aceasta cu ajutorul anotaților specifice Angular va afișa anumite butoane în funcție de status-ul utlizatorului, dacă acesta este logat sau nu. Conform Fig. 3.41, pagina HTML are un selector specific de tip „nav” care a fost decorată cu elemente de stilizare din librăria Boostrap[10]. Bara de navigare este împărțită în două, prima parte conține butoanele de navigare care sunt vizibile în orice caz, chit că utilizatorul este logat sau nu în aplicație, aceasta este formată din liste cu ajutorul tag-urilor „ul” și „li”. Au fost adăugate texte pentru ca utilizatorul să știe unde să navigheze, acestea fiind traduse cu ajutorul uneltei de translatare prezentate în punctele de mai sus. În cadrul elemtelor din listă, a fost adăugat un element HTML de tipul „a”, care la apăsarea acestuia se va apela funcția prezentă în fișierul TypeScript, numită „goToPage”. Această funcție conform fig. 3.42, are rolul de a naviga către celelalte componente în funcție de parametrul dat. Posibilitățile de rutare sunt salvate în enum-ul „Pages”, iar la nivelul clasei „NavigationBarComponent” s-a construit o variabilă de tip „dicționar” (Record), care face o legătură între variabila prezentată anterior, „Pages” și ruta la care trebuie să se ducă. Astfel, în funcția „goToPage” se extrage ruta în funcție de parametrul „page” dat în codul HTML în funcție de elementul din listă selectat, iar cu ajutorul service-ului predefinit în librăria Angular se face rutarea folosind constructor-ul „route” astfel: „this.route.navigate([route])”.

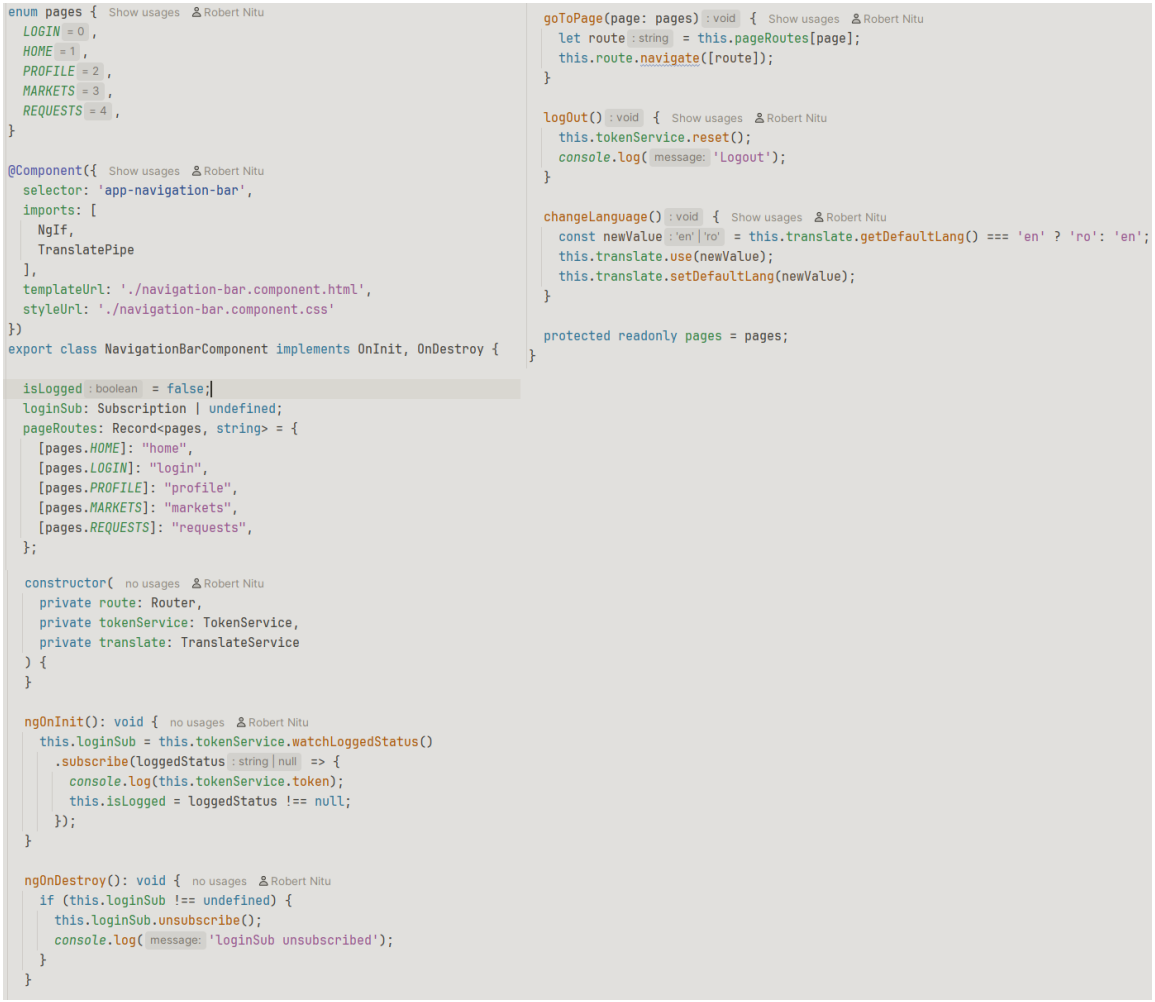
În cealaltă parte a bării de navigare sunt prezente butoane de control al contului, acestea sunt afișate în funcție de status-ul utilizatorului. Dacă acesta este logat pe aplicație, se va folosi container-ul construit pe baza variabilei „isLogged”, care este inițializată cu valoarea de „true” sau „false” în momentul în care aplicația pornește și este apelată componenta de bară de navigare. În cazul în care utilizatorul nu este logat va avea la dispoziție butonul de „sign-in” care îl va dute către componenta specifică logării. În cazul contrar, utilizatorul este logat și va avea la dispoziție butonul de „logout”, care îl va deconecta din contul cu care a fost logat, iar un alt buton este cel de vizualizare de profil, care îl va duce la componenta respectivă. Un buton comun situat în partea dreaptă a bării de navigare este cel de schimbare al limbii în care sunt afișate textele în cadrul aplicației. Acest buton la apăsare apelează funcția „changeLanguage”, care ulterior preia limba curentă setată de către utilizator cu ajutorul service-ului de translatare prezentat anterior, utilizând metoda „getDefaultLang” și aplică un operator terțial care aplică valoarea opusă, dacă are limba setată pe română, se va setal limba engleza și vice versa.

Pentru ca valoarea status-ului de logare al utilizatorului să fie mereu cea adevărată, în cadrul acestei componente s-a adăugat în momentul inițializării o variabilă de tipul observabilă, care

urmarește în fiecare moment dacă s-a schimbat valoarea din variabila „token” din cadrul service-ului „TokenService”. Dacă utilizatorul apasă pe funcția de „logOut”, aplicația îl va deconecta, iar datele setate prin intermediul service-ului „TokenService” vor fi șterse, astfel în cadrul variabilei de tip observabil prezentate anterior va fi actualizată variabila „isLogged”, iar funcționalitățile disponibile vor fi restrânse. Totodată, la distrugerea componentei, în cazuri de închidere de browser, variabila de tip observabil „loginSub” trebuie să fie șterasă.



Fig 3. 41 Componentă bară navigare – HTML



```typescript
enum pages {  Show usages  Robert Nitu
  LOGIN = 0,
  HOME = 1,
  PROFILE = 2,
  MARKETS = 3,
  REQUESTS = 4,
}

@Component({  Show usages  Robert Nitu
  selector: 'app-navigation-bar',
  imports: [
    NgIf,
    TranslatePipe
  ],
  templateUrl: './navigation-bar.component.html',
  styleUrl: './navigation-bar.component.css'
})
export class NavigationBarComponent implements OnInit, OnDestroy {

  isLogged : boolean  = false;
  loginSub: Subscription | undefined;
  pageRoutes: Record<pages, string> = {
    [pages.HOME]: "home",
    [pages.LOGIN]: "login",
    [pages.PROFILE]: "profile",
    [pages.MARKETS]: "markets",
    [pages.REQUESTS]: "requests",
  };

  constructor(  no usages  Robert Nitu
    private route: Router,
    private tokenService: TokenService,
    private translate: TranslateService
  ) {
  }

  ngOnInit(): void {  no usages  Robert Nitu
    this.loginSub = this.tokenService.watchLoggedStatus()
      .subscribe(loggedStatus : string | null  => {
        console.log(this.tokenService.token);
        this.isLogged = loggedStatus !== null;
      });
  }

  ngOnDestroy(): void {  no usages  Robert Nitu
    if (this.loginSub !== undefined) {
      this.loginSub.unsubscribe();
      console.log( message: 'loginSub unsubscribed');
    }
  }

  goToPage(page: pages) : void  {  Show usages  Robert Nitu
    let route : string  = this.pageRoutes[page];
    this.route.navigate([route]);
  }

  logOut() : void  {  Show usages  Robert Nitu
    this.tokenService.reset();
    console.log( message: 'Logout');
  }

  changeLanguage() : void  {  Show usages  Robert Nitu
    const newValue : 'en' | 'ro'  = this.translate.getDefaultLang() === 'en' ? 'ro': 'en';
    this.translate.use(newValue);
    this.translate.setDefaultLang(newValue);
  }

  protected readonly pages = pages;
}
```

Fig 3. 42 Componentă bară navigare – TypeScript

3.5.7 Componenta Înregistrare

Această componentă este responsabilă pentru înregistrarea utilizatorilor în cadrul aplicației. Construcția sa cuprinde multiple input-uri, tag-uri specifice HTML, stilizate cu ajutorul librăriei Bootstrap. Conform fig. 3.43, întreaga fereastră de înregistrare este încadrată într-un „div” ce are clasa de „card”, un design prestabilit preluat din librăria Bootstrap, iar in interior găsim un alt „div” ce are o condiție de afișare, mai exact dacă în variabila „errorMessages” au fost introduse valori, astfel se compară lungimea obiectului cu metoda „length”. Mesajele de eroare sunt afișate sub formă de listă neorganizată, folosind tag-urile „ul” și „li”. Următoarea parte din cadrul paginii HTML este completarea câmpurillor de input pentru înregistrarea contului. Input-urile au în componența sa tag-ul specific Angular „[(ngModel)]”, acesta salvează la fiecare modificare a input-ului valoarea text în variabila respectivă, astfel în momentul în care se apelează funcția de „register”, definită în fig. 3.44, valorile trebuiesc deja stocate în variabila denumită „registerRequest”. Butonul de înregistrare este vizibil doar dacă utilizatorul nu este logat pe pagină, astfel evitând situații de creere inutilă a conturilor.

Pe lângă input-urile de construire a contului, mai este prezent un buton de întoarcere la pagina de logare, prezentată în cele ce urmează.

Serviciile injectate în această componentă sunt: „router”, care face posibilă navigarea către pagina de logare atunci când se apasă pe butonul de „înapoi la logare”, „tokenService”, care verifică în cadrul funcției cu numele „isAlreadyLoggedIn” dacă utilizatorul este conectat sau nu, iar ultima fiind „authenticationService”, acest service este generat în mod automat de către unealta OpenApi, pe baza metodelor și rutelor de pe server-ul de back-end. În funcția de „register”, în primă fază variabila mesajelor de eroare se resetează, se atribuie un obiect gol „[]”, iar apoi se apelează metoda de „register” din cadrul „authService”, care este o metodă care returnează un observabil. Parametrul acestei funcții este obiecut construit pe baza input-ului din HTML. În cazul în care sunt erori pe partea de server de back-end, acestea vor fi extrase din variabila „err” din a doua ramură de execuție a observabilului, iar în cazul în care contul a fost înregistrat cu success se va realiza rutarea către pagina de activare de cont (activate account).

```
<div class="container py-5">
  <div class="card shadow-lg border-0 rounded-4 p-4 mx-auto register-container">
    <h3 class="text-center mb-3">{{ 'login-and-register.register' | translate }}</h3>
    <hr class="mb-4">

    <div *ngIf="errorMessages.length > 0" class="alert alert-danger">
      <ul class="mb-0">
        <li *ngFor="let message of errorMessages">{{ message }}</li>
      </ul>
    </div>

    <div class="mb-3">
      <label for="firstName" class="form-label">{{ 'login-and-register.first-name' | translate }}</label>
      <input [(ngModel)]="registerRequest.firstName"
             type="text"
             class="form-control rounded-3"
             id="firstName"
             placeholder="{{ query: 'login-and-register.first-name' | translate }}">
    </div>

    <div class="mb-3">
      <label for="lastName" class="form-label">{{ query: 'login-and-register.last-name' | translate }}</label>
      <input [(ngModel)]="registerRequest.lastName"
             type="text"
             class="form-control rounded-3"
             id="lastName"
             placeholder="{{ query: 'login-and-register.last-name' | translate }}">
    </div>

    <div class="mb-3">
      <label for="email" class="form-label">Email</label>
      <input [(ngModel)]="registerRequest.email"
             type="email"
             class="form-control rounded-3"
             id="email"
             placeholder="name@example.com">
    </div>

    <div class="mb-4">
      <label for="imageUrl" class="form-label">{{ query: 'login-and-register.image-url' | translate }}</label>
      <input [(ngModel)]="registerRequest.imageUrl"
             type="url"
             class="form-control rounded-3"
             id="imageUrl"
             placeholder="{{ query: 'login-and-register.image-url-placeholder' | translate }}">
    </div>

    <div class="text-center mb-4" *ngIf="registerRequest.imageUrl">
      <img src="{{registerRequest.imageUrl}}"
           alt="Profile preview"
           class="rounded-circle shadow-sm image-preview"/>
      <p class="small text-muted mt-2">{{ query: 'login-and-register.image-preview' | translate }}</p>
    </div>

    <div class="mb-4">
      <label for="password" class="form-label">{{ query: 'login-and-register.password' | translate }}</label>
      <input [(ngModel)]="registerRequest.password"
             type="password"
             class="form-control rounded-3"
             id="password"
             placeholder="{{ query: 'login-and-register.password' | translate }}">
    </div>

    <div class="d-flex justify-content-between">
      <button *ngIf="!isAlreadyLoggedIn()"
              (click)="register()"
              type="button"
              class="btn btn-primary px-4 py-2 rounded-pill shadow-sm">
        <i class="fas fa-user-plus me-2"></i> {{ query: 'login-and-register.register' | translate }}
      </button>

      <button (click)="goToLogin()" type="button" class="btn btn-outline-secondary px-4 py-2 rounded-pill">
        {{ query: 'login-and-register.back-to-login' | translate }}
      </button>
    </div>
  </div>
</div>
```

Fig 3. 43 Componentă înregistare – HTML



```
export class RegisterComponent {

  constructor(  no usages  Robert Nitu
    private router: Router,
    private authService: AuthenticationService,
    private tokenService: TokenService
  ) {
  }

  registerRequest: RegistrationRequest = { imageUrl: 'https://i.imgur.com/6jmGZxO.png' };
  errorMessages: Array<string> = [];

  register() : void  {  Show usages  Robert Nitu
    this.errorMessages = [];
    this.authService.register({
      body: this.registerRequest
    }).subscribe({
      next: (res: RegistrationRequest) : void  => {
        this.router.navigate(['activate-account']);
      },
      error: err : any  => {
        console.log(JSON.stringify(err));
        if (err.error.validationErrors) {
          this.errorMessages = err.error.validationErrors;
        } else {
          this.errorMessages.push(err.error.error);
        }
      }
    });
  }

  goToLogin() : void  {  Show usages  Robert Nitu
    this.router.navigate(['login']);
  }

  isAlreadyLoggedIn(): boolean {  Show usages  Robert Nitu
    return this.tokenService.isAlreadyLoggedIn();
  }
}
```

Fig 3. 44 Componentă înregistare – TypeScript

3.5.8 Componenta Activare cont

Componenta de activare cont este necesară pentru a seta un cont din status-ul de nefolosința la cel de folosință. Conform fig. 3.45, această componentă are două vederi, una fiind cea marcată cu tag-ul „ng-template” specific Angular, iar cealaltă fiind în primul „div”, acolo fiind afișat mesajul de eroare, în cazul nostru în interiorul template-ului cu tag-ul de „activationError”, va fi afișat un text cu tag-ul „h2”, iar apoi un buton de „try again” (încearcă din nou), care la apăsare va seta valoarea variabilei „submitted” înapoi pe cea prestabilită, anume cu valoarea „fals”. Valoarea de adevăr a variabilei „isOkay” indică că totul a funcționat corect, astfel utilizatorul trebuie să se ducă pe pagina de logare pentru a se putea loga, iar accest lucru va putea fi făcut cu ajutorul butonul prezent, iar odată apasăt se va apela funcția „goToLogin” unde îl va redirecționa. În cealaltă parte a paginii este prezent un input de tip-ul „code-input”, acest tag a fost implementat prin intermediul librăriei Angular, fiind specific acestui framework. În componența sa are câteva atribute setate, cum ar fi: „isCodeHidden”, face ca valoarea introdusă să fie vizibilă, „codeLength”, setează maximul de caractere ce îl va avea codul, „code”, setează tipul de date, în cazul nostrum fiind de tip număr, iar ultima fiind „codeCompleted”, care este funcția apelată în momentul în care se introduc toate caracterele în input-ul respectiv.

Conform fig. 3.46, în interiorul clasei de TypeScript au fost injectate serviciile de rutare (router) și de autentificare (AuthenticationService). Funcția definită cu numele de „goToLogin” are ca rol rutarea folosind service-ul „router” către pagina de logare. Funcția „onCodeCompleted”, care se apelează în momentul completării input-ului prezentat anterior, apelează funcția „confirmAccount”, care aceasta apelează metoda „confirm” de tip observabil din service-ul „authService”, generat în mod automat de către unealta OpenApi, care are două scenarii: Cazul în care totul a fost validat de către server-ul de back-end ca fiind corect, valorile variabilelor „submitted” și „isOkay” se vor seta pe valoarea „true”, astfel elementele din componenta HTML vor fi schimbate. În cazul celelalt, în care server-ul de back-end a întâmpinat vreo problemă, valoarea variabilei „isOkay” se setează pe „false”, iar componenta de HTML va merge pe cazul de apăsare de buton pentru a reintroduce codul. Pe lânmgă aceste setări de variabile, se va seta și mesajul de eroare sau informare specific cazului.

```
<div *ngIf="submitted; else codeForm" class="container">
  <div *ngIf="isOkay; else activationError" class="activate-message">
    <h2>{{ query: 'login-and-register.activate-successfully-title' | translate }}</h2>
    <p>{{ message | translate }}</p>
    <button class="btn btn-primary" (click)="goToLogin()">{{ query: 'login-and-register.sign-in' | translate }}</button>
  </div>

  <ng-template #activationError : TemplateRef<any> >
    <div class="activate-error">
      <h2>{{ query: 'login-and-register.activate-bad-title' | translate }}</h2>
      <p>{{ message | translate }}</p>
      <button class="btn btn-primary" (click)="submitted = false">{{ query: 'login-and-register.try-again' | translate }}</button>
    </div>
  </ng-template>
</div>

<ng-template #codeForm : TemplateRef<any> >
  <div class="container">
    <div class="text-center code-form-container">
      <h2>{{ query: 'login-and-register.type-activation-code' | translate }}</h2>
      <code-input
        [isCodeHidden]="false"
        [codeLength]="6"
        [code]="'number'"
        (codeCompleted)="onCodeCompleted($event)">

      </code-input>
    </div>
  </div>
</ng-template>
```

Fig 3. 45 Componentă activare cont – HTML

```typescript
export class ActivateAccountComponent {

  message : string  = '';
  isOkay : boolean  = true;
  submitted : boolean  = false;

  constructor(  no usages  Robert Nitu
    private router: Router,
    private authService: AuthenticationService
  ) {

  }

  onCodeCompleted(token: string) : void  {  Show usages  Robert Nitu
    this.confirmAccount(token);
  }

  goToLogin() : void  {  Show usages  Robert Nitu
    this.router.navigate(['login']);
  }

  private confirmAccount(token: string) : void  {  Show usages  Robert Nitu
    this.authService.confirm({
      token
    }).subscribe({
      next: () : void  => {
        this.message = 'login-and-register.activate-successfully';
        this.submitted = true;
        this.isOkay = true;
      },
      error: () : void  => {
        this.message = 'login-and-register.activate-bad';
        this.submitted = true;
        this.isOkay = false;
      }
    });
  }
}
```

Fig 3. 46 Componentă activare cont – TypeScript

3.5.9 Componenta Logare

Componenta de logare este folosită pentru a permite utilizatorului să își introducă credențialele contului și să se logheze în aplicație. Conform fig. 3.47, această componentă are în mare aceași componență ca cea prezentată anterior, cea de register mai exact, doar că input-urile sunt restrânse la adresa de email și parola. Ambele au aceleași input-uri cu tag-ul de „ngModel”, unde în momentul populării în caseta de text, se va actualiza și câmpurile de „email” sau de „password”din variabila „authRequest”. În partea de jos avem un „div” care conține un buton de logare, acesta este afișat doar dacă utilizatorul nu este logat, acest lucru este verificat prin intermediul tag-ului „ngIf”, iar funcția din cadrul acestui tag este „isAllreadyLoggedIn” din service-ul prezentat ulterior „TokenService”. În partea dreapta a acestui buton se află metoda de navigare mai rapidă către creare de cont.

În fișierul TypeScript, conform fig. 3.48, avem injectate în constructor serviciile de rutare (router), service-ul de autentificare generat în mod automat de către OpenAPI și service-ul creat anterior numit „TokenService”. Funcția numită „register” are rolul de a ruta utilizatorul către

componenta de înregistrare. Funcția menită să facă acțiunea de logare este numită „login”, această în primă fază resetează mesajele de eroare cu atribuirea unui obiect gol, „[]”, iar ulterior folosește metoda „authenticate” din cadrul service-ului de autentificare, iar ca parametru este dat obiectul din variabila „authRequest”, ce conține email-ul și parola utilizatorului conform valorilor din input-uri. Această funcție este de tip observabil, aceasta având două ramuri, de succes și de eroare. În cazul unei erori din partea server-ului de back-end, variabila „errorMessages” este populată cu eroare sau erorile venite de pe server-ul de back-end. În caz contrat, în care utilizatorul a introdus credențialele corecte, se va face logarea în aplicație a acestuia, acest lucru fiind realizabil cu ajutorul service-ului creat anterior de „TokenService”. Se vor utiliza metodele specifice acestui service, mai exact salvarea token-ului venit ca răspuns de pe server-ul de back-end, care confirmă că utilizatorul este logat în aplicație. Pe lângă token-ul trimis de pe server-ul de back-end, mai sunt trimise și valori pentru „id”, „admin”, „marketId” și „stallID”. Acestea sunt verificate dacă există, dacă valoarea lor este diferită de „undefined”, iar apoi sunt salvate prin intermediul service-ului în variabila specifică denumirii. Aceste informații sunt folosite în alte componente pentru status-ul utilizatorului de administator, proprietar de piață sau proprietar de tarabă. După aceste setări de variabile locale, există un timeout care după o secundă va naviga utilizatorul către ruta de prima pagină pentru a începe navigarea completă.



Fig 3. 47 Componentă logare – HTML



```
export class LoginComponent {

  authRequest: AuthenticationRequest = { email: '', password: '' };
  errorMessages: Array<string> = [];

  constructor( no usages  Robert Nitu
    private router: Router,
    private authService: AuthenticationService,
    public tokenService: TokenService,
  ) {
  }

  login() : void  {  Show usages  Robert Nitu
    this.errorMessages = [];

    this.authService.authenticate({
      body: this.authRequest
    }).subscribe({
      next: (res: AuthenticationResponse) : void  => {
        this.tokenService.token = res.token as string;
        if (res.id !== undefined)
          this.tokenService.id = +res.id;
        if (res.admin !== undefined)
          this.tokenService.admin = +res.admin;
        if (res.marketId !== undefined)
          this.tokenService.market = +res.marketId;
        if (res.stallId !== undefined)
          this.tokenService.stall = +res.stallId;

        setTimeout(() : void  => {
          this.router.navigate(['/home']);
        }, delay: 1000);
      },
      error: (err : any ) : void  => {
        console.log(JSON.stringify(err));
        if (err.error.validationErrors) {
          this.errorMessages = err.error.validationErrors;
        } else {
          this.errorMessages.push(err.error.error);
        }
      }
    });
  }

  register() : void  {  Show usages  Robert Nitu
    this.router.navigate(['register']);
  }
}
```

Fig 3. 48 Componentă logare – TypeScript

3.5.10 Componenta Profil

Această componentă are rolul de a afișa informațiile principale despre contul cu care este logat utilizatorul în cadrul aplicației. Conform fig. 3.49, această componentă are în partea de cod HTML un „div” care este împărțit în două moduri de vizionare. Prima fiind cazul în cazul utilizatorul este considerat fiind logat în aplicație, iar cealaltă fiind nelogat, iar pentru această componentă trebuie să fie logat pentru a vizualiza informațiile contului. Vizualizarea împărțită a fost realizată cu ajutorul tag-ului specific Angular „ngIf”. În cadrul acestui tag a fost folosită funcția „isAllreadyLoggedIn” din service-ul „TokenService”, care asigură dacă utilizatorul este logat. Dacă funcția returnează valoarea „false”, înseamnă că utilizatorul nu este logat, i se va afișa bucata de cod scrisă în template-ul „needToLoginFirst”, care este un mesaj de informare că acesta nu este logat, iar alături este un buton care îl va redirecționa către ruta de logare. Informațiile despre utilizator sunt stocate în variabila „profileData”, variabilă populată în cadrul fișierului TypeScript. Pentru a nu exista probleme de lag în momentul inițializării componentei, a fost adăugat un „ngIf” unde verifică dacă variabila „profileData” a fost populată. Informațiile afișate despre contul utilizatorului sunt următoarele: imaginea de profil selectată la înregistrare, numele și prenumele acestuia, cât și ID-ul salvat în baza de date, care reprezintă numărul de ordine al contului, mai exact cheia de identificare a utilizatorului. În plus mai sunt afișate informațiile despre email-ul utilizatorului, cât și dacă acesta are contul activat sau nu. Următoarea parte sunt informații despre rolurile utilizatorului, dacă acesta este admin i se va afișa un mesaj informativ cu gradul acestuia, acest lucru este realizabil prin

intermediul tag-ului „ngIf”. Următoarele roluri sunt de proprietar de piață sau proprietar de tarabă. Dacă condițiile precizate în „ngIf” sunt adevărate, mai precis dacă în variabilele „marketId” sau „stallId” valorile sunt diferite de 0, se va afișa un buton care îl va redirecționa către pagina informativă a pieței, respectiv a tarabei unde este proprietar. În ultima parte a paginii HTML, se vor afișa toate comenziile utillizatorului, indiferent de status-ul acestora. Ele sunt afișate prin intermediul unui tabel care are mai multe coloane cu informațiile din tabela „Orders” din baza de date. Înainte de afișare se verifică dacă variabila „ordersData” a fost populată și există comenzi procesate de utilizator, în caz contrar se va afișa container-ul „noOrders”, unde va fi un mesaj informativ că nu are nicio comandă activă / procesată. Informațiile din tabel sunt afișate prin intermediul unei bucle de tip for specifice Angular, cu ajutorul tag-ului „ngFor”. În cazul coloanei „products”, a fost creată funcția „beautifyProducts” unde va afișa într-un mod ușor de citit produsele comandate.

În fișierul TypeScript, conform fig. 3.50, au fost injectate serviciile de rutare (router), „TokenService” pentru a extrage valori despre autentificare și rolurile utilizatorului, „userService” și „orderService” care sunt servicii generate cu ajutorul OpenAPI. Această clasă are implementate metodele de „OnInit” și „OnDestroy”. În cazul metodei „OnInit”, care este apelată imediat cum este creată componenta și afișată prin intermediul rutării, se vor stoca local în componentă valorile din service-ul „TokenService” în variabilele „userId”, „marketId”, „stallId” și „isAdmin”. Cu aceste valori putem influența ce va vedea utilizatorul în pagina HTML. O primă funcție apelată după extragerea acestor valori, este cea de „getProfileData”, care apelează metoda „getUser” din service-ul „userService”, adăugând ca parametru valoarea extrase anterior cu privire la ID-ul de ordine al utilizatorului. Această metodă returnează un observabil care este stocat în variabila „profileSub”, care ulterior în cazul de „OnDestroy”, când componenta se va distruge din rutarea către o altă componentă / ieșirea din aplicație, se va verifica dacă aceasta a fost inițializată și se va apela funcția de „unsubscribe”, care oprește observabilul din rulare. Funcția „getUser” va apea serverul de back-end, iar acesta în urma verificărilor va returna un obiect de tipul „UserResponse”, care va fi stocat în variabila „profileData”. Cu această variabilă este posibilă afișarea informațiilor în pagina HTML. Următoarea funcție din cadrul metodei „OnInit” este „getOrdersData”, care apelează metoda „getOrdersFromUserId” din service-ul „orderService”, care are ca parametru valoarea ID care reprezintă număru de ordine al utilizatorului. Această funcție returnează un observabil care este stocat în variabila ”ordersSub”, care ulterior în cazul de „OnDestroy”, se va apela funcția de „unsubscribe”. Metoda din „orderService” va face un apel către server-ul de back-end unde va returna o listă de variabile de tipul „OrderResponse”, care va fi salvată în variabila „ordersData”. Cu această variabilă se vor afișa informațiile despre comenzile utilizatorului. Funcțiile „goToMarket” și „goToStall” vor redirecționa utilizatorul către piața sau taraba la care este proprietar. Acest lucru se realizează folosind service-ul „router”, unde se va naviga în funcție de fiecare dintre metode, la final adăugându-se ca parametru id-ul de ordine al acestora. Ultima funcție prezentă este „beautifyProducts”, care primește ca parametru produsele din cadrul unei comenzi, acestea fiind de formă „ProductData”, care conține informații despre cantitate, nume, preț și id de ordine. Această funcție folosește operația de deserializare a parametrului cu ajutorul librăriei JSON, iar poi construiește un string care cuprinde informații simplificate sub forma de „nume produs x cantitate”.



```
<div class="container py-5">
  <div *ngIf="tokenService.isAlreadyLoggedIn(); else needToLoginFirst">
    <div *ngIf="profileData" class="card shadow-lg mx-auto mb-5 rounded-4 profile-container">
      <div class="card-body text-center">

        <div class="mb-4">
          <img src="{{profileData.imageUrl}}"
               alt="User Image"
               class="rounded-circle border border-2 profile-image"/>
        </div>

        <h4 class="card-title mb-1 fw-bold">{{ profileData.firstName }} {{ profileData.lastName }}</h4>
        <h6 class="card-subtitle text-muted mb-4">ID: {{ profileData.id }}</h6>

        <ul class="list-group list-group-flush text-start mb-3">
          <li class="list-group-item"><strong>Email:</strong> {{ profileData.email }}</li>
          <li class="list-group-item">
            <strong>{{ query: 'profile.activated' | translate }}:</strong>
            <span [class.text-success]="profileData.enabled" [class.text-danger]="!profileData.enabled">
              {{ (profileData.enabled ? 'profile.activated-yes' : 'profile.activated-no') | translate }}
            </span>
          </li>
          <li *ngIf="isAdmin" class="list-group-item">
            <strong>{{ query: 'profile.is-admin' | translate }}:</strong> 1
          </li>
          <li *ngIf="marketId !== 0" class="list-group-item">
            <strong>{{ query: 'profile.is-market-owner' | translate }}:</strong>
            <button (click)="goToMarket()"
                    class="btn btn-outline-success btn-sm rounded-pill ms-2">
              {{ query: 'profile.view-market' | translate }}
            </button>
          </li>
          <li *ngIf="stallId !== 0" class="list-group-item">
            <strong>{{ query: 'profile.is-stall-owner' | translate }}:</strong>
            <button class="btn btn-outline-success btn-sm rounded-pill ms-2"
                    (click)="goToStall()">
              {{ query: 'profile.view-stall' | translate }}
            </button>
          </li>
        </ul>
      </div>
    </div>
```

```
    <div *ngIf="ordersData && ordersData.length > 0; else noOrders"
         class="card shadow-lg mx-auto rounded-4 p-4 orders-container">
      <h4 class="mb-4 text-center fw-bold text-primary">
        {{ query: 'orders.title' | translate }}
      </h4>

      <div class="table-responsive">
        <table class="table table-striped table-hover align-middle text-center">
          <thead class="table-dark">
          <tr>
            <th>#</th>
            <th>{{ query: 'orders.market' | translate }}</th>
            <th>{{ query: 'orders.stall' | translate }}</th>
            <th>{{ query: 'orders.address' | translate }}</th>
            <th>{{ query: 'orders.products' | translate }}</th>
            <th>{{ query: 'orders.price' | translate }}</th>
            <th>{{ query: 'orders.order-date' | translate }}</th>
            <th>{{ query: 'orders.delivery-date' | translate }}</th>
            <th>{{ query: 'orders.accepted-date' | translate }}</th>
            <th>{{ query: 'orders.status' | translate }}</th>
          </tr>
          </thead>
          <tbody>
          <tr *ngFor="let order : OrderResponse of ordersData; let i : number = index">
            <td>{{ i + 1 }}</td>
            <td>{{ order.marketName }}</td>
            <td>{{ order.stallName }}</td>
            <td>{{ order.address }}</td>
            <td>{{ beautifyProducts(order.products!) }}</td>
            <td>{{ order.price }} RON</td>
            <td>{{ order.orderDate | date: format: 'short' }}</td>
            <td>{{ (order.acceptedDate | date: format: 'short' ) || '-' }}</td>
            <td>{{ (order.deliveryDate | date: format: 'short') || '-' }}</td>
            <td>{{ ('orders.status-type.' + order.status) | translate }}</td>
          </tr>
          </tbody>
        </table>
      </div>
    </div>

    <ng-template #noOrders : TemplateRef<any> class="alert alert-warning text-center mt-4">
      <div class="text-center mt-5">
        {{ query: 'orders.no-orders' | translate }}
      </div>
    </ng-template>
  </div>

  <ng-template #needToLoginFirst : TemplateRef<any> >
    <div class="text-center mt-5">
      <h2 class="text-danger">{{ query: 'profile.need-to-login-first' | translate }}</h2>
      <p class="lead">{{ query: 'profile.please-login' | translate }}</p>
    </div>
  </ng-template>
</div>
```

Fig 3. 49 Componentă profil – HTML



```
export class ProfileComponent implements OnInit, OnDestroy {

  userId = 0;
  profileData : UserResponse | undefined = undefined;
  profileSub: Subscription | undefined;

  ordersData: OrderResponse[] = [];
  ordersSub: Subscription | undefined;

  marketId: number = -1;
  stallId: number = -1;
  isAdmin: boolean = false;

  constructor( no usages  Robert Nitu
    private router: Router,
    public tokenService: TokenService,
    private userService: UserService,
    private orderService: OrderService
  ) {

  }

  ngOnInit(): void {  no usages  Robert Nitu
    this.userId = this.tokenService.id;
    this.marketId = this.tokenService.market;
    this.stallId = this.tokenService.stall;
    this.isAdmin = this.tokenService.admin === 1;

    this.getProfileData();
    this.getOrdersData();
  }

  ngOnDestroy(): void {  no usages  Robert Nitu
    if (this.profileSub !== undefined) {
      this.profileSub.unsubscribe();
      console.log( message: `profileSub unsubscribed`);
    }

    if (this.ordersSub !== undefined) {
      this.ordersSub.unsubscribe();
      console.log( message: `ordersSub unsubscribed`);
    }
  }
```

```
  getProfileData() : void  {  Show usages  Robert Nitu
    this.profileSub = this.userService.getUser({
      id: this.userId
    }).subscribe({
      next: (data : UserResponse ) : void  => {
        this.profileData = data;
      },
      error: (err : any ) : void  => {
        console.log(err);
      }
    })
  }

  getOrdersData() : void  {  Show usages  Robert Nitu
    this.ordersSub = this.orderService.getOrdersFromUserId({
      id: this.userId
    }).subscribe({
      next: (data : OrderResponse[] ) : void  => {
        this.ordersData = data;
      },
      error: (err : any ) : void  => {
        console.log(err);
      }
    });
  }

  goToMarket() : void  {  Show usages  Robert Nitu
    this.router.navigate(['stalls', this.marketId]);
  }

  goToStall() : void  {  Show usages  Robert Nitu
    this.router.navigate(['stall', this.stallId]);
  }

  beautifyProducts(products: string): string {  Show usages
    let final : string  = '';
    const dataParsed: ProductData[] = JSON.parse(products);
    dataParsed.forEach((product: ProductData) : void  => {
      final += `${product.name} x ${product.quantity}; `;
    });
    return final;
  }
}
```

Fig 3. 50 Componentă profil – TypeScript

3.5.11 Componenta Cereri

Această componentă este una la fel de specială ca cea de logare, deoarece cu ajutorul acestia utilizatorii pot face cereri pentru rolurile de administrator și de proprietar de piață, acestea fiind analizate și corectate de către un administrator. Conform fig. 3.51, componenta are în configurație mai multe moduri de vizualizare. Aceste moduri sunt din perspectiva unui utilizator care este deja logat, cu ajutorul tag-ului „ngIf" asupra „div-ului" principal se verifică dacă utilizatorul este conectat, în caz contrat se va afișa template-ul inițialiat cu tag-ul „needToLoginFirst". Dacă utilizatorul este conectat, în cadrul acestui „div" se vor afișa 2, sau 3 containere în funcție de rolul pe care utilizatorul îl are.



Fig 3. 51 Componentă cereri – HTML, partea 1

Utilizatorul fiind logat își va putea vedea toate cererile completate și trimise, alături de informații despre când s-a răspuns la aceasta, ce răspuns a primit și justificarea răspunsului. Conform fig. 3.52, întregul „div" este afișat doar dacă se îndeplinește condiția din interiorul tag-ului „ngIf", anume ca variabila „hasRequests" sa aibe valoarea de „true". În caz contrar se va afișa template-ul cu tag-ul „noRequest", care conform fig 3.69, afișează un mesaj informativ că nu există cereri făcute. Dacă utilizatorul are cereri, se vor afișa toate cererile cu ajutorul unei bucle repetitive de tip „ngFor", specifice Angular, care are in componență informații de tipul „RequestResponse" care sunt salvate în variabila „userRequests". Sunt afișate informații despre imaginea utilizatorului „userImageUrl", întregul nume și adresa de email „userFullName", „userEmail", se va afișa și tipul de cerere, dacă este de administrator sau proprietar de piață, link-urile atașate cererii în cazul în care câmpul „links" există, a fost făcută verificarea cu ajutorul „ngIf", status-ul cereri, dacă aceasta este în așteptare, acceptată sau respinsă, motivul este afișat doar dacă acesta există, coloana respectivă se populează în momentul în care un administrator răspunde cererii, iar în final sunt afișate informații despre data

când a fost creată cererea folosind „date: ‚short'", pentru a afișa timpul într-o formă ușor de citit pentru utilizator. Câmpul „answeredAt", la fel ca cel de „reason", se populează în momentul în care un administrator răspunde cererii, astfel trebuie verificat cu ajutorul „ngIf" dacă a fost completat pentru afișare.

```
<div *ngIf="hasRequests; else noRequests" class="row g-4">
  <div class="col-md-6" *ngFor="let request : RequestResponse of userRequests">
    <div class="card h-100 border-light shadow-sm rounded-4 p-3">
      <div class="d-flex align-items-center mb-3">
        <img src="{{request.userImageUrl}}"
             alt="User"
             class="rounded-circle me-3 border user-image"/>
        <div>
          <h5 class="mb-0 fw-semibold">{{ request.userFullName }}</h5>
          <small class="text-muted">{{ request.userEmail }}</small>
        </div>
      </div>

      <p class="mb-1">
        <strong>{{ query: 'requests.type' | translate }}:</strong> {{ ('requests.types.' + request.type) | translate }}
      </p>
      <p class="mb-1">
        <strong>{{ query: 'requests.links' | translate }}:</strong>
        <span *ngIf="request.links" class="link-primary">
          {{ request.links }}
        </span>
      </p>
      <p class="mb-1">
        <strong>{{ query: 'requests.status' | translate }}:</strong> {{ ('requests.status-type.' + request.status) | translate }}
      </p>
      <p class="mb-1" *ngIf="request.reason !== ''">
        <strong>{{ query: 'requests.reason' | translate }}:</strong> {{ request.reason }}
      </p>
      <p class="mb-3">
        <strong>{{ query: 'requests.message' | translate }}:</strong> {{ request.message }}
      </p>

      <div class="text-muted small">
        <div>
          <strong>{{ query: 'requests.createdAt' | translate }}:</strong> {{ request.createdAt | date: format: 'short' }}
        </div>
        <div *ngIf="request.answeredAt">
          <strong>{{ query: 'requests.answeredAt' | translate }}:</strong> {{ request.answeredAt | date: format: 'short' }}
        </div>
      </div>
    </div>
  </div>
</div>
```

Fig 3. 52 Componentă cereri – HTML, partea 2

Următoarea perspectivă este cea de utilizator cu rol de administrator, acesta va putea vedea și interacționa cu toate cererile create și a le schimba status-ul în acceptat sau respins, urmat de un motiv pentru aceste acțiuni. Conform fig. 3.53, se va afișa container-ul pentru admini doar dacă valoarea variabilei „isAdmin" este pozitivă. În variabila din a doua condiție „ngIf" se verifică dacă există cereri create, iar în caz contrat se va afișa container-ul cu tag-ul „noRequests". Acest container, conform fig 3.69 afișează un mesaj informativ că nu există cereri create. Ca în cazul anterior, de vizualizare a tuturor cererilor create de utilizator, se va folosi această metodă, doar că se aplică pentru toate cererile ale tuturor utilizatorilor. Variabila care are stocate aceste date este denumită „usersRequsts". În plus față de prezentarea anterioară există în josul container-ului un „div" care este afișat doar dacă status-ul cererii are valoarea 0, adică este în așteptare. Dacă cererea are status-ul în așteptare, înseamnă că se poate interacționa cu aceasta. Au fost adăugate 2 modele de input, una de tipul „select", unde se va seta noul status al cererii, care poate avea valoarea de „ACCEPTED" (acceptat) sau „REJECTED" (respins). Următorul input este de tipul „textarea" care are rolul de introducere al unui mesaj pentru a justifica alegerea făcută. În josul container-ului de interacționare

cu cererea se află un buton care actualizează status-ul și motivul cererii. Variabila „requestUpdateData” este menită să salveze status-ul și motivul cererii, iar în cazul în care administratorul apasă pe butonul de „submit”, datele să fie extrase din variabila respectivă.



```
<div *ngIf="isAdmin" class="container py-4">
  <div class="card shadow border-0 p-4">
    <h3 class="text-center mb-4">{{ query: 'requests.admin-requests' | translate }}</h3>

    <div *ngIf="usersRequests.length > 0; else noRequests" class="row g-4">
      <div class="col-md-6" *ngFor="let request : RequestResponse of usersRequests">
        <div class="card h-100 border-light shadow-sm rounded-4 p-3">
          <div class="d-flex align-items-center mb-3">
            <img src="{{request.userImageUrl}}"
                 alt="User"
                 class="rounded-circle me-3 border user-image"/>
            <div>
              <h5 class="mb-0 fw-semibold">{{ request.userFullName }}</h5>
              <small class="text-muted">{{ request.userEmail }}</small>
            </div>
          </div>

          <p class="mb-1">
            <strong>{{ query: 'requests.type' | translate }}:</strong>
            {{ ('requests.types.' + request.type) | translate }}
          </p>
          <p class="mb-1">
            <strong>{{ query: 'requests.status' | translate }}:</strong>
            {{ ('requests.status-type.' + request.status) | translate }}
          </p>
          <p class="mb-1">
            <strong>{{ query: 'requests.message' | translate }}:</strong>
            {{ request.message }}
          </p>
          <p class="mb-3">
            <strong>{{ query: 'requests.links' | translate }}:</strong>
            <span *ngIf="request.links" class="link-primary">
              {{ request.links }}
            </span>
          </p>
```

```
          <div class="text-muted small mb-3">
            <div>
              <strong>{{ query: 'requests.createdAt' | translate }}:</strong>
              {{ request.createdAt | date: format: 'short' }}
            </div>
            <div *ngIf="request.answeredAt">
              <strong>{{ query: 'requests.answeredAt' | translate }}:</strong>
              {{ request.answeredAt | date: format: 'short' }}
            </div>
          </div>

          <div *ngIf="request.status === 0" class="border-top pt-3 mt-3">
            <label class="form-label fw-semibold">{{ query: 'requests.admin-action' | translate }}</label>
            <select class="form-select mb-2"
                    [(ngModel)]="requestUpdateData[request.id!].status">
              <option value="ACCEPTED">{{ query: 'requests.approve' | translate }}</option>
              <option value="REJECTED">{{ query: 'requests.reject' | translate }}</option>
            </select>

            <textarea class="form-control mb-3"
                      rows="3"
                      [(ngModel)]="requestUpdateData[request.id!].reason"
                      placeholder="{{ query: 'requests.reason-placeholder' | translate }}">
            </textarea>

            <button class="btn btn-outline-primary w-100"
                    (click)="submitRequestAnswer(request.id!)">
              {{ query: 'requests.submit' | translate }}
            </button>
          </div>
        </div>
      </div>
```

Fig 3. 53 Componentă cereri – HTML, partea 3

Ultima parte din fișierul HTML este formată din completarea cererii și trimiterea către server-ul de back-end. Conform fig. 3.54, în prima parte este prezent un „div” care are rolul de a afișa mesajele de eroare întâmpinate în momentul în care se încearcă să se trimite cererea către server-ul de back-end. Acestea sunt afișate printr-o buclă repetitivă de tip „ngFor”. Următoarele elemente sunt de tipul „input”, care se salvează prin intermediul tag-ului „ngModel” în diferitele câmpuri specifice din variabila „requestData”. Primul câmp este cel de selctare al tipului de cerere, care este ori de administrator, ori de proprietar de piață. Următorul input este cel de mesaj, unde utilizatorul poate introduce un mesaj informativ despre rolul la care face cerere. Input-ul de link-uri se referă la câteva dintre documentele necesare de a dovedi că utilizatorul chiar deține o piață și nu este ceva ireal, pe când, în celelalt rol de administrator acesta își poate introduce link-ul către CV-ul său. Fiecare cerere are în componență o image specifică, care ulterior este afișată pentru ca utilizatorul să vadă cum va arăta pe restul site-ului. În final este secțiunea de trimitere a cererii, unde se va anliza pe partea de TypeScript, iar apoi se va introduce în baza de date prin intermediul server-ului de back-end dacă totul este în parametri.



```
<div class="container py-4">
  <div class="card shadow border-0 p-4 rounded-4">
    <h3 class="text-center mb-4">{{ query: 'requests.title' | translate }}</h3>

    <div *ngIf="errorMessages.length > 0" class="alert alert-danger">
      <ul class="mb-0">
        <li *ngFor="let message : string of errorMessages">{{ message }}</li>
      </ul>
    </div>

    <div class="mb-3">
      <label for="requestType" class="form-label fw-medium">
        {{ query: 'requests.type' | translate }}</label>
      <select [(ngModel)]="requestData.type"
              class="form-select"
              id="requestType">
        <option value="0">{{ query: 'requests.types.0' | translate }}</option>
        <option value="1">{{ query: 'requests.types.1' | translate }}</option>
      </select>
    </div>

    <div class="mb-3">
      <label for="requestMessage" class="form-label fw-medium">
        {{ query: 'requests.message' | translate }}</label>
      <textarea [(ngModel)]="requestData.message"
                class="form-control"
                id="requestMessage"
                rows="4"
                placeholder="{{ query: 'requests.message-place-holder' | translate }}">
      </textarea>
    </div>
```

```
    <div class="mb-4">
      <label for="requestLinks" class="form-label fw-medium">{{ query: 'requests.links' | translate }}</label>
      <textarea [(ngModel)]="requestData.links"
                class="form-control"
                id="requestLinks"
                rows="2"
                placeholder="{{ query: 'requests.links-place-holder' | translate }}">
      </textarea>
    </div>

    <div class="mb-4">
      <label for="imageUrl" class="form-label">{{ query: 'requests.image-url' | translate }}</label>
      <input [(ngModel)]="requestData.imageUrl"
             type="url"
             class="form-control rounded-3"
             id="imageUrl"
             placeholder="{{ query: 'requests.image-url-placeholder' | translate }}">
    </div>

    <div class="text-center mb-4" *ngIf="requestData.imageUrl">
      <img src="{{requestData.imageUrl}}"
           alt="Profile preview"
           class="rounded-circle shadow-sm image-preview"/>
      <p class="small text-muted mt-2">{{ query: 'requests.image-preview' | translate }}</p>
    </div>

    <div class="d-flex justify-content-end">
      <button (click)="sendRequest()" type="button" class="btn btn-primary px-4 py-2 rounded-pill shadow-sm">
        <i class="fas fa-paper-plane me-2"></i> {{ query: 'requests.send' | translate }}
      </button>
    </div>
  </div>
</div>
```

Fig 3. 54 Componentă cereri – HTML, partea 4

Conform fig. 3.55, clasa de „RequestsComponent” este folosită pentru a extrage sau trimite informații de pe server-ul de back-end. În partea de sus a clasei sunt definite variabile ce vor fi folosite în partea de HTML a componentei, mai exact variabila „requestData” conține valorile puse în input-ul de completare de cerere, „userRequests” conține toate cererile din tabela „Requests” din baza de date completate de către utilizatorul care este logat, acest lucru este realizabil prin intermediul variabilei „userId”. Variabila „usersRequests” este populată doar dacă utilizatorul este administrator, iar aceasta conține informații despre toate cererile existente în tabela „Requests” din baza de date. Variabilele „sendRequestSub”, userRequestSub”, „usersRequestsSub” și „sendUpdateSub” sunt de tipul observabile, iar acestea în momentul în care se declanșează metoda „OnDestroy” sunt oprite din execuție. Variabilele „hasRequests” și „isAdmin” sunt populate în interiorul metodei „OnInit”, unde în funcție de valorile din service-ul „TokenService” se obțin informații despre rolul de administrator și ID-ul de ordine din baza de date a utilizatorului. Tot în metoda de inițializare, este populat câmpul de „requesterId” cu variabila „userId” extrasă anterior.



```
export class RequestsComponent implements OnInit, OnDestroy {

  userId = 0;

  requestData: Request = { type: 0, status: 0, message: '', links: '', requesterId: this.userId, imageUrl: '' };
  errorMessages: Array<string> = [];

  sendRequestSub: Subscription | undefined;

  userRequests: RequestResponse[] = [];
  userRequestSub: Subscription | undefined;
  hasRequests = false;

  isAdmin = false;
  usersRequests: RequestResponse[] = [];
  requestUpdateData: RequestStatusData[] = [];
  usersRequestsSub: Subscription | undefined;
  sendUpdateSub: Subscription | undefined;

  constructor( no usages  Robert Nitu
    private requestService: RequestService,
    public tokenService: TokenService) {
  }
```

```
ngOnInit(): void {  no usages  Robert Nitu
  this.userId = this.tokenService.id;
  this.isAdmin = this.tokenService.admin === 1;

  this.requestData.requesterId = this.userId;

  this.getUserRequests();
  this.getAdminRequests();
}

ngOnDestroy(): void {  no usages  Robert Nitu
  if (this.sendRequestSub !== undefined) {
    this.sendRequestSub.unsubscribe();
    console.log('sendRequestSub unsubscribed');
  }

  if (this.userRequestSub !== undefined) {
    this.userRequestSub.unsubscribe();
    console.log( message: 'userRequestSub unsubscribed');
  }

  if (this.usersRequestsSub !== undefined) {
    this.usersRequestsSub.unsubscribe();
    console.log( message: 'usersRequestsSub unsubscribed');
  }

  if (this.sendUpdateSub !== undefined) {
    this.sendUpdateSub.unsubscribe();
    console.log( message: 'sendUpdateSub unsubscribed');
  }
}
```

Fig 3. 55 Componentă cereri – TypeScript, partea 1

Conform fig. 3.56, funcțiile „getUserRequests” și „getAdminRequests” sunt folosite pentru a extrage informații de pe server-ul de back-end. Acestea folosesc service-ul „requestService”, generat cu ajutorul uneltei OpenAPI. Funcția „getUserRequests” extrage de pe server-ul de back-end toate cererile completate de către utilizator, având ca parametru valoarea de „userId”, care reprezintă ID-ul de ordine din baza de date al utilizatorului. Metoda returnează un observabil, iar în cazul în care au fost găsite cereri, se stochează în variabila „userRequests”, iar apoi în variabila numită „hasRequests” se va salva valoarea expresiei „res.length > 0”, care reprezintă dacă utilizatorul are cereri create. Următoarea funcție este „getAdminRequests”, care folosind metoda „getAllRequests” va extrage informații despre toate cererile existente în tabela „Requests” din baza de date. În momentul în care se populează în variabila „usersRequests”, se va face o copie locală care va avea setate date despre cererile respective, doar că într-un mod mai simplist, având date doar despre ID-ul de ordine, status-ul și motivul. Variabila „status” este setată implicit pe „PENDING” (în așteptare), iar motivul fiind un string gol „ ‘’ ”, lăsând ca utilizatorul să completeze aceste două câmpuri.



Fig 3. 56 Componentă cereri – TypeScript, partea 2

Ultimele funcționalități din cadrul acestui fișier TypeScript, sunt cele de trimitere de informații către server-ul de back-end. Conform fig. 3.57, funcția „sendRequest” resetează mesajele de eroare cu un obiect gol „[]”, iar apoi se verifică dacă input-urile au fost completate corect. Funcția „isRequestValid” verifică dacă câmpurile din variabila „requestData” au fost completate și nu au fost lăsate valori goale. Următorul pas este de apelare a metodei „addRequest” din cadrul service-ului „requestService”, care primește ca parametru variabila „requestData” populată prin input-urile din HTML, iar în cazul în care a fost validat și introdus de către server-ul de back-end se va apela funcția de „refreshPage”, care aceasta va reîmprospăta pagina web dupa un „timeout” de 2 secunde. Funcția de „submitRequestAnswer” va actualiza status-ul unei cereri doar dacă funcția „isSubmitValid” returnează valoarea „true”. Funcția „isSubmitValid” verifică dacă câmpurile de motiv și status au fost schimbate și dacă există valoare în variabila „requestUpdateData” accesată prin intermediul index-ului primit ca parametru de „requestId”. Dacă input-urile au fost completate corect, se va apela metoda „updateRequestStauts”, care primește ca parametru valoare din variabila „requestUpdateData” accesibilă la index-ul „requestId” primit ca parametru. Dacă totul a fost validat și actualizat conform server-ului de back-end, comportamentul va fi același ca în funcția prezentată anterior, se va apela funcția „refreshPage”, iar utilizatorul va putea continua să răspunde la cererile utilizatorilor.

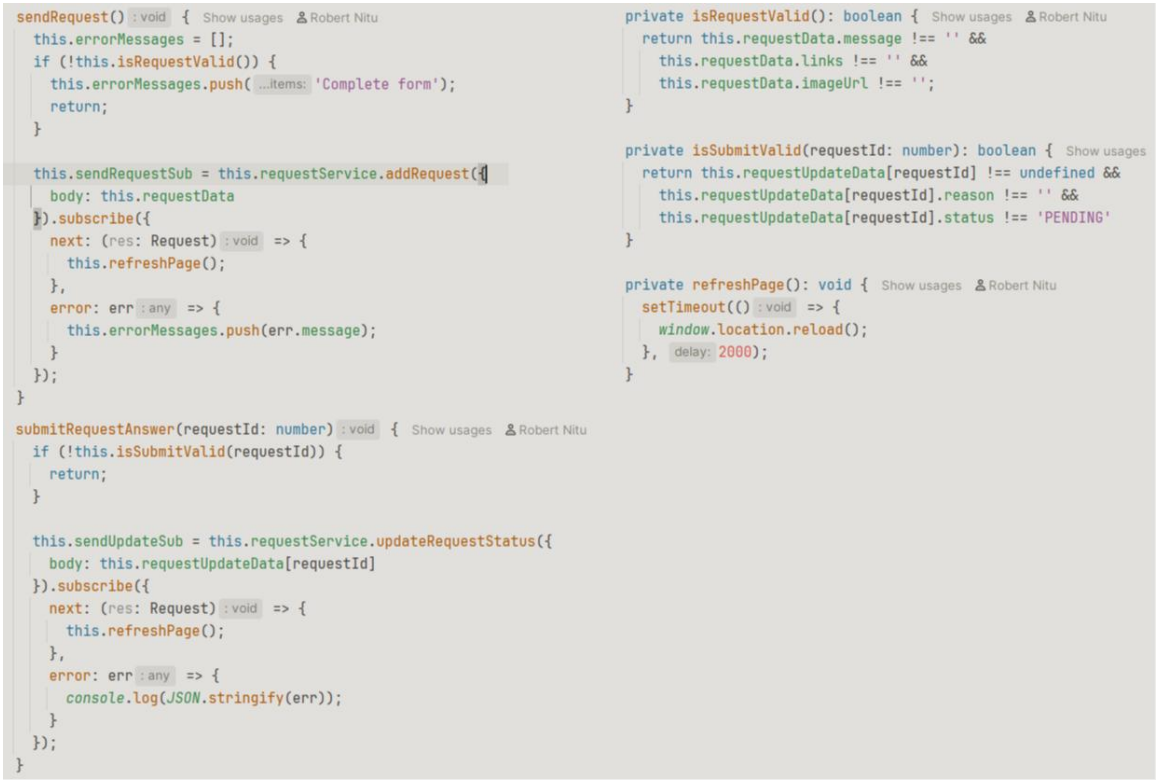


Fig 3. 57 Componentă cereri – TypeScript, partea 3

3.5.12 Componenta Piețelor

Această componentă va afișa toate piețele agroalimentare înscrise în aplicație. Conform fig. 3.58, este folosită o buclă repetitivă de tip „ngFor”, care afișează informații precum imaginea de ansamblu a pieței, numele, locația unde poate fi găsită, descrierea, iar în final un buton care va trimite utilizator către toate tarabele care se găsesc în intermediul pieței, urmat de un program de funcționare.

```
<div class="container py-4">
  <div class="row row-cols-1 row-cols-md-2 g-4 mb-5">
    <div class="col" *ngFor="let market : MarketResponse of markets">
      <div class="card shadow-sm rounded-4 overflow-hidden h-100">
        <img src="{{market.imageUrl}}"
             alt="{{market.name}}"
             class="card-img-top object-fit-cover market-image"/>
        <div class="card-body d-flex flex-column justify-content-between">
          <div>
            <h4 class="card-title fw-bold mb-2">
              {{ market.name }}
              <span class="text-muted fs-6">- {{ market.location }}</span>
            </h4>
            <p class="card-text mb-3">{{ market.description }}</p>
          </div>

          <div class="d-flex justify-content-between align-items-center mt-auto">
            <a class="btn btn-outline-primary px-4 rounded-pill"
               (click)="goToMarketStalls(market.id ?? 0)">
              {{ query: 'markets.to-stalls' | translate }}
            </a>
            <small class="text-muted">
              {{ query: 'markets.schedule' | translate }}: {{ market.schedule }}
            </small>
          </div>
        </div>
      </div>
    </div>
  </div>

  <div *ngIf="isAdmin" class="card shadow border-0 rounded-4 p-4 mx-auto admin-container"...>
</div>
```

Fig 3. 58 Componentă piețe – HTML, partea 1

Pe lângă afișarea piețelor, utilizatorul care are rol de administrator poate crea piețe pe baza cererilor create. Conform fig. 3.59, întregul „div” este afișat doar dacă variabila „isAdmin” are valoarea „true”. În acest caz, utilizatorul va avea un meniu în plus față de restul utilizatorilor, fiind vorba de multiple input-uri care definesc o piață, anume nume, locație, descriere, program de funcționare, imagine reprezentativă și ce utilizator este proprietar asupra acesteia. Toate aceste input-uri sunt salvate în variabila „marketData”, cu ajutorul tag-ului „ngModel”, iar ulterior în funcția de „createMarket” prezentă la finalul tuturor input-urilor, se va trimite acest model construit pe baza răspunsurilor din input.



Fig 3. 59 Componentă piețe – HTML, partea 2

Conform fig. 3.60, clasa „MarketsComponent” are implementate metodele „OnInit” și „OnDestroy”, specific Angular. În partea de sus sunt definite variabilele ce vor interacționa cu partea de HTML. Variabila „markets” conține informațiile despre toate pieţele găsite în tabela „Markets” din baza de date, alături de aceasta este definităi variabila „marketSub” care este de tip observabil. Cealaltă variabila de același tip este „addMarketSub”, care va fi folosită în momentul în are se creează o piață nouă. Variabila „isAdmin” este responsabilă pentru rolul de administrator al utilizatorului, iar „marketData” conține datele necesare pentru crearea unei piețe, câmpurile din această variabilă fiind modificate prin intermediul input-urilor prezentate în pagina HTML. Serviciile injectate în această componentă sunt „router”, folosită la rutarea către diferite componente, „marketsService”, folosită la adăugarea și extragerea informațiilor din server-ul de back-end, iar ultima fiind „TokenService” ce va fi folosită pentru a extrage valoarea de administrator din utilizator. În funcția specifică Angular de „OnInit”, se setează variabila „isAdmin” cu ajutorul service-ului de „TokenService” și se apelează funcția „getMarketsData”. În cealaltă funcție specifică, „OnDestroy”, se verfică dacă variabilele de tip observabil au fost inițialiate și se opresc din execuție.

```
export class MarketsComponent implements OnInit, OnDestroy {

  markets: MarketResponse[] = [];
  marketSub: Subscription | undefined;

  isAdmin = false;
  marketData: Market = {
    description: '',
    imageUrl: '',
    location: '',
    name: '',
    ownerId: 0,
    schedule: ''
  };
  addMarketSub: Subscription | undefined = undefined;

  constructor( no usages  Robert Nitu
    private router: Router,
    private marketsService: MarketService,
    private tokenService: TokenService
  ) {

  }
```

```
ngOnInit(): void {  no usages  Robert Nitu
  this.isAdmin = this.tokenService.admin === 1;

  this.getMarketsData();
}

ngOnDestroy(): void {  no usages  Robert Nitu
    if (this.marketSub !== undefined)
    {
      this.marketSub.unsubscribe();
      console.log( message: 'marketSub unsubscribed');
    }

    if (this.addMarketSub !== undefined) {
      this.addMarketSub.unsubscribe();
      console.log( message: 'addMarketSub unsubscribed');
    }
}
```

Fig 3. 60 Componentă piețe – TypeScript, partea 1

Conform fig. 3.61, funcția precizată anterior numită „getMarketsData” este responsabilă de extragerea informațiilor legate de toate pieţele găsite în tabela „Markets” din baza de date. În cadrul acestei funcții se apelează metoda „getAllMarkets” din service-ul „marketService”, acesta fiind generat cu ajutorul OpenAPI. Valoarea returnată de această metoda este una tip de observabil, care este stocată în variabila „marketSub”. Odată primit răspuns de pe server-ul de back-end, variabila „markets” este populată cu un obiect de tipul „MarketResponse[]”. Funcția specifică pentru crearea de piețe este numită „createMarket”, care în primă fază verifică dacă input-urile au fost introduse cu ajutorul funcției „isValidMarket”, iar apoi se apelează metoda „addMarket” tot din același service prezentat anterior, doar că de această dată se va trimite ca parametru variabila „marketData”, iar în cazul în care totul este validat și introdus, se va apela funcția „refreshPage”. Această funcție va reîmprospăta pagina web dupa un timeout de 2 secunde. Ultima funcție prezentă este cea numită „goToMarketStalls”, care va naviga către ruta „stalls” alături de „marketId” primit ca parametru.



Fig 3. 61 Componentă piețe – TypeScript, partea 2

3.5.13 Componenta Tarabelor

Această componentă are rolul de a afișa toate tarabele înregistrate unei piețe. Conform fig. 3.62, întregul container „div” se va afișa doar dacă în variabilele „market” și „stalls” au fost stocate date cu privire la piețe și tarabe. În interiorul acestui container există un buton care va ruta utilizatorul către pagina generală de vizualizat toate piețele înscrise în aplicație. Mai jos este un „div” care are c lasa de „card”, ce afișează informații despre numele și locația pieței din care fac parte următoarele tarabe. Dacă există tarabe înregistrare în cadrul pieței, condiția din „ngIf” va fi îndeplinită, iar apoi cu ajutorul unei bucle repetitive de tip „ngFor” se vor afișa informații despre fiecare tarabă. Aceste informații sunt imaginea tarabei, numele acesteia, programul de funcționare și o scurtă descriere. Fiecare tarabă va avea un buton de „comandă”, unde la apăsare va redirecționa utilizatorul către componenta specifică tarabei.

```
<div *ngIf="market && stalls" class="container py-4">
  <div class="mb-4">
    <button class="btn btn-outline-secondary rounded-pill shadow-sm"
            (click)="goBack()">
      <i class="fas fa-arrow-left me-2"></i> {{ query: 'stalls.go-back' | translate }}
    </button>
  </div>

  <div class="card mb-4 text-center">
    <div class="card-body">
      <h4 class="card-title">{{ market.name }}</h4>
      <p class="card-text text-muted">{{ market.location }}</p>
    </div>
  </div>

  <div *ngIf="stalls.length > 0; else noStalls" class="row g-4">
    <div class="col-12 col-md-6 col-lg-4" *ngFor="let stall : StallResponse of stalls">
      <div class="card h-100 shadow-sm border-0">
        <img src="{{stall.imageUrl}}"
             class="card-img-top"
             alt="{{ stall.name }}"/>

        <div class="card-body d-flex flex-column">
          <h5 class="card-title">{{ stall.name }}</h5>
          <h5 class="card-subtitle">{{ stall.schedule }}</h5>
          <p class="card-text flex-grow-1">{{ stall.description }}</p>
          <a class="btn btn-primary mt-auto"
             (click)="goToStall(stall.id!)">
            {{ query: 'stalls.order' | translate }}
          </a>
        </div>
      </div>
    </div>
  </div>
</div>
```

Fig 3. 62 Componentă tarabe – HTML, partea 1

Următoarea parte a acestei componente este văzută doar din perspectivă proprietarului de piață, unde acesta poate adăuga tarabe în cadrul pieței sale. Conform fig. 3.63, container-ul este vizibil doar dacă variabila „isMarketOwner” are valoarea pozitivă, în cazul acesta va avea de completat câteva input-uri care se vor salva în variabila numită „stallData”. Vor fi adăugate în input-uri valori despre numele tarabei, o scurtă descriere, programul de funcționare și desigur, ID-ul de ordine din baza de date al proprietarului de tarabă. Pe lângă acestea, trebuie definit și stoc-ul de produse al tarabei. Acest lucru se realizează cu ajutorul tipului de date „ProductData”, care are în componență valori pentru „id”, „name”, „quantity” și „price”. Acestea sunt stocate în variabila „productsData”, iar adăugarea unui produs nou se face cu ajutorul funcției „addProduct”, iar ulterior cu ajutorul buclei repetitive „ngFor”, se va permite editarea câmpurilor fiind prezent tag-ul „ngModel”. Dacă proprietarul de piață a introdus un produs greșit sau este în plus, a fost adăugat un buton pentru a șterge respectiva intrare, iar apoi elemntele HTML se actualizează în mod automat cu ajutorul construcției Angular. După ce aceste date au fost introduse, în partea de jos se poate crea taraba respectivă folosind butonul de „create”.



```
<div *ngIf="isMarketOwner" class="container py-4">
  <div class="card shadow border-0 rounded-4 p-4 mb-5 mx-auto market-owner-container">
    <h3 class="text-center mb-4">{{ query: 'stalls.create-stall' | translate }}</h3>
    <hr class="mb-4">

    <div class="mb-3">
      <label for="stallName" class="form-label">{{ query: 'stalls.name' | translate }}</label>
      <input [(ngModel)]="stallData.name"
             type="text"
             class="form-control rounded-3"
             id="stallName"
             required>
    </div>

    <div class="mb-3">
      <label for="stallDescription" class="form-label">{{ query: 'stalls.description' | translate }}</label>
      <textarea [(ngModel)]="stallData.description"
                class="form-control rounded-3"
                id="stallDescription"
                rows="3"
                required></textarea>
    </div>

    <div class="mb-3">
      <label for="stallSchedule" class="form-label">{{ query: 'stalls.schedule' | translate }}</label>
      <input [(ngModel)]="stallData.schedule"
             type="text"
             class="form-control rounded-3"
             id="stallSchedule"
             placeholder="ex: 08:00 - 16:00"
             required>
    </div>

    <div class="mb-3">
      <label class="form-label d-block">{{ query: 'stalls.products' | translate }}</label>
      <button (click)="addProduct()"
              type="button"
              class="btn btn-success px-4 py-2 mb-3 rounded-pill shadow-sm">
        <i class="fas fa-plus me-2"></i> {{ query: 'stalls.add-product' | translate }}
      </button>

      <div *ngFor="let product : ProductData of productsData" class="card mb-3 shadow-sm border-0 rounded-
        <div class="d-flex justify-content-between align-items-center mb-3">
          <h5 class="mb-0">{{ query: 'stalls.product' | translate }} #{{ product.id }}</h5>
          <button class="btn btn-outline-danger btn-sm rounded-pill"
                  type="button"
                  (click)="removeProduct(product.id)">
            <i class="fas fa-trash-alt me-2"></i> {{ query: 'stalls.remove-product' | translate }}
          </button>
        </div>

        <div class="mb-3">
          <label class="form-label">{{ query: 'stalls.product-name' | translate }}</label>
          <input [(ngModel)]="product.name"
                 type="text"
                 class="form-control rounded-3"
                 placeholder="{{ query: 'stalls.product-name' | translate }}">
        </div>

        <div class="mb-2">
          <label class="form-label">{{ query: 'stalls.product-quantity' | translate }}</label>
          <input [(ngModel)]="product.quantity"
                 type="number"
                 class="form-control rounded-3"
                 placeholder="{{ query: 'stalls.product-quantity' | translate }}">
        </div>

        <div class="mb-2">
          <label class="form-label">{{ query: 'stalls.product-price-kg' | translate }}</label>
          <input [(ngModel)]="product.price"
                 type="number"
                 class="form-control rounded-3"
                 placeholder="{{ query: 'stalls.product-price-kg' | translate }}">
        </div>
      </div>
    </div>

    <div class="mb-3">
      <label for="stallImageUrl" class="form-label">{{ query: 'stalls.image-url' | translate }}</label>
      <input [(ngModel)]="stallData.imageUrl"
             type="url"
             class="form-control rounded-3"
             id="stallImageUrl"
             placeholder="https://example.com/image.jpg"
             required>
    </div>

    <div class="text-center mb-4" *ngIf="stallData.imageUrl">
      <img src="{{stallData.imageUrl}}"
           alt="Profile stallData"
           class="rounded-circle shadow-sm stall-image-preview"/>
      <p class="small text-muted mt-2">{{ query: 'stalls.image-preview' | translate }}</p>
    </div>

    <div class="mb-3">
      <label for="stallOwnerId" class="form-label">{{ query: 'stalls.owner-id' | translate }}</label>
      <input [(ngModel)]="stallData.ownerId"
             type="text"
             class="form-control rounded-3"
             id="stallOwnerId">
    </div>

    <div class="d-flex justify-content-end">
      <button (click)="createStall()"
              type="button"
              class="btn btn-success px-4 py-2 rounded-pill shadow-sm">
        <i class="fas fa-plus me-2"></i> {{ query: 'stalls.add-stall' | translate }}
      </button>
    </div>
  </div>
</div>
```

Fig 3. 63 Componentă tarabe – HTML, partea 2

Fișierul de TypeScript are clasa „StallsComponent”, conform fig. 3.64. Această clasa are implementate metodele specifice Angular de „OnInit” și „OnDestroy”. În susul clasei sunt definite câteva variabile precum cele pentru „market”, anume „marketId”, care reprezintă ID-ul de ordine din baza de date pentru piața în care navighează, această valoare este inițialiată în cadrul metodei „OnInit”, unde se extrage folosind „route.snapshot.paramMap.get” valoarea „id” din rută. Variabila „market” este de tipul „MarketResponse”, iar aceasta se populează când se accesează server-ul de back-end. Pentru această variabilă a fost creată si variabila de tip observabil numită „marketSub”. Următoarele variabile sunt „stalls”, care este de tipul „StallResponse[]”, unde se va inițializa cu informații despre toate tarabele specifice pieței marcate cu valoarea din variabila „marketId”. A fost creată și o variabilă de tip observabil pentru aceasta, având numele „stallsSub”. Variabila numită „isMarketOwner” va verifica dacă utilizatorul deține sau nu piața marcată cu ajutorul variabilei „marketId”. În variabbila numită „stallData” vor fi salvate valorile din input-ul pentru crearea tarabei. Totodată, variabila „productsData” este cea responsabilă cu stocarea produselor în momentul creării, utlerior devenind un obiect de tip string, în urmă operației de „JSON.stringify”, care va transforma obiectul într-un string. Serviciile injectate în această componentă sunt „ActivatedRoute”, care va putea returna valoarea parametrului „id” din adresa rutei, „Router”, care va fi folosită pentru schimbarea rutei către o altă componentă, „stallsService” și „marketService”, care sunt servicii

generate în mod automat cu ajutorul uneltei OpenAPI, iar în final, service-ul „TokenService”. În cadrul metodei „OnInit” se inițializează valoarea variabilei numite „marketId”, prezentate anterior, se va popula variabila „isMarketOwner”, folosindu-se de service-ul „TokenService”, unde se va compara dacă „marketId” salvat în datele utilizatorului este egal cu valoarea variabilei „marketId” definite anterior. În cazul în care sunt egale, înseamnă că utilizatorul este proprietar al tarabei și are drepturi specifice. În metoda „ngOnDestroy”, se verifică dacă cele două variabile de tip observabil au fost inițializate, iar în caz contrat se va opri execuția acestora.

```
export class StallsComponent implements OnInit, OnDestroy {

  marketId: number = -1;
  market: MarketResponse | undefined;
  marketSub: Subscription | undefined;

  stalls: StallResponse[] = [];
  stallsSub: Subscription | undefined;

  isMarketOwner = false;
  stallData: Stall = {
    description: '',
    imageUrl: '',
    marketPlacedId: 0,
    name: '',
    products: '',
    schedule: '',
    ownerId: 0
  };
  productsData: ProductData[] = [];
  createStallSub: Subscription | undefined = undefined;

  constructor( no usages  Robert Nitu
    private route: ActivatedRoute,
    private router: Router,
    private stallsService: StallService,
    private marketService: MarketService,
    private tokenService: TokenService
  ) {}

  ngOnInit() {  no usages  Robert Nitu
    this.marketId = Number(this.route.snapshot.paramMap.get('id'));

    this.isMarketOwner = this.tokenService.market === this.marketId;

    this.getStalls();
    this.getMarket();
  }

  ngOnDestroy(): void {  no usages  Robert Nitu
    if (this.stallsSub !== undefined) {
      console.log('stallsSub unsubscribed');
      this.stallsSub.unsubscribe();
    }

    if (this.marketSub !== undefined) {
      console.log('marketSub unsubscribed');
      this.marketSub.unsubscribe();
    }
  }
```

Fig 3. 64 Componentă tarabe – TypeScript, partea 1

Conform fig. 3.65, funcția denumită „getStalls” va apela metoda „getStallsByMarketId” din cadrul service-ului „stallsService”, adăugând ca parametru valoarea „marketId” definită anterior. Această metodă va returna o variabilă de tip observabil. Când va veni răspunsul de pe server-ul de back-end, variabila „stalls” se va popula cu răspunsul venit, iar pagina HTML va fi populată cu informații. În sinteză, funcția numită „getMarket” va apela metoda „getMarket” din cadrul service-ului „marketService”, adăugând ca parametru valoarea „marketId” definită anterior. Această metoda va returna o variabilă de tip observabil. Când va veni răspunsul de pe server-ul de back-end, variabila „market” se va popular cu răspunsul venit, iar pagina HTML va fi populate cu informații.

```
private getStalls(): void {  Show usages  Robert Nitu
  this.stallsSub = this.stallsService.getStallsByMarketId({
    id: this.marketId
  }).subscribe({
    next: (data: StallResponse[]) : void  => {
      this.stalls = data;
    },
    error: (error: Error) : void  => {
      console.log(error);
    }
  });
}

private getMarket(): void {  Show usages  Robert Nitu
  this.marketSub = this.marketService.getMarket({
    id: this.marketId
  }).subscribe({
    next: (data: MarketResponse) : void  => {
      this.market = data;
    },
    error: (error: Error) : void  => {
      console.log(error);
    }
  });
}
```

Fig 3. 65 Componentă tarabe – TypeScript, partea 2

Conform fig. 3.66, funcția denumită „createStall” este responsabilă pentru crearea unei tarabe în cadrul pieței selectate. În primă fază se inițializează câmpul „products” cu ajutorul operației de „stringify” specifică „JSON”, unde se trec elementele din forma „ProductData[]” în tipul de date text (string). După inițializare se va verifica cu ajutorul funcției „isStallValid”, dacă input-urile au fost completate. Câmpul „marketPlacedId” este inițializat imediat după verificarea anterioară, făcând referire la care piață trebuie să aparțină taraba creată. Ulterior se va apela metoda „addStall” din cadrul service-ului „stallsService”, având ca parametru valoarea variabilei „stallData”. Această metodă va returna o variabilă de tip observabil unde se va salva în variabila numită „createStallSub”. După ce server-ul de back-end va valida și crea taraba în tabela „Stalls” din baza de date, se va apela funcția „refreshPage”, unde după un timeout de 2 secunde se va reîmprospăta pagina web.



Fig 3. 66 Componentă tarabe – TypeScript, partea 3

Conform fig. 3.67, funcția denumită „addProduct” este responsabilă pentru adăugare de produse noi pentru meniul de creare de tarabă, specific doar proprietarului de piață. În această funcție se inițializează o variabila cu tipul de date „ProductData”, setându-se valori implicite pentru câmpurile acesteia. Câmpul „id” va fi asignat mereu cu lungimea la momentul respectiv al variabilei „productsData”. După ce a fost inițializată, variabila „data” este adăugată în variabila de tip array „productsData”. Funcția „removeProduct” primește ca parametru un număr întreg „id”, care va fi căutat în cadrul variabilei „productsData”. Căutarea se face cu ajutorul metodei „findIndex” specific array-urilor în cadrul TypeScript. Dacă valoarea este găsită se va returna o valoare diferită de „-1”. În cazul în care a fost găsită, se va șterge intrarea respectivă folosind metoda „splice”. Următoarele funcții au rolul de navigare către alte componente, funcția „goToStall”, va naviga către taraba cu id-ul specific, iar funcția „goBack”, va naviga utilizatorul către ruta „markets” unde se află toate pieţele disponibile.



Fig 3. 67 Componentă tarabe – TypeScript, partea 4

3.5.14 Componenta Tarabă

Această componentă este responsabilă pentru afișarea produselor din cadrul unei tarabe și plasarea unei comenzi de produse. Conform fig. 3.67, această component este vizibilă doar dacă există valoare în variabila „stall”, cu ajutorul „ngIf”, unde în prima parte din container este plasat un buton ce are ca scop navigarea către toate tarabele din cadrul pieței, cu ajutorul funcției „goBack”. Următoarea informație afișată este numele pieței din care face parte taraba respectivă, cu ajutorul variabilei „marketName”, iar lângă este afișat numele tarabei, cu ajutorul variabilei „name”. În josul acestor informații se află descrierea generală a tarabei. Următoarele doua containere cu tag-ul de „div”, anume partea vizibilă doar de proprietarul de tarabă și cel al utilizatorului logat, unde va putea să plaseze comenzi de produse.

```
<div *ngIf="stall" class="container py-4">
  <div class="mb-4">
    <button class="btn btn-outline-secondary rounded-pill shadow-sm"
            (click)="goBack()">
      <i class="fas fa-arrow-left me-2"></i> {{ query: 'stalls.go-back' | translate }}
    </button>
  </div>

  <div class="card mb-4 text-center">
    <div class="card-body">
      <h4 class="card-title">{{ stall.marketName }} - {{ stall.name }}</h4>
      <p class="card-text text-muted">{{ stall.description }}</p>
    </div>
  </div>

  <div *ngIf="isStallOwner" class="container py-4"...>

  <div class="card mb-4 shadow-sm rounded-4 overflow-hidden mx-auto stall-container"...>
</div>
```

Fig 3. 67 Componentă tarabă – HTML, partea 1

Conform fig. 3.90, această parte de cod are rolul de a plasa comenzi cu produsele prezente la tarabă. În susul acestui container este situate o imagine reprezentativă tarabei selectate, iar următoarea parte este alcătuită dintr-un „div” de tipul „card”, care are adăugate informații legate de programul de funcționare și o scurtă descriere a tarabei. Cea mai important parte a acestui container este plasarea comenzii. Produsele sunt copiate în variabila „productsData”, unde cu ajutorul buclei repetitive „ngFor” se vor afișa informații despre numele produslui, cantitatea disponibilă și prețul per kilogram. În continuare va fi disponibil un input de tip „number”, unde se va putea introduce cantitatea dorită pentru cumpărare. Valorile introduse în aceste imputuri se vor salva în variabila „orderData” cu ajutoul tag-ului „ngModel”. Ultimul camp ce trebuie completat este adresa de livrare, unde a fost adăugat un input de tip text imediat după introducerea cantității dorite de cumpărat. Ultima parte a acestui container este reprezentată prin butonul de „submit”, unde se plasează comanda, dar și o informație legată de prețul total al comenzii până în acel moment. Această valoare se va actualiza în

mod automat pe măsură ce utilizatorul completează input-ul de tip „number” din bucla repetitivă prezentată anterior.

```
<div class="card mb-4 shadow-sm rounded-4 overflow-hidden mx-auto stall-container">
  <img src="{{stall.imageUrl}}"
       class="card-img-top object-fit-cover stall-image"
       alt="{{stall.name}}">

  <div class="card-body">
    <h4 class="card-title fw-bold mb-2">
      {{ stall.schedule }} <span class="text-muted fs-6"></span>
    </h4>

    <p class="card-text mb-3">
      {{ stall.description }}
    </p>

    <div class="mb-4">
      <label class="form-label fs-5 fw-semibold d-block mb-3">
        {{ query: 'stalls.products' | translate }}
      </label>

      <div *ngFor="let product : ProductData of productsData" class="card mb-3 shadow-sm border-0 rounded-4 p-3">
        <div class="mb-2">
          <h5 class="mb-1">{{ product.name }}</h5>
          <p class="text-muted mb-0">{{ query: 'stalls.quantity-available' | translate }}:
            {{ product.quantity }}</p>
          <p class="text-muted mb-1">{{ query: 'stalls.product-price-kg' | translate }}:
            {{ product.price }}RON</p>
        </div>

        <div class="mt-3">
          <label class="form-label">{{ query: 'stalls.enter-quantity' | translate }}</label>
          <input [(ngModel)]="orderData[product.id].quantity"
                 type="number"
                 min="1"
                 [max]="product.quantity"
                 class="form-control rounded-3">
        </div>
      </div>

      <div class="mt-3">
        <label class="form-label">{{ query: 'stalls.enter-address' | translate }}</label>
        <input [(ngModel)]="address"
               type="text"
               class="form-control rounded-3">
      </div>

      <div class="text-end mt-4">
        <label class="form-label fs-5 fw-semibold d-block mb-3">
          {{ query: 'stalls.total' | translate }}: {{ calculateTotal() }} RON
        </label>
        <button class="btn btn-primary px-4 py-2 rounded-pill shadow-sm"
                (click)="submitOrder()">
          <i class="fas fa-shopping-cart me-2"></i> {{ query: 'stalls.place-order' | translate }}
        </button>
      </div>
    </div>

  </div>
```

Fig 3. 68 Componentă tarabă – HTML, partea 2

Prima parte din cadrul container-ului dedicat doar proprietarului de tarabă constă în actualizarea stocului de produse al tarabei. Conform fig. 3.69, în primă parte sunt afișate informații cu privire la categoria în care se află. Următoarea parte este reprezentată prin afișarea tuturor produselor curente, cât și informațiile despre acestea, anume numele produsului, cantitatea totală și prețul per kilogram. A fost adăugat și un buton care permite adăugarea unui produs nou, această acțiune fiind posibile cu ajutorul funcției „addProduct”. Produsele vor fi afișate cu ajutorul buclei

repetitive „ngFor”, unde se afișează produsele prin intermediul variabilei „stallOwnerProductsData”. Pentru fiecare element va fi afișat un buton pentru ștergerea produsului curent, însă și câte un input pentru fiecare din câmpurile specifice unui produs, numele, cantitate și prețul per kilogram. Ultima interacțiune ce se poate face este actualizarea tuturor produselor.



Fig 3. 69 Componentă tarabă – HTML, partea 3

Ultima parte din containerul dedicat proprietarului de tarabă afișează toate comenzile din cadrul tarabei respective. Conform fig. 3.70, este afișat un text informativ unde ajută la împărțirea containerului de proprietar de tarabă. Ulterior după, dacă vor exista comenzi vor fi stocate în variabila „stallOrders” și se verifică cu ajutorul tag-ului „ngIf”. Astfel, este creat un table care are coloanele specifice cu tabela de „Orders” din baza de date, mai exact sunt informații cu privire la numele utilizatorului, adresa de livrare, produsele comandate, prețul total, datele despre plasarea comenzii, acceptarea și livrarea comenzii, cât și statusul. În continuare, cu ajutorul buclei repetitive „ngFor” se va popula tabelul cu valori din variabila „stallOrders”, care reprezintă toate comenzile din cadrul tarabei respective. Proprietarul de piață va putea schimba status-ul comenzii cu ajutoul tag-ului de „select”, unde se poate schimba status-ul. Valorile acestui status este setat cu ajutorul tag-ului „ngValue”, 0 reprezentând „PENDING” (În așteptare), 1 reprezentând „ACCEPTED” (În livrare), iar ultima, 2 reprezentând „DELIVERED” (Livrată). Actualizarea în baza de date se face cu ajutorul tag-ului „onStatusChange”, care este declansată în momentul în care se selectează o valoare din tag-ul „select”. Ca parametrii sunt transmise id-ul de ordine din baza de date al comenzii, alături de „$event” care reprezintă valoarea din interiorul tag-ului „option”.



Fig 3. 70 Componentă tarabă – HTML, partea 4

Conform fig. 3.71, clasa „StallComponent” are implementate metodele „OnInit” și „OnDestroy” specifice Angular. Ulterior, este zona de declarare de variabile care vor fi folosite în cadrul fișierului TypeScript. Variabilele care stochează id-uri sunt „userId”, care salvează ID-ul de ordine al utilizatorului din baza de date, iar „stallId”, salvează ID-ul de ordine al utilizatorului al tarabei din baza de date. Informațiile despre tarabă sunt salvate în variabila „stall”, care ulterior are nevoie de variabila de tip observabil numită „stallSub”. Tot de același tip, variabila „stallOrders” stochează informații despre toate comenzile asupra tarabei cu ID-ul de ordine din variabila „stallId”. Această variabilă are nevoie la rândul ei de o variabila de tip observabil, iar aceea este „stallOrdersSub”. Au fost create și variabile care ajută la plasarea de comenzi, iar acelea sunt „productsData”, lista de produse care va conține datele actualizate ale produselor, „orderData”, conține informații despre plasarea comenzii, aceasta fiind populată cu ajutorul input-urilor prezente în pagina de HTML. Totodată, variabila „address”, va conține informația cu privire la adresa de livrare care va fi introdusă în baza de date. Variabila „isStallOwner” se va seta doar dacă taraba la care este utilizatorul este a lui, verificându-se ulterior. Această clasă are injectate următoarele servicii: „ActivatedRoute”, pentru a prelua din rută id-ul tarabei, „Router”, care va ajuta la rutarea utilizatorului către o altă componentă, „TokenService”, care va prelua informații setate în momentul logării, „StallService” și „OrderService” sunt servicii generate în mod automat cu ajutorul uneltei OpenAPI. Acestea au rolul de a interacționa cu server-ul de back-end. Sunt prezentate metodele specific Angular. Metoda „ngOnInit” este apelată în momentul în care se inițializează componenta. În cadrul acesteia, se vor popular variabilele de „userId”, cu id-ul de ordine al utilizatorului din baza de date luat din service-ul de „TokenService”, „stallId” este preluat ruta activă, „isStallOwner”, este verificat dacă utilizatorul deține taraba la care are componenta active, acest lucru se face comparand valoarea „stall” din service-ul „TokenService” cu valoarea „stallId” extrasă anterior. Metoda „ngOnDestroy” se declanșează în momentul în care se

navighează pe altă componentă. În această funcție, se va verifica fiecare variabilă de tip observabil dacă a fost inițializată, iar în cazul contrat se va opri execuția acesteia.

```
export class StallComponent implements OnInit, OnDestroy {

  userId: number = -1;

  stallId: number = -1;
  stall: StallResponse | undefined;
  stallSub: Subscription | undefined;

  productsData: ProductData[] = [];
  orderData: ProductData[] = [];

  isStallOwner = false;
  stallOwnerProductsData: ProductData[] = [];
  updateSub: Subscription | undefined;

  orderSub: Subscription | undefined;
  address: string = '';

  stallOrders: OrderResponse[] = [];
  stallOrdersSub: Subscription | undefined;

  orderUpdateSub: Subscription | undefined;

  constructor(  no usages  Robert Nitu
    private route: ActivatedRoute,
    private router: Router,
    private tokenService: TokenService,
    private stallService: StallService,
    private orderService: OrderService) {
  }
```

```
  ngOnInit(): void {  no usages  Robert Nitu
    this.userId = this.tokenService.id;
    this.stallId = Number(this.route.snapshot.paramMap.get('id'));

    this.isStallOwner = this.tokenService.stall === this.stallId;

    this.getStall();
    this.getOrdersFromStall();
  }

  ngOnDestroy(): void {  no usages  Robert Nitu
    if (this.stallSub !== undefined) {
      this.stallSub.unsubscribe();
      console.log('stallSub unsubscribed');
    }

    if (this.updateSub !== undefined) {
      this.updateSub.unsubscribe();
      console.log('updateSub unsubscribed');
    }

    if (this.orderSub !== undefined) {
      this.orderSub.unsubscribe();
      console.log('orderSub unsubscribed');
    }

    if (this.stallOrdersSub !== undefined) {
      this.stallOrdersSub.unsubscribe();
      console.log('stallOrdersSub unsubscribed');
    }

    if (this.orderUpdateSub !== undefined) {
      this.orderUpdateSub.unsubscribe();
      console.log('orderUpdateSub unsubscribed');
    }
  }
```

Fig 3. 71 Componentă tarabă – TypeScript, partea 1

Conform fig. 3.72, funcția „getStall” folosește metoda cu același nume din cadrul service-ului „stallService”, adăugând ca parametru id-ul de ordine al tarabei din baza de date, folosind variabila „stallId”. Metoda va returna o variabilă de tip observabil care va fi salvată în variabila „stallSub”. Răspunsul venit de la server-ul de back-end are tipul „StallResponse”. Odată primit răspunsul, se va salva în variabila „stall”, unde se vor afișa toate informațiile din cadrul acesteia în pagina de HTML, iar ulterior se vor popula variabilele „productsData”, care va fi folosită pentru actualizarea stocului de produse prin intermediul proprietarului de tarabă și „orderData”, care va conține toate produsele din tarabă, având cantitatea 0, urmând ca utilizatorul să completeze acest câmp cu cantitatea dorită. Variabila „stallOwnerProductsData” este o copie realizată cu funcția specifică TypeScript, numită „structuredClone” a variabilei „productsData”. Următoarea funcție prezentă este „getOrdersFromStall”, aceasta apelează metoda „getOrdersFromStallId” din cadrul service-ului „orderService”, adăugând ca parametru id-ul de ordine al tarabei prin variabila „stallId”. Metoda returnează o variabilă de tipul observabil, ulterior salvată în variabila „stallOrdersSub”. Odată primit răspunsul de la server-ul de back-end, se va salva răspunsul în variabila „stallOrders”.

```
private getStall(): void {  Show usages  Robert Nitu
  this.stallSub = this.stallService.getStall({
    id: this.stallId
  }).subscribe({
    next: (data: StallResponse) : void  => {
      console.log(data);
      this.stall = data;

      if (this.stall.products !== undefined) {
        this.productsData = JSON.parse(this.stall.products);
        this.productsData.forEach(el : ProductData  => {
          this.orderData.push({
            id: el.id,
            name: el.name,
            quantity: 0,
            price: el.price
          });
        });

        this.stallOwnerProductsData = structuredClone(this.productsData);
      }
    },
    error: err : any  => {
      console.log(err);
    }
  });
}
```

```
private getOrdersFromStall(): void {  Show usages  Robert Nitu
  this.stallOrdersSub = this.orderService.getOrdersFromStallId({
    id: this.stallId
  }).subscribe({
    next: (data: OrderResponse[]) : void  => {
      this.stallOrders = data;
    },
    error: err : any  => {
      console.log(err);
    }
  });
}
```

Fig 3. 72 Componentă tarabă – TypeScript, partea 2

Conform fig. 3.73, funcția definită „submitOrder” va plasa comanda dacă aceasta corespunde cerințelor. În variabila „finalOrderData” se va stoca varianta finală a produselor ce utilizatorul dorește să le comande. În cadrul funcției „getFinalOrder”, se va verifica fiecare intrare din variabila „orderData” dacă cantitatea este pozitivă și aceasta este mai mică sau egală decât numărul de produse al tarabei, neputând să se comande o valoare mai mare decât stocul tarabei. Următoarea verificare o face funcția „isOrderValid”, unde se verifică dacă input-urile sunt completate, mai exact dacă a fost introdusă valoare pentru adresă și sunt produse în coșul de cumpărătorui. Următorul pas este de a construi obiectul „data” care este de tipul „Order”, acest tip este modelul al tabelei „Orders” din baza de date. Câmpul de „products” va fi de tipul text, iar pentru acest lucru s-a folosit operația „stringify” din librăria JSON specifică TypeScript, prețul comenzii este valoarea de tip număr cu zecimale care este returnată din funcția „calculateTotal”. Această funcție returnează prețul care este calculat în funcție de cantitate și prețul per kilogram setat de tarabă. Câmpul de status are setată valoarea 0, care este valoarea prestabilită pentru status-ul de „PENDING” (În așteptare). În continuare se vor adăuga informații despre „requeterId”, care reprezintă id-ul de ordine al utilizatorului din baza de date, iar „stallId”, care reprezintă id-ul de ordine al tarabei din baza de date. Având construit acest obiect, se va apela metoda „addOrder” din service-ul „orderService”, având ca parametru obiectul creat anterior. Această metodă returnează o variabilă de tipu observabil. Server-ul de back-end când va vallida și introduce intrarea în tabela „Orders” din baza de date.



Fig 3. 73 Componentă tarabă – TypeScript, partea 3

Conform fig. 3.74, funcția „addProduct” va adăuga un produs nou pentru a actualiza stocul de produse al proprietarului de tarabă. Această funcție inițializează variabila „data” cu tipul de data „ProductData”. Câmpul „id” este setat în funcție de câte produse sunt deja existente. Celelate câmpuri având valori prestabilite, urmând ca ulterior să fie editate de utillizatorul cu rolul de proprietar de tarabă. Funcția definită cu numele „removeProduct”, va șterge produsul cu id-ul dat ca parametru. Acesta va apela metoda „findIndex” asupra listei „stallOwnerProductsData”, iar dacă valoarea acestuia este diferită de „-1”, se va apela metoda de ștergere, anume „splice”.



Fig 3. 74 Componentă tarabă – TypeScript, partea 4

Conform fig. 3.75, funcția definită cu numele „updateProducts”, va salva în variabila „newProducts” valoarea de tip text prin metoda „stringify” a variabilei „stallOwnerProductsData”. Următorul pas este de a construi variabila „productData”, care este de tipul „UpdateProduct”, unde se populează câmpurile de „id”, care reprezintă id-ul de ordine al tarabei din baza de date, iar „products”, reprezintă noile produse în cadrul tarabei. Având obiectul inițializat, se va apela metoda „updateProducts” din service-ul „stallService”, adăugând ca parametru obiectul construit anterior. Metoda va returna o variabilă de tipul observabil care este salvată în variabila „updateSub”. În urma apelului către server-ul de back-end, se va acorda o reîmprospătare a paginii web. Funcția definită cu numele „onStatusChange”, este responsabilă cu actualizarea status-urilor comenzilor plasate în cadrul comenzilor. În primă fază se va construi obiectul cu numele „data” de tipul „UpdateOrderData”. Câmpul „id”, este populat cu id-ul de ordine al tarabei din baza de date, iar câmpul „status” este noua valoare setată prin intermediul tag-ului prezentat în pagina HTML. Ulterior, se va apela metoda „updateOrder” din cadrul service-ului „orderService”, unde ca parametru va fi setată variabila anterior inițializată. În momentul în care se va face actualizarea pe partea server-ului de back-end, se va apela funcția de „refreshPage”, unde se va reîmprospăta pagina web.



```
updateProducts() : void { Show usages Robert Nitu
  const newProducts : string = JSON.stringify(this.stallOwnerProductsData);

  const productData: UpdateProduct = {
    id: this.stall!.id!,
    products: newProducts
  };

  this.updateSub = this.stallService.updateProducts({
    body: productData
  }).subscribe({
    next: (data : {} ) : void => {
      console.log(data);
      setTimeout(() : void => {
        console.log( message: 'enter');
        window.location.reload();
      }, delay: 2000);
    },
    error: err : any => {
      console.log(err);
    }
  });
}
```

```
onStatusChange(orderId: number, $event: any) : void { Show us
  const data: UpdateOrderData = {
    id: orderId,
    status: $event.valueOf()
  };

  this.orderUpdateSub = this.orderService.updateOrder({
    body: data
  }).subscribe({
    next: (data: any) => {
      console.log(data);
      this.refreshPage();
    },
    error: err => {
      console.log(err);
    }
  });
}

goBack() : void { Show usages Robert Nitu
  this.router.navigate(['stalls', this.stall?.marketId]);
}
```

Fig 3. 75 Componentă tarabă – TypeScript, partea 5

4. TESTAREA FUNCȚIONALITĂȚII APLICAȚIEI

Testarea aplicației a fost făcută în două faze, în primă fază fiind nevoie să se testeze server-ul de back-end scris în limbajul de programare Java, cu ajutoru framework-ului Spring Boot. A trebuit să se testeze toate rutele care au fost scrise prezentate anterior, fiind un pas important să funcționeze corect și optim, până la integrarea cu front-end-ul scris în framework-ul Angular. A doua fază fiind testarea front-end-ului, mai exact apelarea metodelor către server-u de back-end, clasele folosite din librăria Bootstrap, serviciile implementate și cele generate cu ajutorul uneltei OpenAPI, iar în final rutele care redirecționează utilizatorul către diferite componente.

4.1 Testarea back-end-ului

Testarea server-ului de back-end a fost realizată cu ajutorul programului destinate apelurilor către un server de back-end, numit „Postman”. Confrom fig. 4.1, se găsește în partea stângă folder-ul numit „AgroNOW”, unde se află toate cererile către server-ul de back-end puse într-un director specific rutei. Au fost create teste pentru fiecare, având pentru „User”, „Markets”, „Stalls”, „Requests” și „Orders”. Acestea având câte un nume specific, cât și operația HTTP care este realizată în urma apelului. Din figură se poate vedea că pentru operațiile făcute în cadrul rutei „User”, sunt folosite operații de POST pentru „register” (înregistare) și „login” (logare). În afară de acestea mai sunt folosite operații de GET pentru „Activate Account” (activarea contului), cât și pentru „Get User” (extragere de informații despre utilizator).

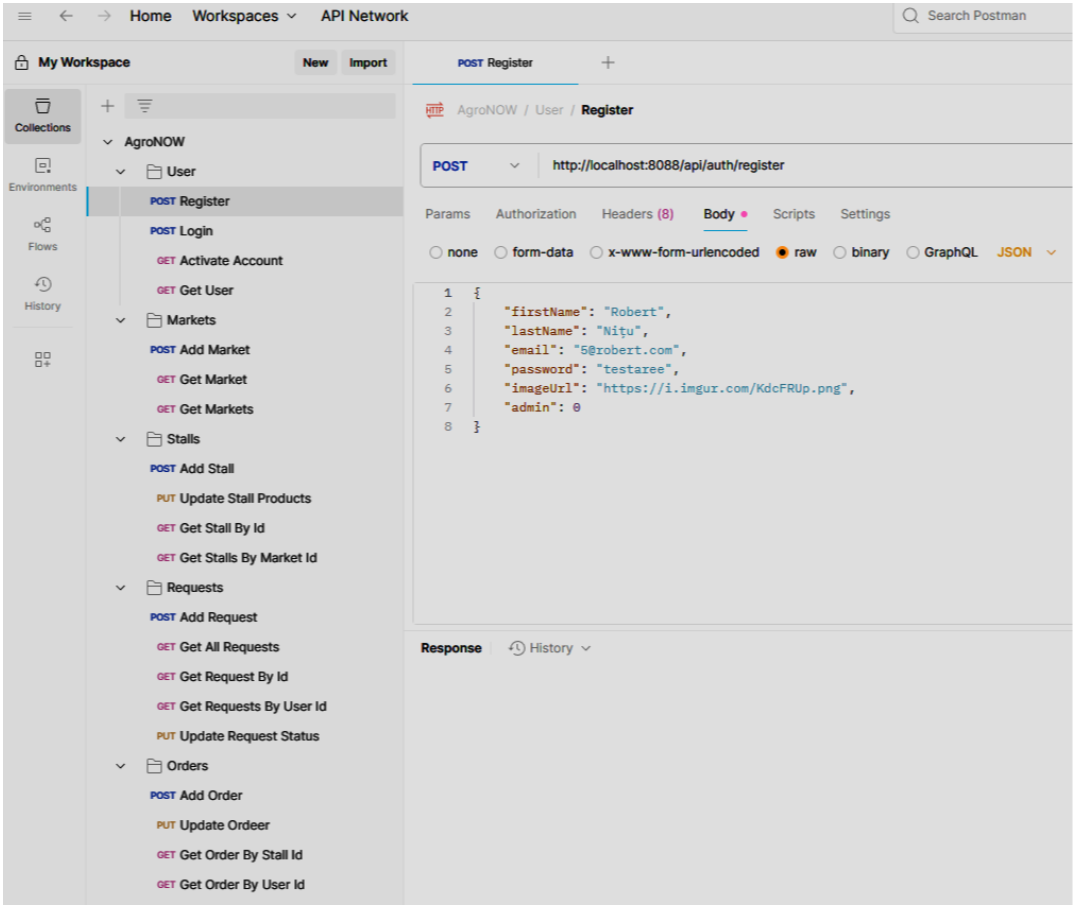


Fig 4. 1 Aplicația „Postman”

Conform fig. 4.1, este prezentat un exemplu de testare al rutei de „auth” pentru crearea unui cont. Este folosită adresa „http://localhost:8080/api/auth/register”, care este responsabilă pentru

introducerea unei intrări noi în tabela „Users” din baza de date. Aceasta are ca paramtru un obiect de tip „JSON”, care are câmpuri legate de „firstName”, „lastName”, „email”, „password”, „imageUrl” și „admin”. Conform fig. 4.2, a fost trimisă cerea cu datele completate în fig. 4.1, iar răspunsul din partea server-ului de back-end a fost unul de tip 500, care indică o eroare pe partea de server. Eroarea este primită într-o formă intuitivă, aceasta indicând ca nu se poate crea contul deoarece este un cont deja înregistrat cu adresa de email dată prin parametru.

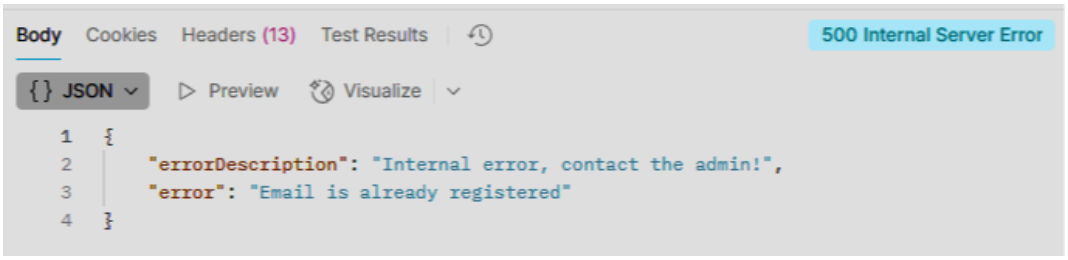


Fig 4. 2 Exemplu de eroare în cadrul operației de „register”

Conform fig. 4.3, a fost modifcat câmpul de email pentru a îndeplini condițiile de înregistrare corecte, iar în urma apelului a fost returnat status-ul de 202, care indică că totul a decurs conform parametrilor.

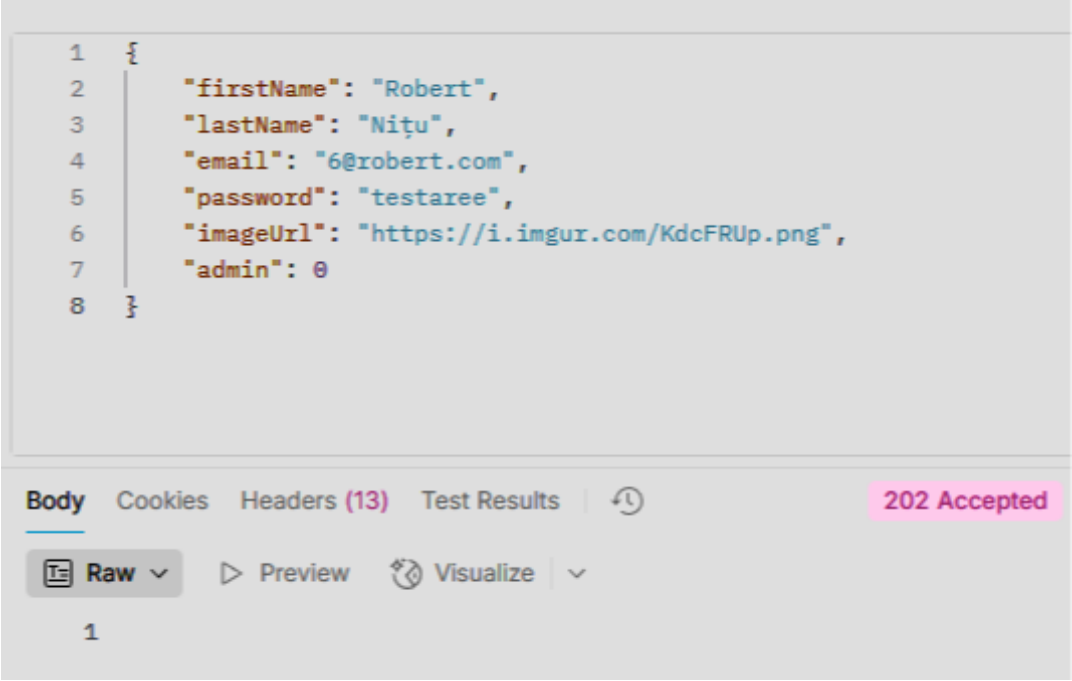


Fig 4. 3 Exemplu correct de folosire a metodei „register”

4.2 Testarea front-end-ului

Pentru testarea aplicației de front-end au fost folosite unelte interne, fără alte aplicații auxiliare. Uneltele folosite au fost prin intermediul browser-ului, anume funcția de „Console”, unde în partea de cod de TypeScript au fost adăugate multiple log-uri pentru a urmări dacă datele sunt stocate și primite în mod correct de pe server-ul de back-end, dar a fost folosit și tab-ul de „Network”, care afișează informațiile sub formă de „JSON”, aceste informații sunt cele care vin de pe server-ul de back-end, iar în acest fel se poate vedea în ce formă se primește, pentru a analiza dacă este o problemă de back-end sau de front-end.

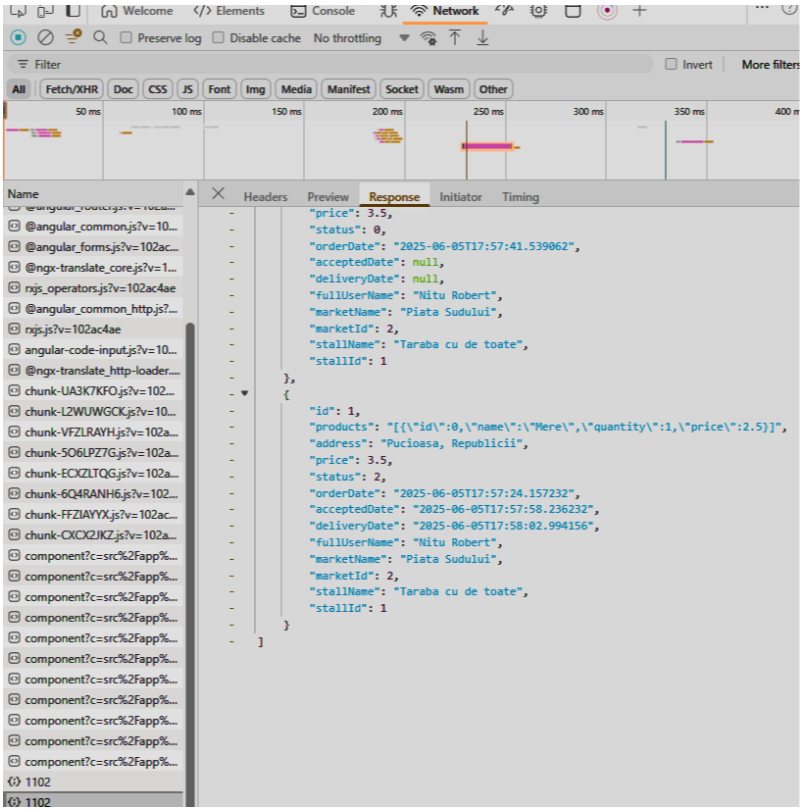


Fig 4. 4 Tab-ul „Network” din browser

5. INTERFAȚA CU UTILIZATORUL

Aplicația permite utilizatorilor să își creeze un cont folosind o adresă de email validă și să se logheze apoi, cu contul creat pe platformă, unde în funcție de rolul atribuit, vor avea acces la vizualizarea mai multor informații și în a le modifica. Aceste funcționalități sunt prezente în figurile 5.1 și 5.2.

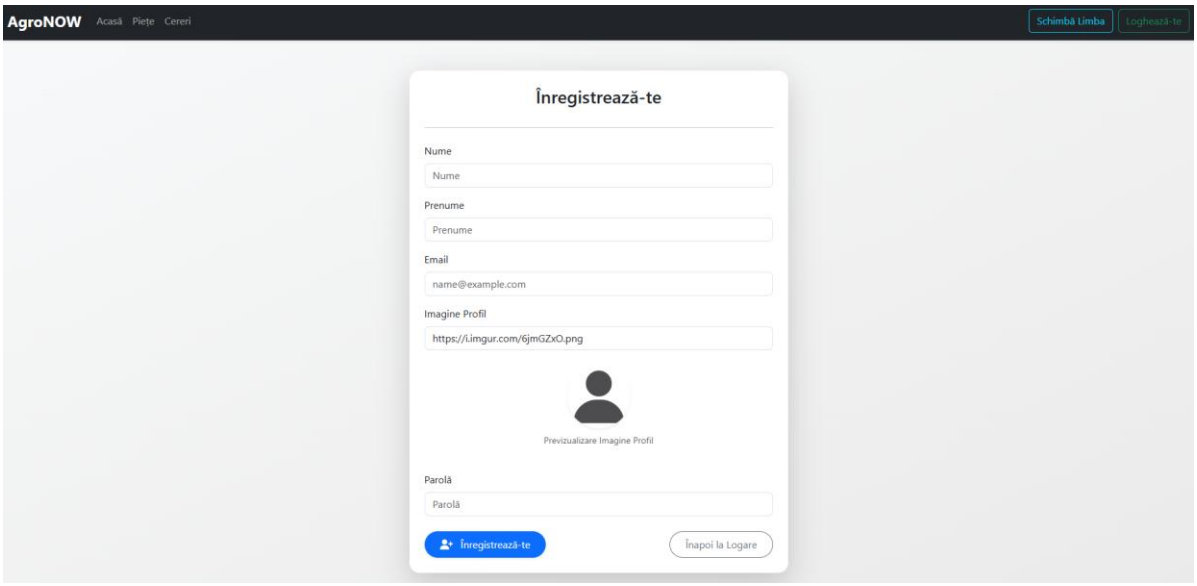


Fig 5. 1 Pagina de înregistrare

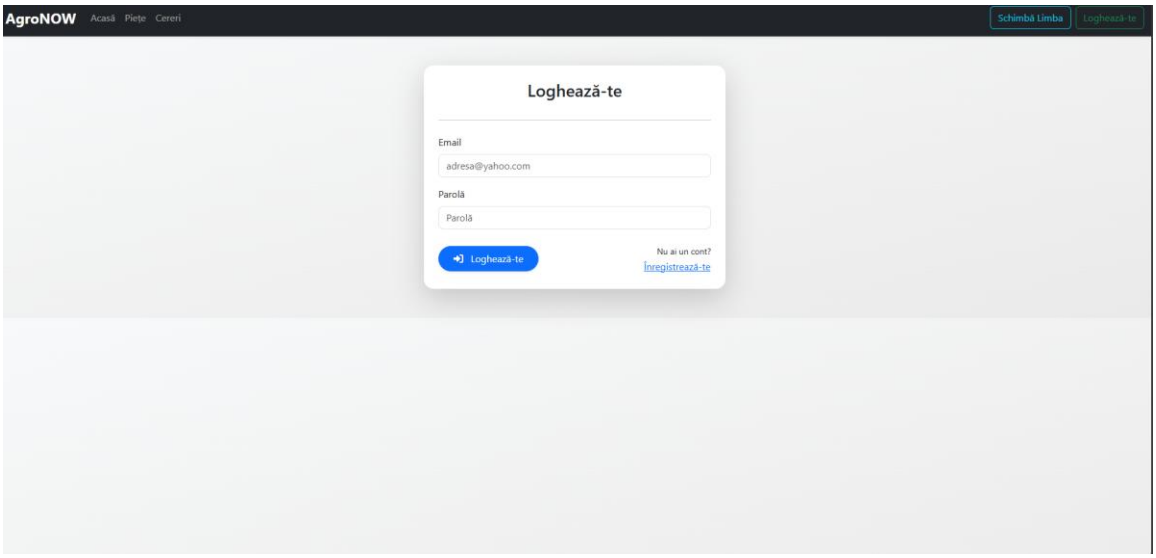


Fig 5. 2 Pagina de logare

Odată logat utilizatorul poate creea cereri pentru anumite roluri, această funcționalitate este prezentată în figura 5.3.

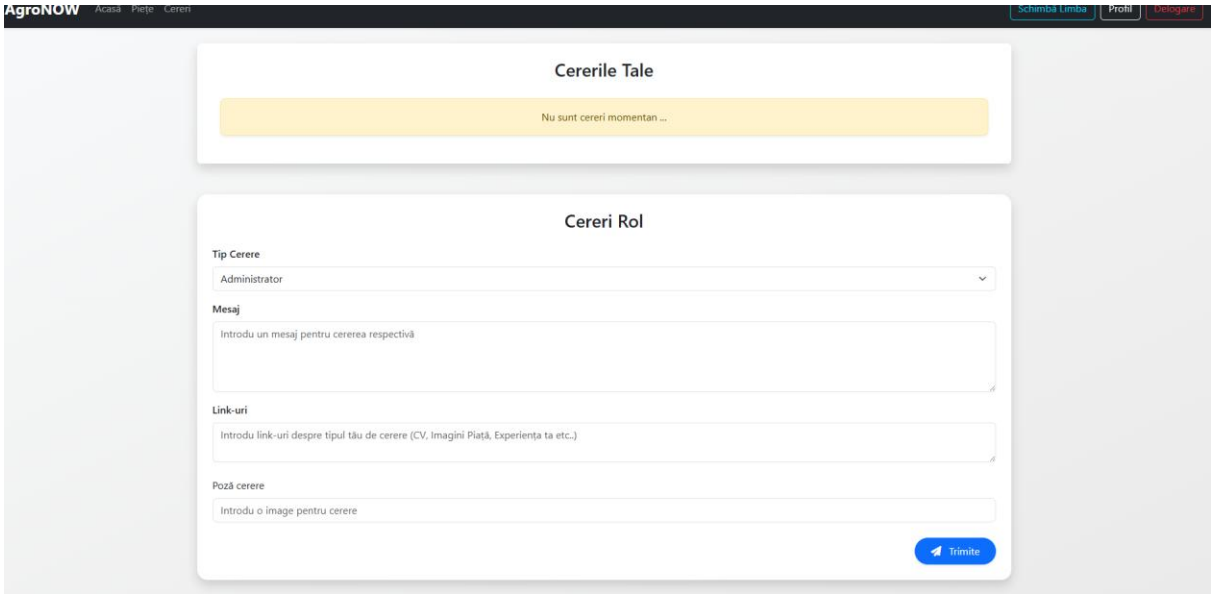


Fig 5. 3 Pagina de cereri

O altă funcționalitate venind cu status-ul de logat al utilizatorului este aceea de a vizualiza profilul, aceasta fiind prezentată în figura 5.4.

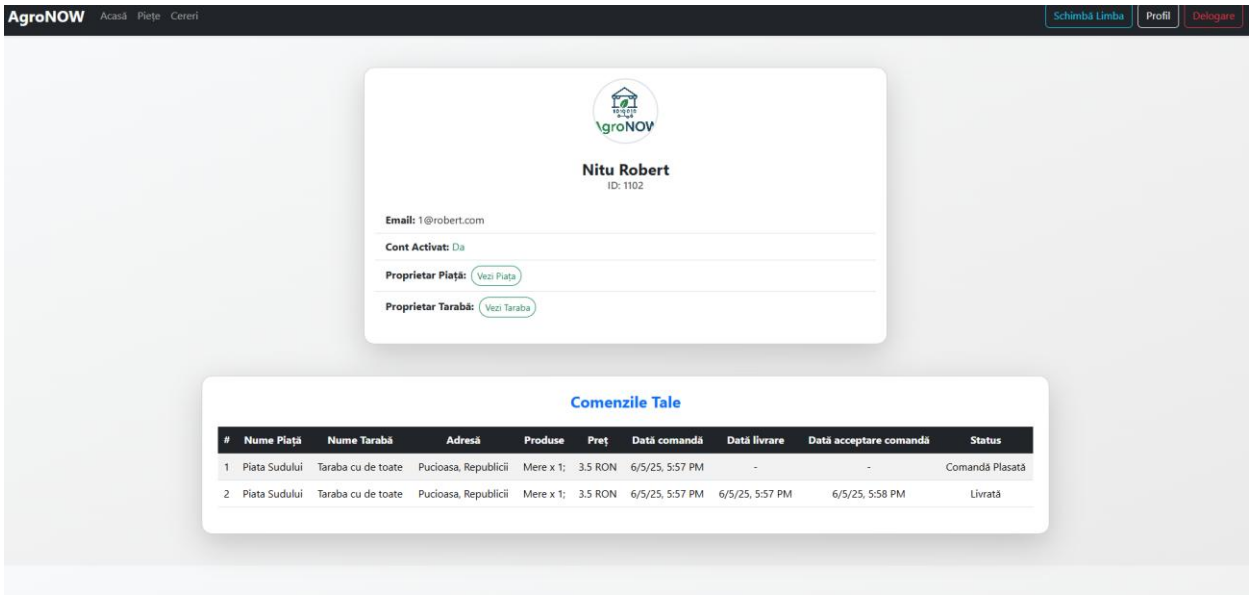


Fig 5. 4 Pagina de profil

Vizualizarea tuturor piețelor înscrise în aplicație se va face accesând ruta de „markets” sau apăsând pe butonul din bara de navigare numit „Piețe”. Această funcționalitate este prezentată în figura 5.5.

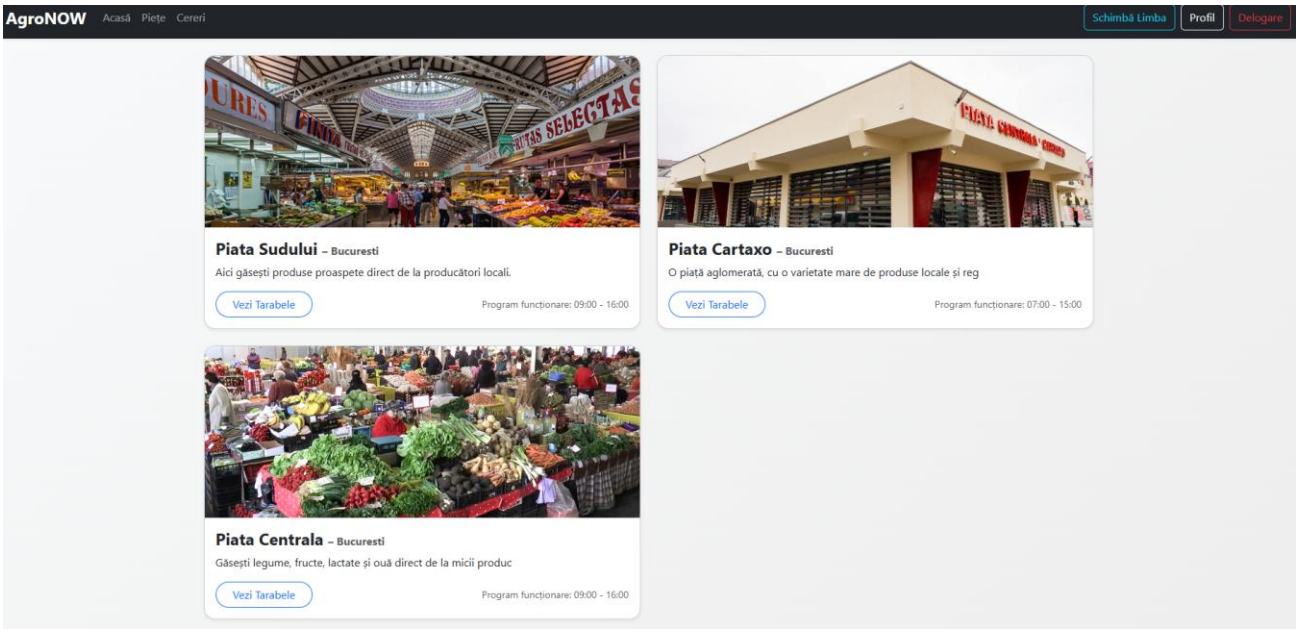


Fig 5. 5 Pagina de piețe

Pentru a vedea toate tarabele din cadrul unei piețe se va apăsa pe butonul de „Vezi Tarabe” care este pus la fiecare dintre piața afișată în pagina anterioară. Apăsând acel buton, aplicația va redirecționa utilizatorul către pagina de vizualizare a tuturor tarabelor din cadrul pieței selectate, conform figurii 5.6.

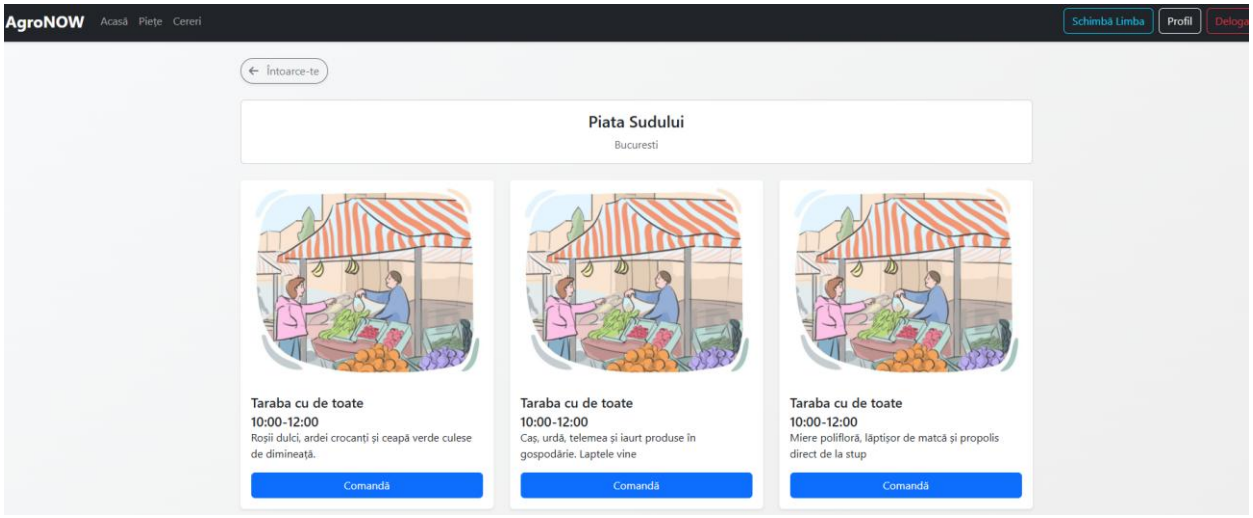


Fig 5. 6 Pagina de tarabe din cadrul unei piețe

Pentru a începe comanda în cadrul unei tarabe, se va apăsa pe butonul de „Comandă”, unde utilizatorul va fi redirecționat spre pagina de vizualiare a produselor din cadrul tarabei selectate, conform figurii 5.7.

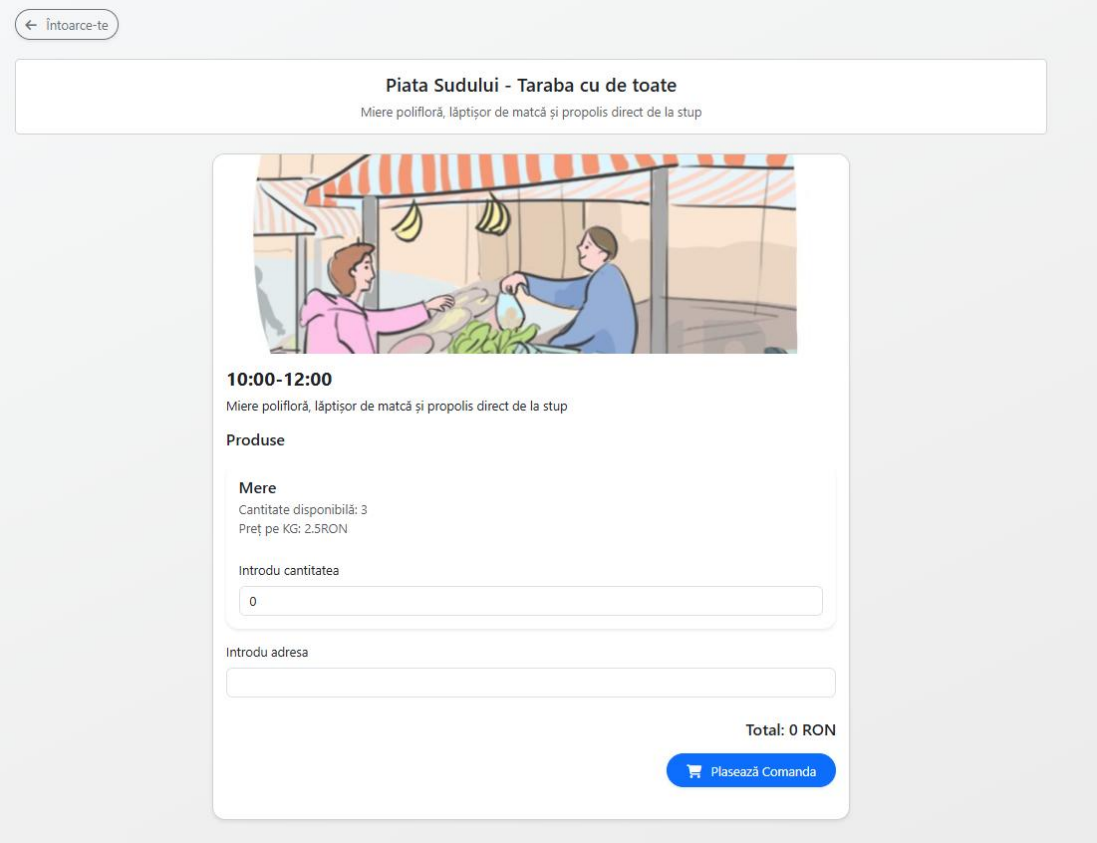


Fig 5. 7 Pagina de comandă din cadrul unei tarabe

5.1 Interfața cu administratorul

Rolul de administrator aduce câteva elemente vizibile în anumite pagini, cum ar fi în pagina de cereri, conform figurii 5.8 și în cea de vizualizare a tuturor piețelor înscrise în aplicate, conform figurii 5.9.

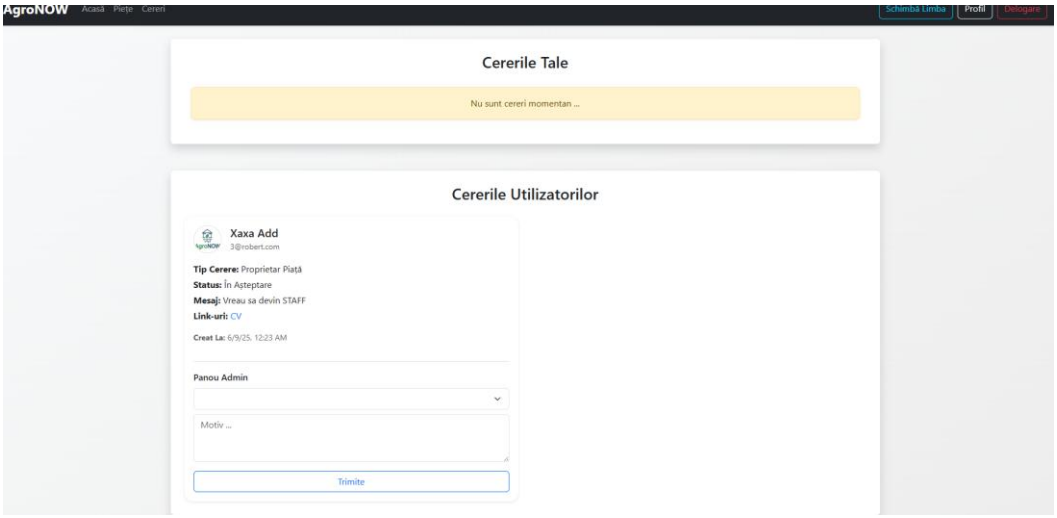


Fig 5. 8 Interfață administrator pentru pagina de cereri

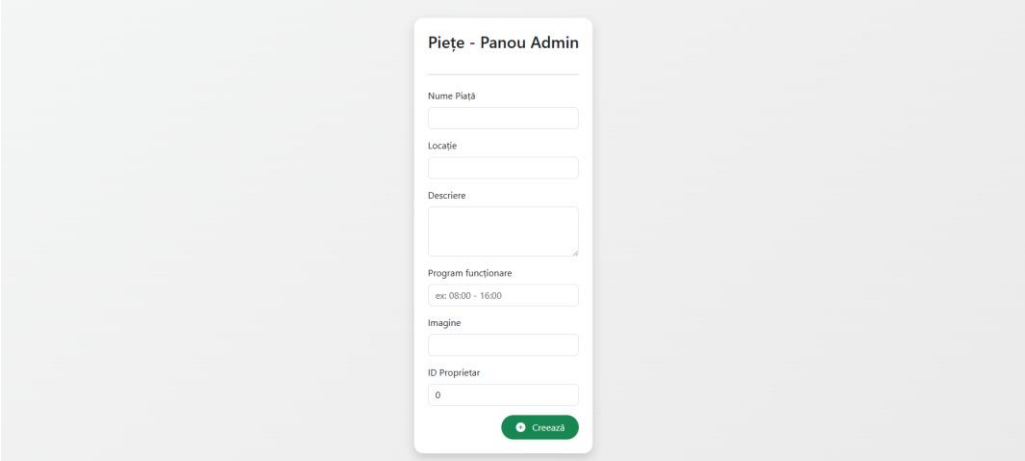


Fig 5. 9 Interfață administrator pentru pagina de piețe

5.2 Interfața cu proprietarul de piață

Proprietarul de piață are drepturi în plus față de un utilizator normal doar dacă accesează pagina cu piața pe care acesta o deține. Dacă este pe pagina cu piața pe care o deține, va avea un meniu unde poate crea tarabe și să le asigneze utilizatorilor care să se ocupe de acestea. Această funcționalitate este prezentată în figura 5.10.

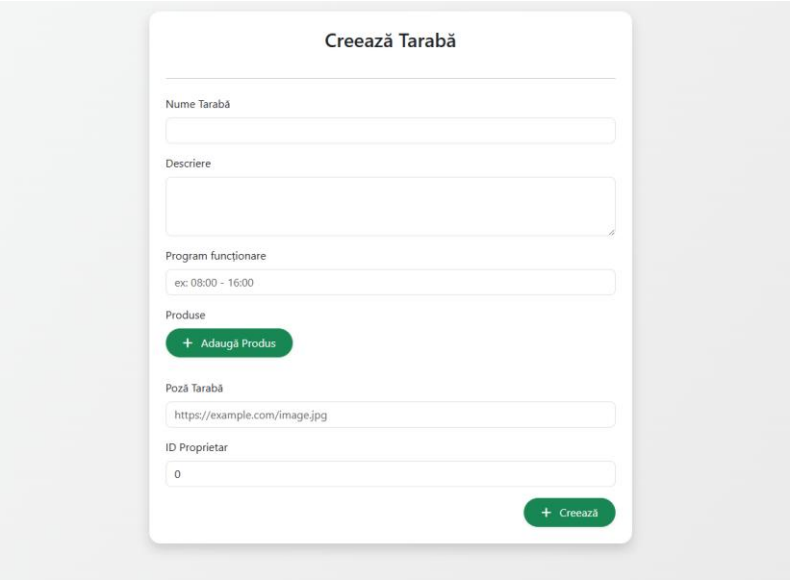


Fig 5. 10 Interfață proprietar piață pentru creare de tarabe

5.3 Interfața cu proprietarul de tarabă

Proprietarul de tarabă are drepturi în plus față de un utilizator normal doar dacă accesează pagina cu taraba pe care acesta o deține. Dacă este pe pagina cu taraba pe care o deține, acesta poate edita stocul de produse curent pentru taraba sa, cât și vizualiza și modifica status-ul comenzilor din cadrul tarabei. Această funcționalitate este prezentată în figura 5.11.

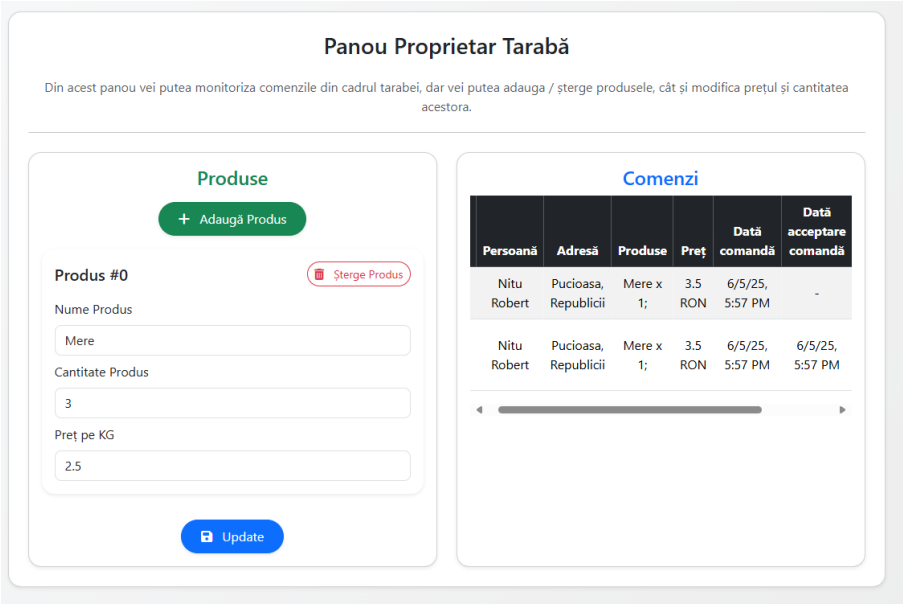


Fig 5. 11 Interfață proprietar tarabă pentru modificare date

6. PROIECTAREA TEHNOLOGICĂ A REPERULUI „FLANȘĂ DE MONTARE” DIN COMPONENȚA ANSAMBLULUI „POMPĂ CU PINIOANE”

6.1 Date inițiale generale

Produs: Flanșă de Montare.

- Unitatea de producție: EMERALD S.A.
- Cerința economică: cost minim de producție.

6.2 Date constructive – funcționale

6.2.1 Schița ansamblului

O schiță a ansamblului „Pompă cu pinioane” se prezintă în figura 6.1, în care produsul considerat „Flanșă de Montare” are poziția 4.

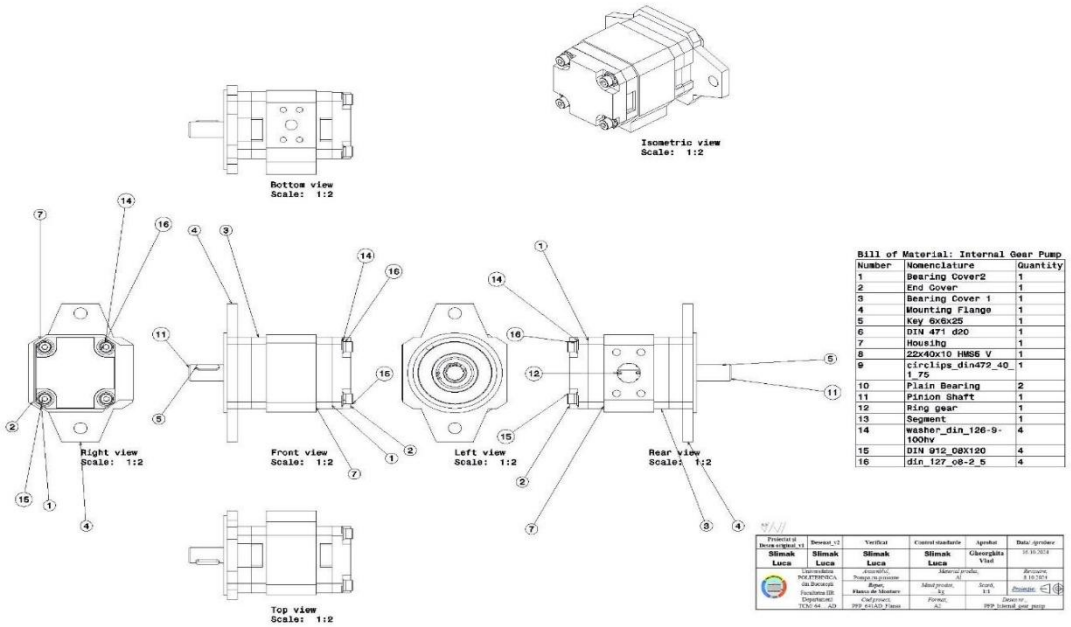


Fig 6. 1 Tabel de componență al ansamblului „Pompă cu pinioane”

6.2.2 Caracteristici constructive prescrise produsului

O schiță a produsului inițial „Pompă cu pinioane” se prezintă în figura 6.2.

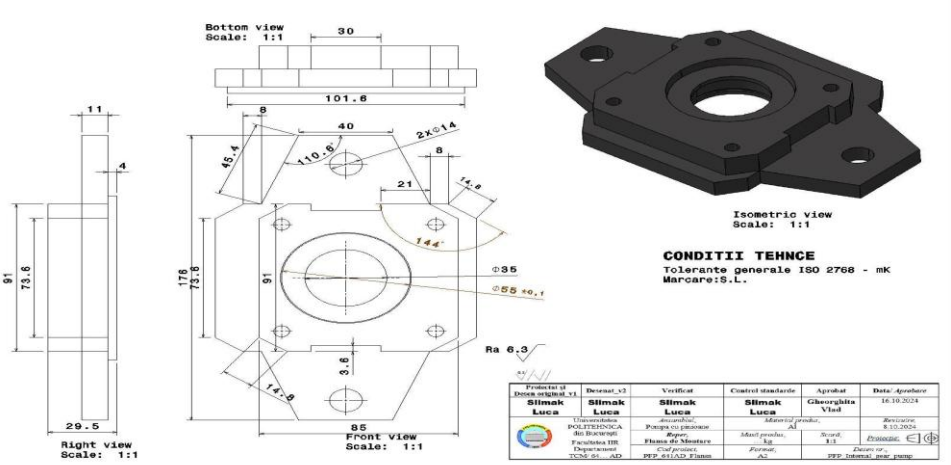


Fig 6. 2 Schița produsului

Caracteristici ale suprafețelor Schița produsului care include suprafețele definitorii Sk, k =1, 2 ..., este prezentată în figura 6.3.

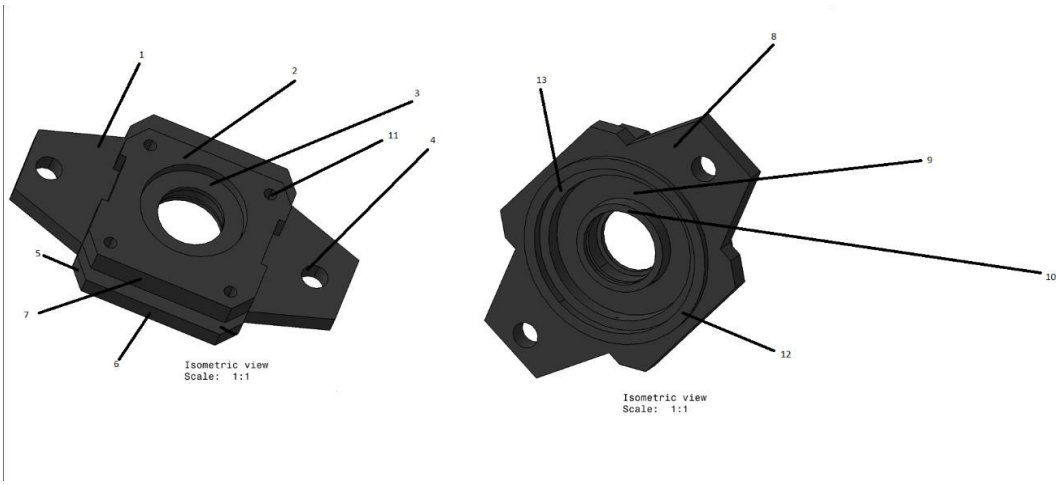


Fig 6. 3 Caractersitici ale suprafețelor

Caracteristicile principale ale suprafețelor Sk se prezintă în tabelul 6.1.

Tabelul 6. 1 Caracteristicile principale ale suprafețelor Sk

Sk	Forma Suprafeței	Dimensiuni principale [mm]	Rugozitatea Ra [µm]	Poziția relativă	Alte condiții
S1	Plană	176x112	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S2	Plană	91x85	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S3	Cilindrică	Ø56 x Ø35	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S4	Cilindrică	Ø14	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S5	Plană	14.82x11	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S6	Plană	73.6xx11	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S7	Plană	73.6x14.5	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S8	Plană	176x112	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S9	Plană	Ø78.25 x Ø49.75	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S10	Cilindrică	Ø48.32 x Ø40.5	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S11	Cilindrică	Ø6.8	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică

S12	Plană	Ø101.6 x Ø90	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică
S13	Plană	Ø90 x 79.65	6.3	-	Toleranțe generale ISO 2768 – mK Brunare chimică

Abaterile limită pentru dimensiunile liniare conform ISO 2768 – mK sunt prezentate în tabelul 6.2.

Tabelul 6. 2

Clasa de toleranță		**Abateri limită pentru dimensiuni nominale [mm]**			
Simbol	Descriere	0.5 – 6	6 – 30	30 - 120	120 - 400
m	Mijlocie	± 0.1	± 0.2	± 0.3	± 0.5

6.2.3 Caracteristici de material

Materialul prescris este A356. Proprietățile fizico – mecanice principale sunt prezentate în tabelul 6.3.

Tabelul 6. 3

Proprietate	**Valoare**
Densitate	2.7 g/cm^3
Rezistență la tracțiune	70 – 700 MPa
Modul de elasticitate	70 GPa
Întindere (deformare)	10 – 20 %
Duritate Brinell	30 – 150 HB
Conductivitate electrică	35.5 x 10^6 S/m
Conductivitate termică	205 W/mK
Temperatura de topire	660 °C
Coeficient de expansiune termică	23.1 x 10^{-6} /°C

Procesul de alegere a materialului produsului este prezentat în tabelul 6.4.

Tabelul 6. 4

Material	Rezistența la rupere (MPa)	Temperatura la topire (°C)	Rezistența la coroziune (notă)	Cost (notă)
A356	250	630	5	5
A 1050-H14	145	657	1	3
A 1050-O	105	660	3	5
A 1060-H112	130	655	5	2
EN-GJS-400-18	290	1250	3	3

Max	290	1250	5	5

Valori normalizate	Rezistența la rupere	Temperatura de topire (°C)	Rezistența la coroziune	Cost
AISI 1045	0.862	0.504	1.000	1.000
AISI 1144	0.500	0.526	0.200	0.600
AISI 4140	0.362	0.528	0.600	1.000
Aisi 304	0.448	0.524	1.000	0.400
PA6	1.000	1.000	0.600	0.600

Factor de importanță	35	30	10	25

Valori normalizate	Rezistența la rupere	Temperatura de topire (°C)	Rezistența la coroziune	Cost	Indice de performanță
AISI 1045	30.172	15.120	10.000	25.000	80.292
AISI 1144	17.500	15.768	2.000	15.000	50.268
AISI 4140	12.672	15.840	6.000	25.000	59.512
AISI 304	15.690	15.720	10.000	10.000	51.410
PA6	35.000	30.000	6.000	15.000	86.000

Material	Indice de performanță
PA6	86.000

Masa produsului Flanșă de montare.

Masa produsului s-a determinat prin utilizarea funcției „Mass” selectată din meniul „properties” din soft-ul Catia V5.

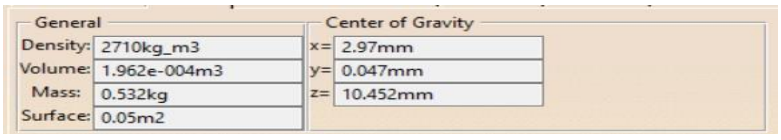


Fig 6. 4 Masa produsului

Astfel, conform figurii 6.4, masa produsului „m” este 0.053kg.

Funcțiile suprafețelor sunt redate în tabelul 6.5.

Tabelul 6. 5 Funcțiile suprafețelor

Suprafața	Funcția
S1	Funcție de asamblare cu mecanismul conjugat
S2	Funcție de delimitare a piesei
S3	Ajută la o realizare ușoară a canelurilor și ușurează asamblarea
S4	Funcție de asamblare
S5	Funcție de fixare
S6	Funcție de fixare
S7	Funcție de fixare
S8	Funcție de asamblare
S9	Funcție de delimitare a piesei
S10	Funcție de delimitare a piesei
S11	Funcție de asamblare
S12	Funcție de delimitare a piesei
S13	Funcție de delimitare a piesei

Gradul de unificare a elementelor constructive, λe, este:

$$\lambda_e = \frac{e_d}{e_t}, \lambda_e \in (0,1] \quad (6.1)$$

Unde: e_d este numărul dimensiunilor diferite, iar e_t este numărul total al elementelor.

Se analizează o serie de caracteristici geometrice, după cum urmează:

- Găuri cilindrice: $\lambda_{e1} = \frac{e_d}{e_t} = \frac{2}{6} = 0.66$ (6.2)
- Raze de racordare: R2, $\lambda_{e3} = \frac{e_d}{e_t} = \frac{2}{3} = 0.67$ (6.3)

Astfel, gradul mediu de unificare constructivă λ_e este:

$$\lambda_e = \frac{1}{q} \sum_{i=1}^{q} \lambda_{ei} = \frac{1}{2} (1 + 1) = 1 \quad (6.4)$$

Condiții de tehnolocitate impuse de unificarea constructivă.

Din punct de edere al condiției principale impuse de unificarea constructivă, respectiv număr minim posibil al dimensiunilor elementelor constructive (v. λ_e), se apreciază că aceasta este accptabilă.

Condiții de tehnologictate impuse de procedeele tehnologice.

Condițiile de tehnologitate impuse de procedeele tehnologice de fabricare sunt prezentate în tabelul 6.6.

Tabelul 6. 6

Nr. Crt.	**Condiție**	**Grad de satisfacere:** Da sau Nu
1	Forma de gabarit să prezinte axe sau plane de simetrie care, după caz, să fie utile pentru definirea planului(-elor) de separație a semimatrițelor.	Da
2	Elementele constructive de tip înclinare, racordare, grosime de perete, de dimensiune l, să îndeplinească condiția $l \geq l_{min}$ sau/și $l \leq l_{max}$, în care l_{min} și l_{max} sunt valori limită impuse de procedeul tehnologic.	Da
3	Forma sau poziția unor suprafețe să fie astfel încât să prezinte „înclinări” în raport cu planul de separație a semimatrițelor.	Da
4	Trecerile dintre suprafețe să fie astfel încât să evite deteriorarea acestora în timpul manevrării semifabricatului.	Da
5	Trecerile dintre anumite suprafețe să fie sau nu sub formă de degajare / canal sau teșitură în funcție de cerințele funcționale și/sau de cerințele impuse de procedeul de prelucrare.	Da
6	Forma și poziția bosajelor, găurilor sau locașurilor să fie astfel încât să conducă la un număr minim de poziții ale piesei/sculei în timpul operației, acces ușor al sculelor în zona de lucru.	Da
7	Suprafețele frontale ale găurilor să fie plane și perpendiculare pe axele găurillor.	Da
8	Forma și poziția suprafețelor să fie astfel încât să permită prinderi simple și sigure în cadrul operaților.	Da

6.2.4 Interdependența toleranțelor dimensionale

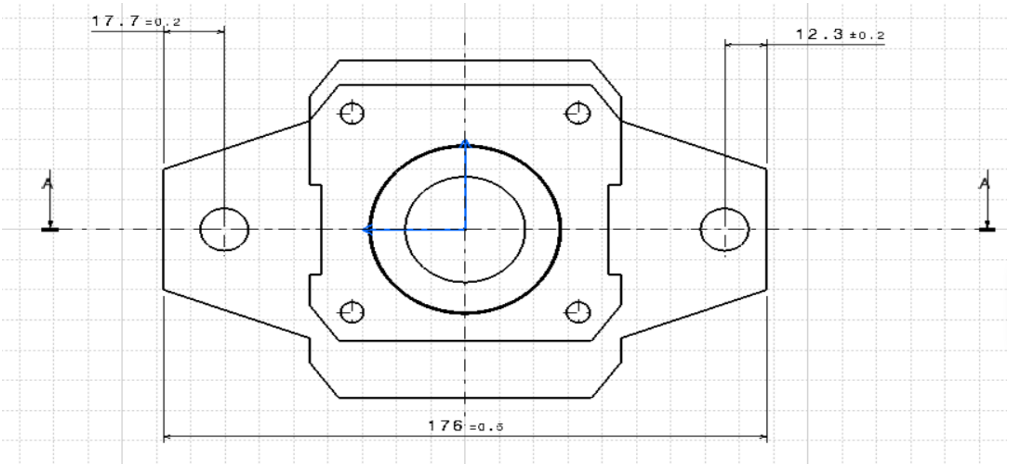


Fig 6. 5

$$J_{MAX} = \sum(Dimensiuni\ măritoare)_{MAX} - \sum(Dimensiuni\ \ reducătoare)_{MIN} = 146.9\ mm \quad (6.5)$$

$$J_{MIN} = \sum(Dimensiuni\ măritoare)_{MIN} - \sum(Dimensiuni\ \ reducătoare)_{MAX} = 145.1\ mm \quad (6.6)$$

$$TOL_J = J_{MAX} - J_{MIN} = 146.9 - 145.1 = 1.8\ mm \quad (6.7)$$

6.2.5 Analiza reperului

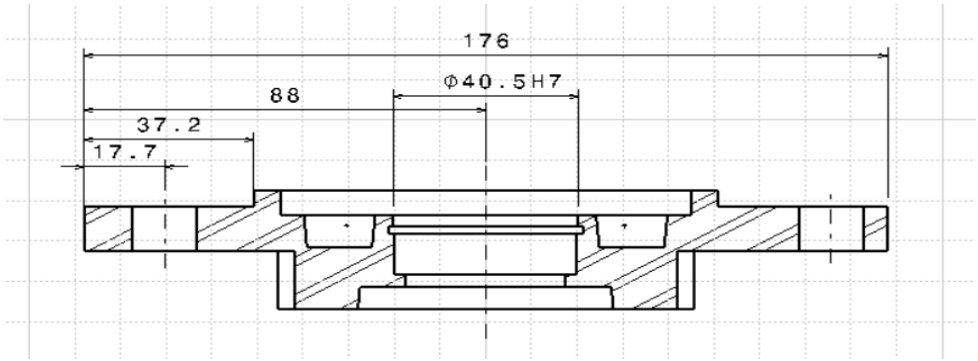


Fig 6. 6 Analiza reperului

Analiza procesului d e fabricație pentru reperul „Suport Mulinetă” este reprezentat prin desenul de execuție din figura 6.7.

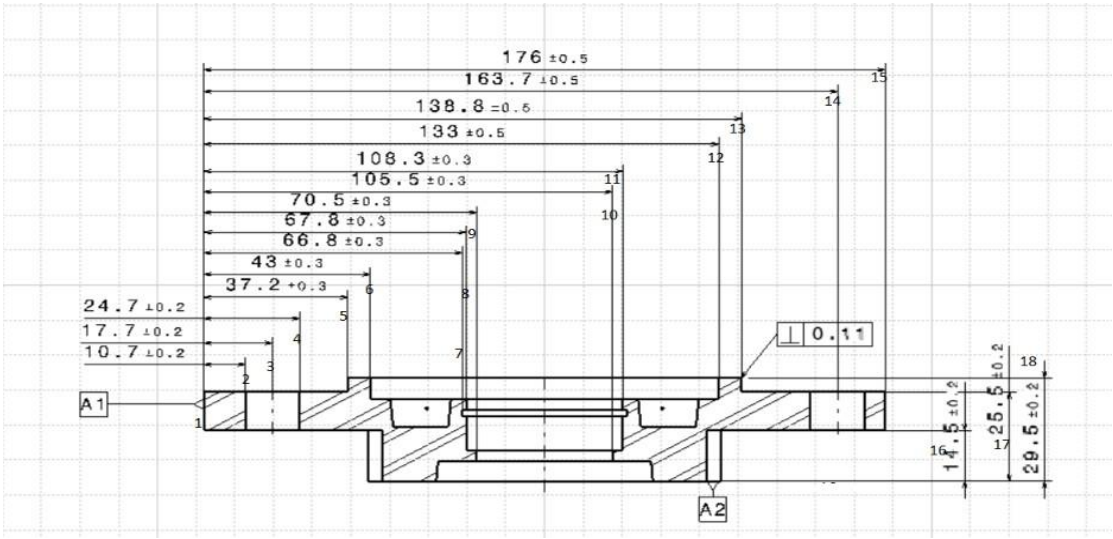
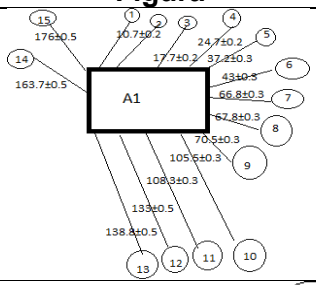
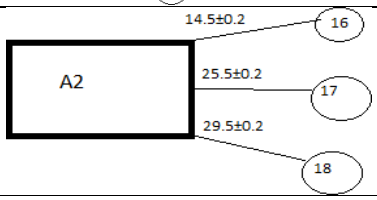
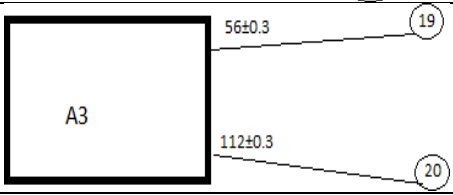


Fig 6. 7

Analiza constrângerilor de anterioritate:

Axă	Figură
OX	
OZ	
OY	

6.3 Proiectarea semifabricatului

Date inițiale:

- Materialul piesei: A356.
- Caractersticile piesei sunt conform tabelului 6.1.

Metoda de semifabricare este: Turnare. Având în vedere materialul impus realizării piesei, metoda de semifabricare este turnarea. Turnarea este procedeul de semifabricare a unei piese prin solidifcarea unei cantități determinate de metal lichid, introdus într-o cavitate de configurație corespunzătoare.

Procedeul: Procedeele de turnare se clasifică în funcție de numărul de piese obținute în forma de turnare. Numărul ridicat de piese de realizat duce la procedeul de turnare în forme permanente. Pe această bază au fost anallizate două variante ethnic acceptabile care se prezintă în tabelul 6.7.

Tabelul 6. 7

Varianta	Tip semifabricat	Metoda de semifabricare	Procedeu de semifabricare
1	cu adaos mic	Turnare	Turnare în forme permanente
2	cu adaos mare	Turnare	Turnare în forme permanente

După turnare piesa „Flanșă de montare” va arăta ca în figura 6.8.

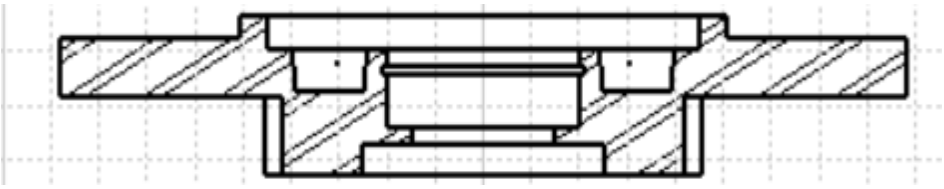


Fig 6. 8

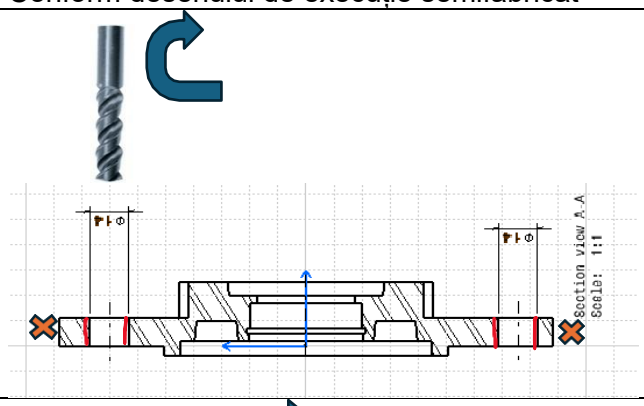
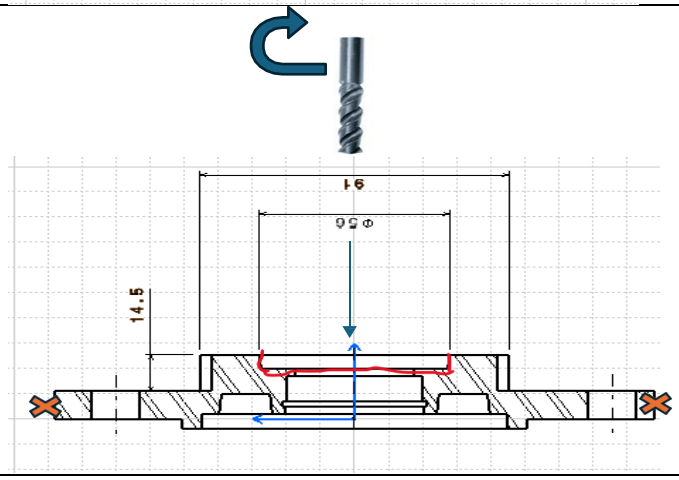
Pentru a ajunge la forma finală, sunt necesare următoarele procese atribuite celor 4 suprafețe, conform tabelului 6.8.

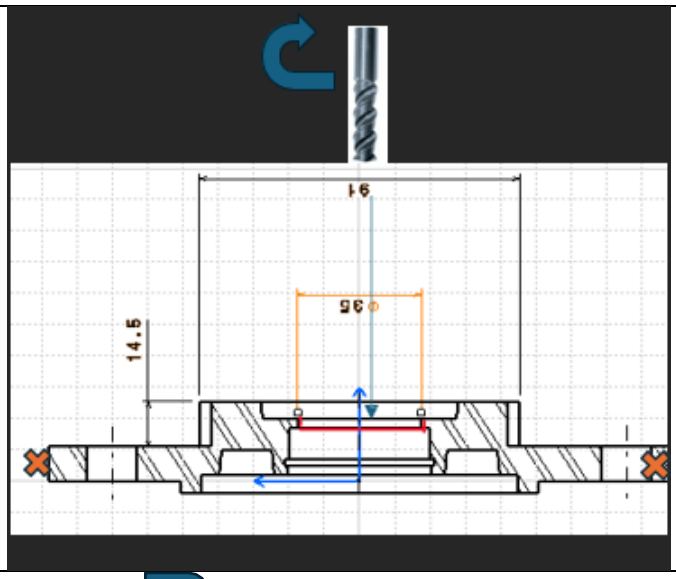
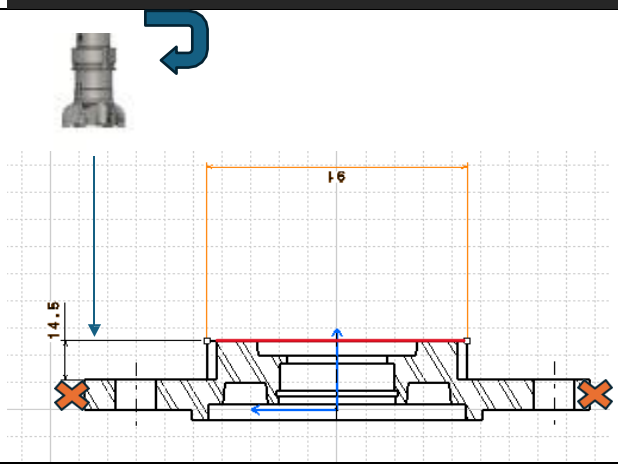
Tabelul 6. 8

Sk	Prelucrări	1	2
S1	Găurire	Găurire IT10 Ra 6.3	Filetare cu tarot 14mm x 2mm IT10 Ra 6.3

S2	Frezare de degrosare	Frezare de degroșare IT12 Ra 6.3	-
S3	Frezare de degrosare	Frezare de degroșare IT12 Ra 6.3	-
S4	Frezare plană	Frezare plană IT12 Ra 6.3	-

Tabelul 6. 9

Nr. de ordine și denumirea operației	Schița simplificată a operației	Schița simplificată a operației
0.0 Turnare	Conform desenului de execuție semifabricat	
1. Găurire		U: Mașină de găuri S: Burghiu-13mm V: Șubler, Calibru
2. Frezare		U: Mașină de frezat S: Freza-20mm V: Șubler

3. Frezare de degrosare		U: Mașină de frezat S: Freza-20mm V: Șubler
4. Frezare plană		U: Mașină de frezat S: Freza-63mm V: Șubler

Masa inițială după turnare conform figurii 6.9.

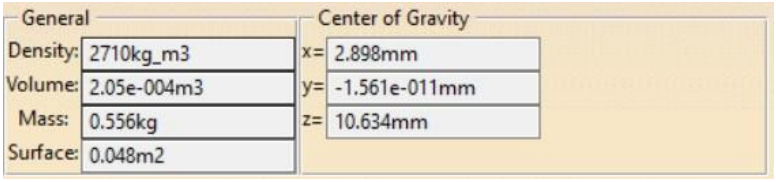


Fig 6. 9

După prelucrări parametrii se modifică conform figurii 6.10.

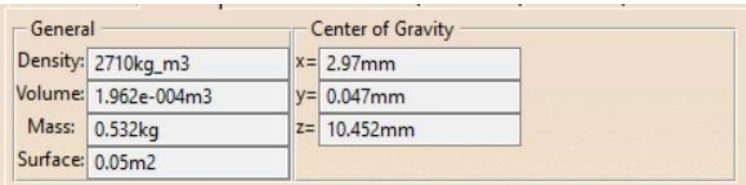


Fig 6. 10

Masa reperului înainte de prelucrare este de 0.556kg, iar masa de după prelucrări este 0.532kg. Pierderea de material în urma proceslor de prelucrare este de 0.024kg.

Prelucrarea	Treapta de precizie	R_z, μm	S, μm
	I. STRUNJIRE , FREZARE , RABOTARE		
	Turnare în forme de amestec de formare		
	a. formare mecanică cu modele metalice		
Intr-o singură fază	11...12	32	30
Degroşare	12	50	50
	b. formare mecanică cu modele din lemn		
Degroşare	14	100	100
Semifinisare	12	50	
	c. formare manuală cu modele din lemn		
Cojire	16...17	320	300
Degroşare	14...15	250	240
Semifinisare	11...12	100	100
	d. pentru toate metodele de formare		
Finisare	10...11	25	25
Netezire	7...9	5	5
	Turnare în cochilă şi turnare centrifugală		
Intr-o singură fază	11		
Degroşare	12	50	50
Finisare	10		
Netezire	7...9	5	5
	Turnare în forme coji		
Intr-o singură fază	10...11	25	25
Degroşare	11	20	20
Finisare	10	10	10
Netezire	7...9	5	5
	Turnare cu modele uşor fuzibile		
Intr-o singură fază	10	16	20
Netezire	7...9	2,5	5
	II.RECTIFICAREA SEMIFABRICATELOR TURNATE PRIN DIFERITE METODE		
Intr-o singură fază	7	5	10
Degroşare	8...9	10	20
Finisare	6...8	5	15
Netezire	5...6	0,8	-

Fig 6. 11 Adaos de prelucrare

Pentru determinarea adaosurilor de prelucrare se folosesc două metode: o metodă de calcul analitic, iar o metodă de calcul experimental statistică. Metoda experimental statistică stabilește adaosurile de prelucrare cu ajutorul unor standarde, normative sau tabele de adaosuri. Metoda de calcul analitic se bazează pe analiza tuturor factorilor care determină mărimea adaosului de prelucrare în condițiile concrete de efectuare a succesiunii prelucrărilor cu prinderile aferente. Se reduce consumul specific de material.

Adaosul de prelucrare intermediar minim se calculează cu relațiile următoare:

$$2A_{min} = 2\left(R_{Z_{i-1}} + S_{i-1}\right) + \sqrt[2]{\rho_{i-1}^2 + \epsilon_i^2}\ [\mu m] \quad (6.8)$$

$$2A_{min} = 2 * 100 = 200\ \mu m$$

Tabelul 6. 10

	Tipul prelucrării necesare	Treapta de precizie obținută	$2A_{1min}$
1	Frezare de degrosare	IT12(T1 = 0.3mm)	$2A_{1min} = 200\ \mu m$

Tabelul 6. 11

Prelucrare I / Metoda de reglare	Reglarea individuală
Frezare de degrosare (T1=IT12, $2A_{min} = 200\ \mu m$)	Dimensiunea prescrisă pentru frezarea de degrosare Ø55 (±0.3) T0 = 1.7mm T1 = 0.3mm Ai = 0 As = 0.3mm $2A1_{nom} = 2\ A1_{min} + T0 = 2.1mm$ $2A1_{max} = 2.2\ mm$ $D0_{min} = 55 - 2.1 = 52.9\ mm$ $D0_{max} = 53 + 1.7 = 54.7\ mm$

6.4 Mașini unelte folosite

Utilaje și SDV-uri, metodele și procedeele de reglare la dimensiune.

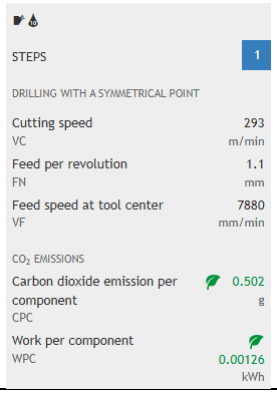
Tabelul 6. 12

Nr. și denumirea operației	Tip utilaj	Imagine utilaj	Caracteristici tehnice

1. Frezare fețe și canale	Mașina de frezat CNC Haas VF-3		Putere: 22.4 kW Turații ax: 50-8100 rpm Precizie repetitivă: ±0.015mm Cuplu maxim: 122 N @ 450 rpm
2. Găurire	Mașina de găurit cu stand și coloana KSBM 5/50		Dimensiuni masa: 560 x 480mm Putere motor: 4000 W Greutate: 1250 kg Turație ax: 45-850 rpm Trepte de viteză: 9 Cursă cap găurire: 200mm Cursa masă găurire: 315mm

6.5 Sculele necesare

Pentru fiecare operație în parte s-au determinat sculele necesare cu ajutorul Sandvik.

Faza prelucrării	Scula	Denumire	Caracteristici
Găurire	 	Burghiu metal HSS-CNC twist drill 124	Diametru de găurire: 13mm Mterial: Oțel rapid (HSS – High-Speec-Steel) Unghi la vârd: 124°. Lungime totală: 150mm Lungime de tăiere: 90-100mm.
Filetare		Tivoly 960ST21 – Tarod cu filet metric,	Clasa de utiliare: profesională

	 STEPS 1 THREAD FORMING WITH TAP Cutting speed VC 83.4 m/min Feed per revolution FN 2 mm CO_2 EMISSIONS Carbon dioxide emission per component CPC 4.65 g Work per component WPC 0.0116 kWh	25x110 mm, M14, pas normal, HSS-CO	Diametru filet: M14 Lunime totală: 110mm
Frezare	 STEPS 1 HELICAL MILLING Cutting speed VC 1100 m/min Feed per tooth FZ 0.366 mm CO_2 EMISSIONS Carbon dioxide emission per component CPC 3.87 g Work per component WPC 0.00968 kWh	Freza cu 4 taisuri, Carbide 45HRC	Lungime de lucru: 55mm Lungime: 100mm Diametru prindere: 20mm Material: Carbura

Frezare	 STEPS 1 PREMACHINING Cutting speed 900 VC m/min Feed per tooth 0.308 FZ mm CO_2 EMISSIONS Carbon dioxide emission per component 3.35 CPC g Work per component 0.00837 WPC kWh	Freza fronală cu plăcuțe vidia amovibile ADKT 1505 la 90°	Diametru: 63mm Tip plăcuță: ADKT 1505 Nr. Pastile: 6 Unghi pastile: 6

6.6 Program Fusion 360

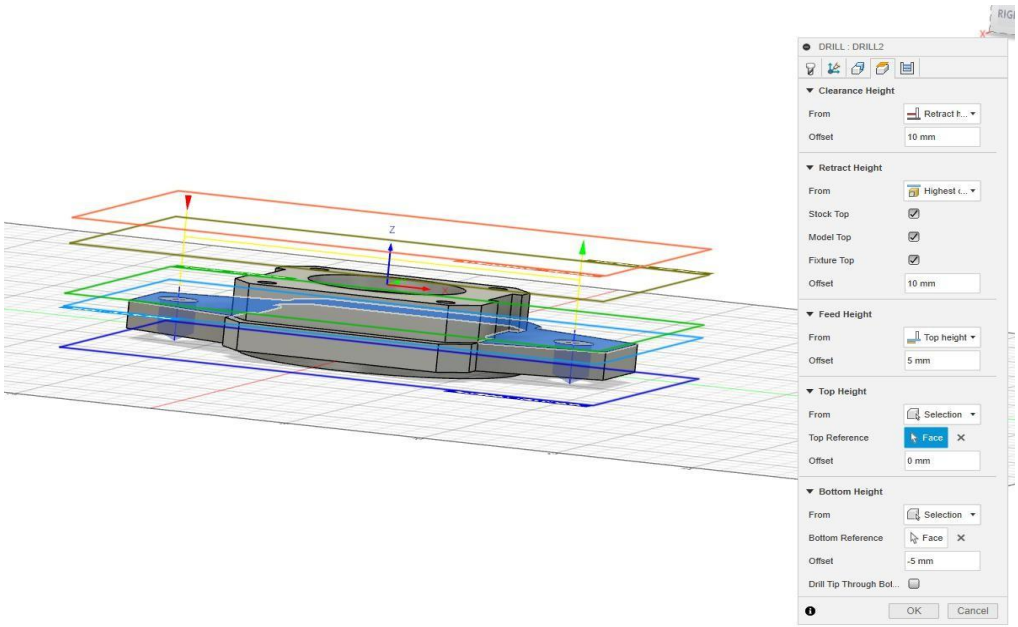


Fig 6. 12 Găurire

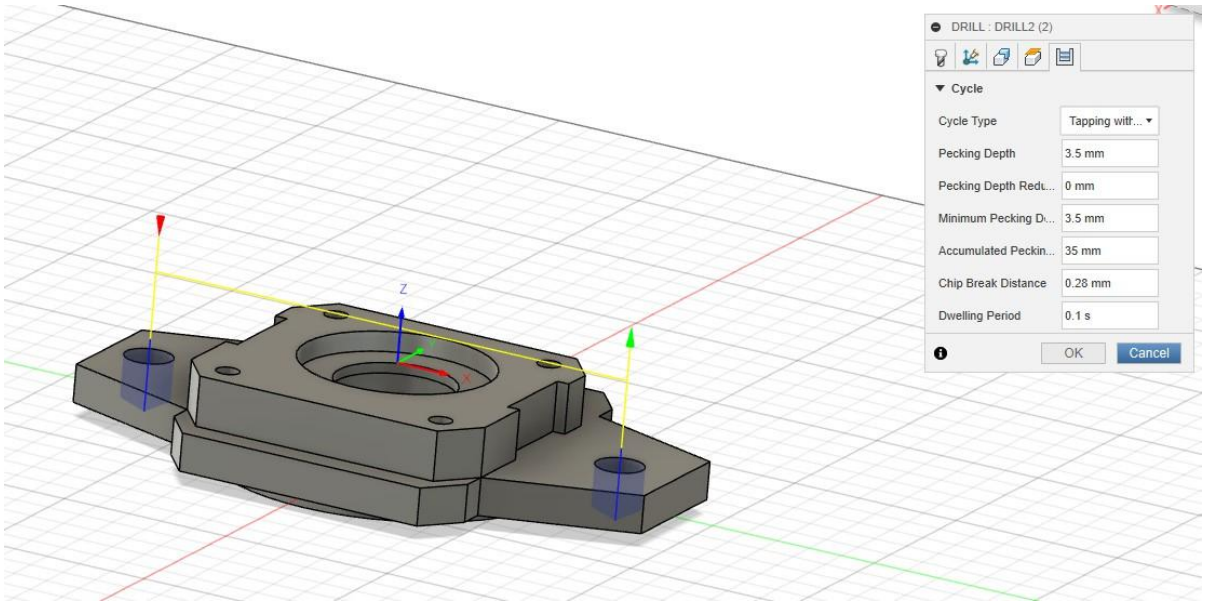


Fig 6. 13 Filetare

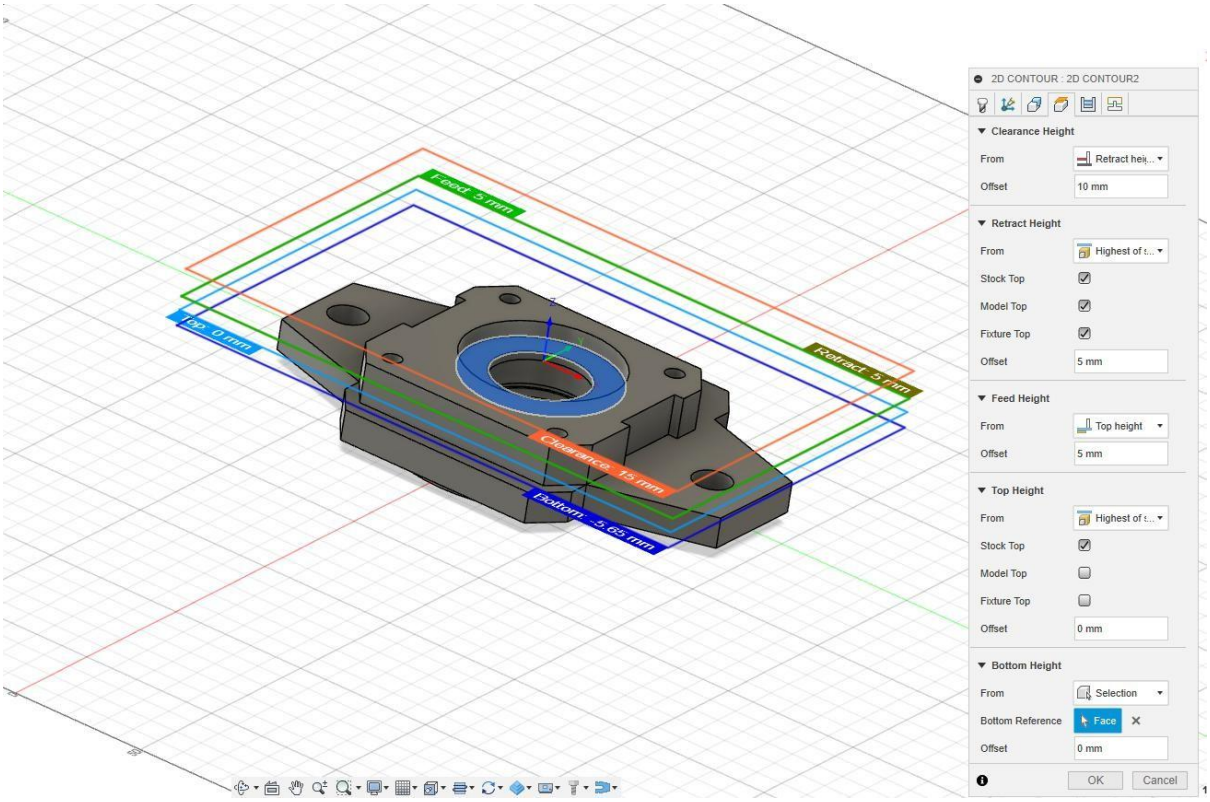


Fig 6. 14 Frezare de degrosare

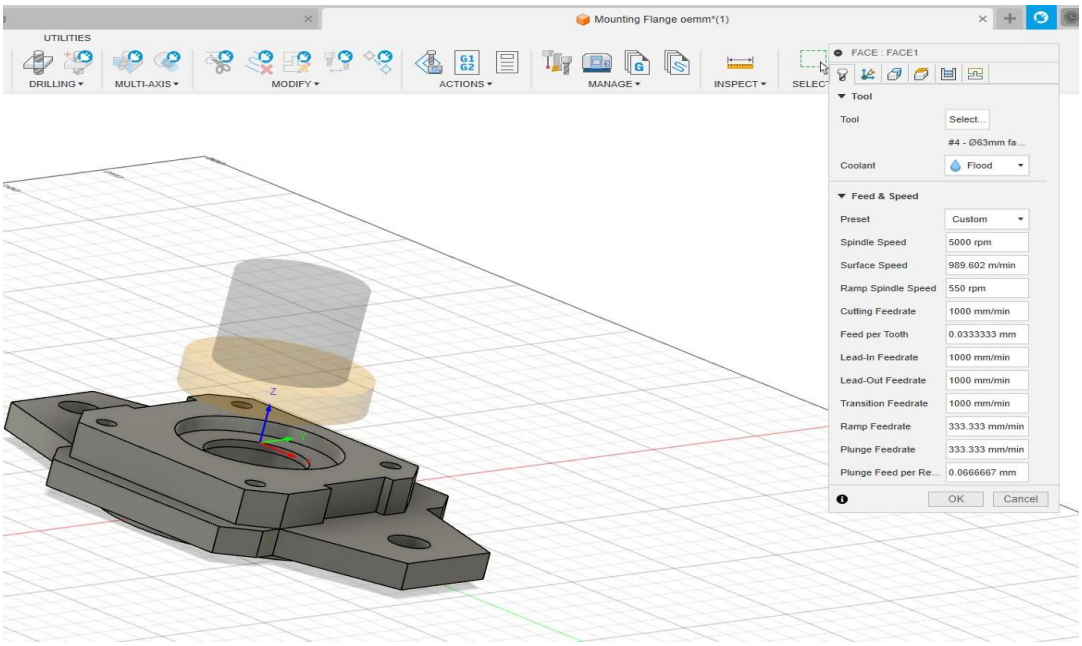


Fig 6. 15 Frezare plană

6.7 Fabricare reper prin fabricare aditivă

Am folosit soft-ul Z-suite pentru fabricare aditivă a reperului „Flanșa de montare”. Am ales imprimanta Zortrax M300 Plus cu o duză de 0.4mm și grosimea stratului de 0.1mm, conform figurii 6.19.

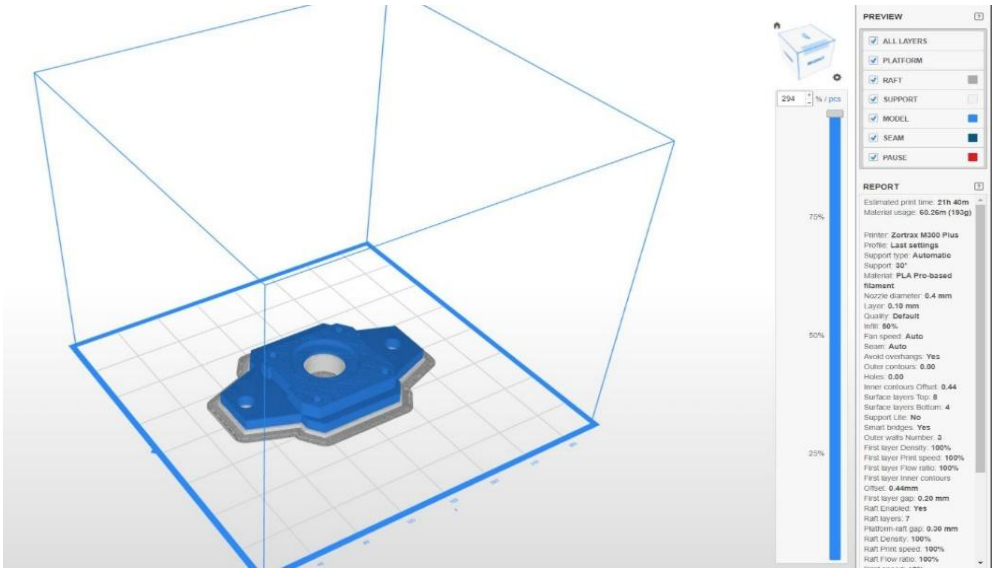


Fig 6. 16

Date suplimentare:

Tip support: automatic.

Material: PLA

Timp estimate de printare (h/m): 21h 40m.

Material folosit: 60.26m (193g).

7. CONCLUZII ȘI DEZVOLTĂRI ULTERIOARE

7.1 Concluzii

Coroborând informațiile menționate, această lucrare a prezentat modalitatea în care a fost realizată o aplicație pentru digitalizarea unor activități din piețele agroalimentare, care este utilă la înregistrarea piețelor în cadrul aplicației, adăugarea tarabelor din piața anterior creată, dar și plasarea de comenzi de produse dintr-o tarabă anume. Aplicația a fost realizată cu tehnologii de ultimă generație, cum ar fi Java Spring Boot pentru server-ul de back-end, Angular19 pentru partea de front-end a aplicației.

Proiectul a avut ca obiectiv realizarea unei aplicații cu interfață „User-Friendly” pentru utilizatori, proprietari de piață și proprietari de tarabă. Există posibilitatea creării de conturi și logare cu adresa de email, dar și gestionarea cererilor de roluri. Aceste funcționalități oferă control asupra proiectului, îmbunătățește relația dintre comerciant în cadrul pieței și utilizatorul normal și crește expunerea piețelor pe plan național.

Tehnologiile au fost alese astfel încât să fie ușor integrabile în aplicație, compatibile între ele și să faciliteze modificările ulterioare pentru a satisface nevoile viitoare ale utilizatorilor. Acestea au fost folositoare la modelarea experienței utilizatorului, oferind platformei estetică, securitate, scalabilitate și fluiditate.

În concluzie, lucrarea a prezentat realizarea cu succes a unei platforme destinate întregii comunități, care își propune să readucă piețele agroalimentare în ochii comunității prin înscrierea tuturor piețelor și a tarabelor din cadrul acestora, plasarea de comenzi în timp real. Acest obiectiv a fost atins folsoind limbaje de programare pentru interfață și funcționalitate, un sistem de baze de date ca PostgreSQL, instrumente ajutătoare pentru aspect, precum Bootstrap, funcționalități inedite precum suportul multi-lingvistic, cu ajutoul NGX-Translate.

7.2 Dezvoltări ulterioare

Aplicația poate fi dezvoltată spre o variantă îmbunătățită atât din punct de vedere al aspectului, cât și a experienței de vizualizare a produselor, cât și a piețelor înscrise în aplicație. Un lucru important ce poate fi dezvoltat ulterior, este acela de afișare a piețelor bazată pe geo-locația utilizatorului, oferind o experiență unică și afisând în prim plan doar piețelor care sunt în promitatea acestuia.

Pe partea de extindere și publicare, această aplicație va fi prezentată în cadrul unei sedințe alături de firma EvaSTAFF, unde vor fi prezenți decidenți ai administrației locale în domeniul piețelor agroalimentare. În urma discuției voi colecta feedback-ul acestora, voi afla interesul lor față de înscrierea piețelor în aplicație și de ce funcționalități au nevoie în plus față de prezentarea actuală.

Aceste lucruri vor fi vitale pentru viitorul aplicației. Aplicația de digitalizare a unor activități din piețele agroalimentare este construită pe un schelete solid, iar adăugirile de funcționalități și modificarea aspectului se face imediat, fără a modifica arhitectura aplicației.

8. BIBLIOGRAFIE

[1] Site oficial piataoborbucuresti.ro: https://piataoborbucuresti.ro/ro ; accesat 10.01.2025

[2] Site oficial pietecraiova.ro/piata-centrala: https://www.pietecraiova.ro/piata-centrala ; accesat 15.01.2025

[3] Site oficial documentație Java, https://en//wikibooks.org/wiki/Java_Programming ; accesat 20.02.2025

[4] Descriere Spring Boot, https://spring.io/projects/spring-boot ; accesat 20.02.2025

[5] Site oficial documentație Spring Boot, https://www.geekforgeeks.org/spring-boot/ ; accesat 20.02.2025

[6] Site oficial documentație Spring Data JPA, https://spring.io/projects/spring-data-jpa ; accesat 20.02.2025

[7] Site oficial documentație HTML, https://developer.mozilla.org/en-US/docs/Web/HTML ; accesat 10.03.2025

[8] Introducere HTML, https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Structuring_content ; accesat 10.03.2025

[9] Site oficial documentație CSS, https://developer.mozilla.org/en-US/docs/Learn/CSS/First_steps/What_is_CSS ; accesat 18.03.2025

[10] Introducere Boostrap, https://getbootstrap.com/docs/5.3/getting-started/introduction/ ; accesat 19.03.2025

[11] Site oficial documentație TypeScript, https://developer.mozilla.org/en-US/docs/Glossary/TypeScript ; accesat 20.03.2025

[12] Site oficial documentație JavaScript, https://developer.mozilla.org/en-US/docs/Web/javascript ; accesat 20.03.2025

[13] Site oficial documentație PostgreSQL, https://www.postgresql.org/about/ ; accesat 27.03.2025

[14] Istoric PostgreSQL, https://www.postgresql.org/docs/current/history.html ; accesat 28.03.2025

[15] Site oficial documentație Angular, https://angular.dev ; accesat 01.04.2025

[16] Site oficial documentație OpenAPI, https://learn.openapis.org/introduction.html ; accesat 02.04.2025

[17] Articol științific NGX-Translate, 2 Septembrie 2024, Andreas Low, https://www.codeandweb.com/babeledit/tutorials/how-to-translate-your-angular-app-with-ngx-translate ; accesat 20.04.2025

[18] Site oficial documentație JSON, https://www.json.org/json-en.html ; accesat 21.02.2025

[19] Site oficial documentație MailDev, https://maildev.github.io/maildev/ ; accesat 25.03.2025

[20] Site oficial documentație Docker, https://www.docker.com ; accesat 25.03.2025

[21] Introducere Docker, https://www.docker-curriculum.com ; accesat 26.03.2025